

00 - Introducción DWS

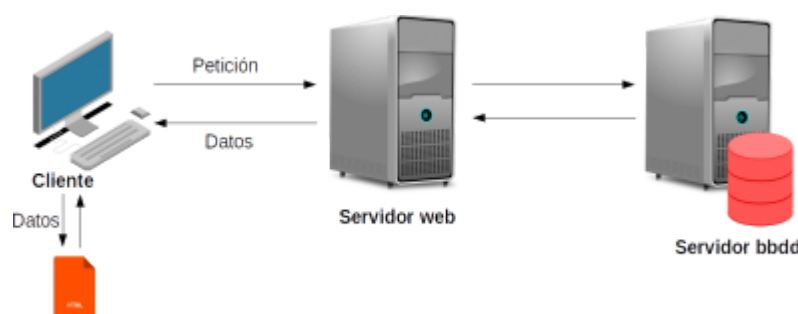
En toda conexión web existen dos partes bien separadas: cliente y servidor. Cuando el cliente (el navegador web, por ejemplo) accede a una `URL()` el servidor procesa la petición y devuelve una respuesta, que puede ser un archivo `HTML()`, `JSON`, `XML`...



Dependiendo de si el contenido del archivo es fijo o se genera en cada petición, hablamos de **páginas web estáticas** o **dinámicas**.

Antiguamente el cliente se dedicaba solo a hacer peticiones y mostrar la respuesta en formato `HTML()`. Era el servidor el encargado de recibir las peticiones de los clientes, procesarlas, acceder a la base de datos, montar las vistas (los diferentes archivos html) y enviarlas al cliente.

Hoy en día existen multitud de dispositivos que pueden realizar peticiones a servidores web, y muchos de ellos no pueden interpretar los archivos html, de ahí la importancia de los servicios web (como las `API()` Rest), donde el servidor se dedica únicamente a recibir las peticiones de los clientes, acceder a la bbdd y devolver los datos al cliente (en formato `XML`, `JSON`...). En este caso, es el cliente el encargado de montar las diferentes vistas con los datos recibidos del servidor.



De esta forma, el servidor no hace todo el trabajo, dejando al cliente la parte de montar las vistas (o gestionando los datos como quiera) aligerando la carga de trabajo.

Páginas web estáticas y dinámicas

En las **páginas web estáticas** el servidor muestra siempre la misma información en todo momento, dependiendo sólo del archivo html solicitado. En el caso que la información que queramos mostrar sea dinámica este enfoque no sirve. Por

ejemplo, si creamos una web para una empresa con 10.000 productos, nos obligaría a crear 10.000 páginas iguales para cada producto (lo único que cambiaría sería la información del producto).

Las **páginas web dinámicas** generan las respuestas de forma dinámica, es decir, dependiendo de alguna opción (como el identificador del producto) devuelve una información u otra.

Para poder generar esas respuestas de forma dinámica necesitamos instalar en nuestro **servidor HTTP** un **interprete del lenguaje de programación** utilizado y, generalmente, un **sistema gestor de base de datos**.

¿Significa éso que siempre es mejor usar páginas webs dinámicas? No, depende del tipo de web que estemos creando. Las webs dinámicas tienen sus ventajas y desventajas, y según la información que queremos mostrar usaremos éstas o webs estáticas.

Si nuestro sitio web se limita a mostrar información sin necesidad de mostrar contenido dinámico es mejor usar archivos html estáticos, ya que el servidor no tendrá que procesar los archivos del lenguaje utilizado para convertirlos en html y servirá las páginas más rápido.

En cualquier caso, hoy en día casi todas las webs tienen algún tipo de contenido dinámico (información almacenada en una bbdd, páginas con información de usuarios...), con lo que si queremos trabajar como programadores web, tendremos que aprender como crear este tipo de contenido.

Componentes servidor web

Para crear un sitio web dinámico y que sea accesible, necesitamos una serie de componentes (tanto en nuestro hosting como en local para desarrollarlo):

- **Servidor HTTP**, que será el encargado de gestionar las peticiones y enviar la respuesta (archivos html, datos JSON...)
- **Intérprete del lenguaje utilizado**, para poder interpretar el lenguaje de programación y crear los archivos html o datos.
- **Sistema Gestor de Base de Datos**, para poder almacenar nuestros datos.

Servidor HTTP

Un servidor HTTP o servidor web es un programa informático que procesa las peticiones de los clientes y genera una respuesta en cualquier lenguaje o aplicación del lado del cliente. El código recibido por el cliente es renderizado por un navegador web. Generalmente se utiliza el protocolo HTTP para estas comunicaciones.

Actualmente existen muchos servidores HTTP. A continuación veremos algunos de los más usados.

- **Apache:** El servidor web *Apache* es un servidor HTTP de código abierto para plataformas UNIX (BSD, GNU/Linux...), Microsoft Windows, Macintosh y otras. Es desarrollado y mantenido por una comunidad de usuarios bajo la supervisión de la *Apache Software Foundation*, dentro del proyecto HTTP Server (httpd). Desde 1996, Apache es el servidor HTTP más utilizado. Alcanzó su máxima cuota de mercado en 2005, siendo el servidor empleado en el 70% de los sitios web en el mundo. Sin embargo, ha sufrido un descenso en su cuota de mercado en los últimos años. En 2009 se convirtió en el primer servidor web que alojó más de 100 millones de sitios web.
- **Nginx:** Es un servidor web/proxy inverso ligero de alto rendimiento, y un proxy para protocolos de correo electrónico (IMAP/POP3). Es software libre y de código abierto, licenciado bajo la licencia BSD simplificada. También existe una versión comercial distribuida bajo el nombre de *Nginx plus*. Es multiplataforma, por lo que corre en sistema UNIX, Windows y Macintosh. El sistema es usado por una larga lista de sitios web conocidos, como WordPress, Netflix, GitHub y partes de Facebook.
- **Internet Information Services (IIS):** Es un servidor web y un conjunto de servicios para el sistema operativo Microsoft Windows. Este servicio convierte a un PC en un servidor web para Internet o una intranet, es decir que en los ordenadores que tienes este servicio instalado se pueden publicar página web tanto local como remotamente.
- **Tomcat:** Es un contenedor web con soporte de *servlets* y *JSPs*. *Tomcat* puede funcionar como servidor web por sí mismo. En sus inicios existió la percepción de que el uso de *Tomcat* de forma autónoma era solo recomendable para entornos de desarrollo y entornos con requisitos mínimos de velocidad y gestión de transacciones. Hoy en día ya no existe esa percepción y *Tomcat* es usado como servidor web autónomo en entornos con alto nivel de tráfico y alta disponibilidad. Dado que *Tomcat* fue escrito en Java, funciona en cualquier sistema operativo que disponga de la máquina virtual Java.

Lenguajes de programación

Para poder servir webs dinámicas, necesitamos algún lenguaje de programación para poder generar la información (archivos html, datos...). Para la parte del cliente (frontend), los más usados son `HTML()`, `CSS()` y Javascript. En la parte del servidor (backend), tenemos varias opciones, por ejemplo:

- **PHP (Hypertext Preprocessor):** Es un lenguaje de código abierto especialmente adecuado para el desarrollo web y que puede ser incrustado en `HTML()`. El código php es ejecutado en el servidor, enviando la respuesta al cliente. El cliente recibirá el resultado de ejecutar el script (archivos `HTML()`, JSON, XML...), aunque no se sabrá el código subyacente que era. Aunque PHP es muy fácil de aprender para el principiante, ofrece muchas características avanzadas para los programadores profesionales.

Actualmente, es el lenguaje más usado del lado servidor, entre otras cosas por ser el utilizado por Wordpress para crear sus plugins.

- **ASP.NET:** Es un modelo de desarrollo web unificado, desarrollado y comercializado por Microsoft, que incluye los servicios necesarios para crear aplicaciones web empresariales con el código mínimo. Es el sucesor de la tecnología ASP. El código de las aplicaciones puede escribirse en cualquier lenguaje compatible con el CLR (Common Language Runtime), entre ellos Microsoft Visual Basic, C#, Jscript .NET y N#.
- **Java:** Su sintaxis deriva en gran medida de C y C++, pero tiene menos utilidades de bajo nivel que cualquiera de ellos. Las aplicaciones de Java son compiladas a bytecode (clase Java), que puede ejecutarse en cualquier máquina virtual Java (JVM) sin importar la arquitectura de la computadora subyacente.
- **JSP (JavaServer Pages)** es una tecnología para crear páginas web dinámicas basadas en HTML() y XML, entre otros tipos de documentos. Es similar a php, pero usa el lenguaje de programación Java. La principal ventaja de JSP frente a otros lenguajes es que el lenguaje Java es un lenguaje de propósito general que excede el mundo web y que es apto para crear clases que manejan lógica de negocio y acceso a datos de una manera prolija.
- **Python** es un lenguaje de programación interpretado, funcional, orientado a objetos e interactivo. Python cuenta con una sintaxis muy limpia y clara. Cuenta con módulos, clases, excepciones, tipos de datos de muy alto nivel, así como tipado dinámico.
- **Node.js:** Es un entorno de ejecución para JavaScript del lado del servidor construido con el motor JavaScript V8, desarrollado por Google para su navegador Chrome. Node.js usa un modelo de operaciones E/S sin bloqueo y orientado a eventos, que lo hace liviano y eficiente. Al contrario que la mayoría del código JavaScript, no se ejecuta en un navegador, sino en el servidor.
- **Go:** Es un lenguaje de programación desarrollado por Google concurrente y compilado inspirado en la sintaxis de C, que intenta ser dinámico como Python y con el rendimiento de C o C++. Es un proyecto open-source. Go es un lenguaje de programación compilado y admite el paradigma orientado a objetos, pero a diferencia de los lenguajes de programación más populares no dispone de herencia de tipos y tampoco de palabras clave que denoten claramente que soporta este paradigma. La sencillez es la característica principal de Go, su sintaxis es clara y concisa.
- **Ruby:** Es un lenguaje de programación interpretado, reflexivo y orientado a objetos. Combina una sintaxis inspirada en Python y Perl con características de programación orientada a objetos similares a Smalltalk. Comparte también funcionalidad con otros lenguajes de programación como Lisp, Lua y Dylan. Ruby es un lenguaje de programación interpretado en una sola pasada y su implementación oficial es distribuida bajo una licencia de software libre.

Sistemas gestores de base de datos

La mayoría de aplicaciones web, utilizan algún sistema gestor de base de datos. Existen multitud de SGBD en el mercado, tanto libres como de pago:

- **MySQL:** SGBD relacional desarrollado bajo licencia dual: Licencia pública general/Licencia comercial por Oracle Corporation, y está considerado como la base de datos de código abierto más popular del mundo, sobre todo para entornos de desarrollo web. MySQL fue inicialmente desarrollado por MYSQL AB, que fue adquirida por Sun Microsystems en 2008, y ésta a su vez fue comprada por Oracle Corporation en 2010.
- **MariaDB:** Derivado de MySQL con licencia GPL(2). Es desarrollado por Michael Widenius (fundador de MySQL), la fundación MariaDB y la comunidad de desarrolladores de software libre. Este SGBD surge a raíz de la compra de Sun Microsystems (compañía que había comprado previamente MYSQL) por parte de Oracle.
- **PostgreSQL:** SGBD relacional orientado a objetos y libre, publicado bajo licencia PostgreSQL, similar a la BSD o la MIT. Como muchos otros proyectos de código abierto, el desarrollo de PostgreSQL no es manejado por una empresa o persona, sino que es dirigido por una comunidad de desarrolladores que trabajan de forma desinteresada, altruista, libre o apoyados por organizaciones comerciales. Dicha comunidad es denominada el PGDG (PostgreSQL Global Development Group).
- **Microsoft SQL Server:** SGBD relacional desarrollado por la empresa Microsoft. SQL Server ha estado tradicionalmente disponible sólo para sistemas operativos Windows de Microsoft, pero desde 2017 también está disponible para Linux y Docker containers.
- **MongoDB:** Sistema de base de datos **NoSQL**, orientado a documentos y de código abierto. En lugar de guardar los datos en tablas, tal y como se hace en las bases de datos relacionales, MongoDB guarda estructuras de datos BSON (una especificación similar a JSON) con un esquema dinámico, haciendo que la integración de los datos en ciertas aplicaciones sea más fácil y rápida. MongoDB es una base de datos adecuada para su uso en producción y con múltiples funcionalidades. El código fuente está disponible para los sistemas operativos Windows, GNU/Linux, OS(2) X y Solaris.

Entorno de desarrollo local

Lo primero que tenemos que hacer para desarrollar un proyecto web es instalar y configurar nuestro entorno de desarrollo local. Si vamos a crear una web estática no haría falta instalar nada. Podemos crear los diferentes archivos html en una carpeta (con los css y las imágenes que utilizemos) y el navegador ya se encargaría de interpretarlos y mostrar su contenido.

Por el contrario, si queremos desarrollar una web dinámica tendremos que instalar los componentes vistos en el punto anterior (servidor HTTP, intérprete del lenguaje de programación y SGBC). Podemos instalar cada uno de estos

componentes por separado, pero existen soluciones integradas que nos instalan todos esos componentes de forma rápida y sencilla.

Alicaciones web integradas

Como hemos visto en el punto anterior, podemos instalar todo lo necesario para desarrollar web dinámica sin necesidad de instalar y configurar cada una de sus partes por separado.

- **XAMPP:** Paquete de software libre, cuyo nombre es un acrónimo de los componentes que incluye. X (que indica el sistema operativo donde se va instalar), Apache como servidor web, MariaDB, Php y Perl. Actualmente dispone de versiones para Windows, Linux y MacOS. Existen versiones exclusivas para Linux (LAMP) o MacOS (MAMP), aunque se puede instalar XAMPP en cualquiera de esas plataformas.
- **EasyPHP:** Ofrece dos módulos orientados a desarrolladores PHP. *Devserver* permite configurar un servidor local para desarrollar en cualquier lugar (en casa, en el trabajo, en el ordenador portátil...) gracias a la portabilidad del sistema. *Webserver* permite utilizar la aplicación como servidor web para alojar contenidos y que éstos sean accesibles desde Internet.
- **Vagrant:** Herramienta gratuita de línea de comandos, disponible para Windows, MacOS X y GNU/Linux, que permite generar entornos de desarrollo reproducibles y compartibles de forma muy sencilla. Para ello, Vagrant crea y configura máquinas virtuales a partir de simples ficheros de configuración.
- **Laravel Valet:** Entorno de desarrollo minimalista para Mac y ahora, gracias a la comunidad, también está disponible para Linux. Aunque no es un paquete oficial de Laravel, esta versión funciona muy bien para distintas distribuciones de Linux como Ubuntu, Fedora y Arch y sus correspondientes derivados. Valet para Linux configura su sistema para ejecutar siempre Nginx en segundo plano cuando se inicia la máquina. Luego, usando Dnsmasq, se toman todas las peticiones y las apunta al dominio *.test (o el que esté configurado), haciendo referencia a todos los sitios que tengas instalados.
- **Docker:** Es un proyecto de código abierto que automatiza el despliegue de aplicaciones dentro de contenedores de software, proporcionando una capa adicional de abstracción y automatización de virtualización de aplicaciones en múltiples sistemas operativos. Un contenedor Docker, a diferencia de una máquina virtual, no requiere incluir un sistema operativo independiente. En su lugar, se basa en las funcionalidades del kernel y utiliza el aislamiento de recursos (CPU, la memoria, el bloque E / S, red, etc.) y namespaces separados para aislar la vista de una aplicación del sistema operativo. Docker nos ofrece multitud de contenedores preparados para ejecutar LAMP, XAMP, Valet (además de muchos otros tipos de software)... sin necesidad de instalarlo en nuestra máquina local.

Entornos de desarrollo

El siguiente paso, una vez hemos instalado y configurado nuestro entorno de desarrollo local, será elegir el programa que vamos a utilizar para crear nuestras aplicaciones.

Aunque se puede utilizar la mayoría de editores de texto que vienen instalados por defecto en los sistemas operativos (notepad, gedit...), sus recursos se quedan cortos a la hora de programar. En este sentido, podemos optar por instalar un *editor de texto enriquecido*, con funciones extras para la programación, o algún *entorno de desarrollo integrado (IDE)*. La principal diferencia radica en las herramientas disponibles en cada uno de ellos para trabajar.

Mientras que un IDE suele tener muchísimas herramientas integradas, los editores de texto tienen bastantes menos en comparación (aunque a la mayoría se le pueden agregar esas herramientas mediante *plugins*). Por contra, la curva de aprendizaje de un editor de texto es mucho menor que la de un IDE. Además, por noma general un editor de texto tiene un tamaño mucho menor que un IDE, con lo que es más ligero que un IDE.

Editores de texto enriquecidos

- **Atom:** Editor de texto orientado al desarrollo de aplicaciones. Se trata de una aplicación de software libre que ha sido desarrollado por *Github*. Atom se ha concebido como un editor flexible al cual se le pueden añadir nuevas funcionalidades y complementos, además de temas, que nos permiten configurar un espacio de trabajo adaptado a nuestras necesidades.
- **Sublime Text:** Editor de texto y editor de código fuente escrito en C++ y Python para los *plugins*. Se puede descargar y evaluar de forma gratuita. Sin embargo, no es software libre o de código abierto, y se debe obtener una licencia para su uso continuado, aunque la versión de evaluación es plenamente funcional, y no tiene fecha de caducidad.
- **Visual Studio Code:** Editor de código fuente desarrollado por Microsoft para Windows, Linux y macOS. Incluye soporte para la depuración, control integrado de Git, resaltado de sintaxis, finalización inteligente de código, fragmentos y refactorización de código. Es personalizable, por lo que los usuarios pueden cambiar el tema del editor, los atajos de teclado y las preferencias. Es gratuito y de código abierto, aunque la descarga oficial está bajo software propietario.

Entornos de desarrollo integrados

- **Eclipse:** Plataforma de software compuesto por un conjunto de herramientas de programación, de código abierto y multiplataforma para desarrollar aplicaciones. Fue desarrollado originalmente por IBM, como el sucesor de su familiar de herramientas para VisualAge. Ahora es desarrollado por la Fundación Eclipse, organización sin ánimo de lucro que fomenta una comunidad de código abierto.
- **NetBeans:** IDE libre, hecho principalmente para el lenguaje de programación Java. Existe además un número importante de módulos para

extenderlo. NetBeans es un producto libre y gratuito sin restricciones de uso. Sun Microsystems fundó el proyecto de código abierto NetBeans en junio de 2000, y continúa siendo el patrocinador principal de los proyectos (Actualmente Sun Microsystems es administrado por Oracle Corporation). La plataforma NetBeans permite que las aplicaciones sean desarrolladas a partir de un conjunto de componentes de software llamados módulos. Debido a que los módulos pueden ser desarrollados independientemente, las aplicaciones basadas en NetBeans pueden ser extendidas fácilmente por otros desarrolladores de software.

- **Microsoft Visual Studio:** IDE para sistemas operativos Windows. Soporta múltiples lenguajes de programación, tales como C++, C#, Visual Basic .NET, F#, Java, Python, Ruby y PHP, al igual que entornos de desarrollo web, como ASP.NET MVC, Django, etc. Visual Studio permite a los desarrolladores crear sitios y aplicaciones web, así como servicios web en cualquier entorno que soporte la plataforma .NET. Así, se pueden crear aplicaciones que se comuniquen entre estaciones de trabajo, páginas web, dispositivos móviles, dispositivos embebidos y consolas, entre otros.
- **IntelliJ IDEA:** IDE para desarrollar aplicaciones con Java, Kotlin, Groovy y otros lenguajes basados en JVM. Está desarrollado por JetBrains y tiene dos tipos de licencia: Apache 2 y comercial. JetBrains también ha desarrollado varios IDEs para diferentes tipos de lenguajes (PHPStorm para PHP, PyCharm para Python, WebStorm para JavaScript...).

Aplicación

Durante el curso, vamos a desarrollar una aplicación web sobre una tienda online de libros. Para eso, usaremos la siguiente base de datos

Ejercicios

Ejercicio 1

Asegúrate de tener instalado JDK en tu ordenador. En caso contrario, instálalo y configura correctamente las variables de entorno.

Ejercicio 2

Instala MariaDB en tu ordenador y algún editor de textos enriquecido o IDE (VSC, IntelliJ...). Puedes instalar también alguna aplicación para gestionar tu SGBD (HeidiSQL, Workbench, DBeaver...)

📄 clase/daw/dws/1eval/introduccion.txt 🕒 Última modificación: 2025/09/16 12:54 por cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-Noncommercial-Share Alike 4.0 International

01 - Ecosistema Java web

Java es un lenguaje de programación versátil y potente que ha ganado una amplia popularidad en la comunidad de desarrollo de software. Una de las razones detrás de su éxito es la capacidad de ejecutar Java en varias plataformas diferentes. Sin embargo, es importante comprender las diferencias entre las distintas plataformas de Java para aprovechar al máximo este lenguaje:

- **Java Standard Edition (JSE):** Proporciona un conjunto básico de bibliotecas y herramientas necesarias para desarrollar aplicaciones Java independientes, que se ejecutan en una máquina virtual Java (JVM). Es la plataforma más comúnmente utilizada y es adecuada para la mayoría de los proyectos de desarrollo de aplicaciones Java.
- **Java Enterprise Edition (JEE):** Se basa en JSE y está diseñada específicamente para el desarrollo de aplicaciones empresariales de gran escala. JEE proporciona un conjunto adicional de bibliotecas y especificaciones que permiten a los desarrolladores crear aplicaciones empresariales robustas y escalables. Estas aplicaciones suelen requerir características avanzadas como transacciones distribuidas, seguridad, comunicación en red, servicios web y acceso a bases de datos.
- **Java Micro Edition (JME):** Está orientada a dispositivos con recursos limitados, como teléfonos móviles, PDAs y dispositivos integrados. JME es una plataforma reducida de Java que se adapta a las restricciones de memoria y capacidad de procesamiento de estos dispositivos.

Durante 1º, utilizamos básicamente JSE, que viene incluido en JDK. Ahora, necesitamos añadir más funcionalidad para crear webs, ~~API()~~ Rest... Por lo tanto, vamos a utilizar, además, JEE, o **Jakarta EE**, que es como se le conoce ahora.

Jakarta EE

Jakarta EE (<https://jakarta.ee/about/jakarta-ee/>) (anteriormente conocida como Java Enterprise Edition o JEE) es una plataforma de desarrollo empresarial que ofrece un conjunto de especificaciones y bibliotecas para construir y desplegar aplicaciones empresariales en Java. Jakarta EE se basa en las tecnologías y estándares de JEE, que han sido desarrollados y evolucionados a lo largo de los años por la comunidad de desarrolladores y diversas organizaciones.

La plataforma Jakarta EE proporciona un conjunto de APIs y servicios para el desarrollo de aplicaciones empresariales robustas y escalables. Estas aplicaciones suelen requerir características avanzadas como transacciones distribuidas, seguridad, comunicación en red, servicios web, acceso a bases de datos y gestión de componentes, entre otros.

Una de las características clave de Jakarta EE es su enfoque modular y extensible. Permite a los desarrolladores elegir las especificaciones y tecnologías que necesitan para sus aplicaciones, lo que evita la sobrecarga de funcionalidades innecesarias. Además, Jakarta EE se puede implementar en varios servidores de aplicaciones compatibles, lo que brinda flexibilidad y portabilidad en el despliegue de las aplicaciones.

En 2017, la plataforma Java EE fue transferida a la Fundación Eclipse y renombrada como Jakarta EE como resultado de un proceso comunitario. Esta transición tuvo como objetivo fortalecer la colaboración y la apertura en el desarrollo de la plataforma, permitiendo una mayor participación de la comunidad y garantizando su evolución continua.

En resumen, Jakarta EE es una plataforma de desarrollo empresarial basada en Java que proporciona un conjunto de especificaciones y bibliotecas para la construcción de aplicaciones empresariales robustas y escalables. Es conocida por su modularidad, extensibilidad y capacidad de implementación en varios servidores de aplicaciones. Jakarta EE continúa evolucionando gracias a la colaboración de la comunidad de desarrolladores y la Fundación Eclipse.

Jakarta está compuesta por un conjunto de especificaciones (<https://jakarta.ee/specifications/>), entre las cuales están:

- **Jakarta Annotations:** Las anotaciones de Jakarta definen una colección de anotaciones que representan conceptos semánticos comunes que permiten un estilo declarativo de programación que se aplica a una variedad de tecnologías Java.
- **Jakarta Dependency Injection:** La inyección de dependencia de Jakarta especifica un medio para obtener objetos de tal manera que se maximice la reutilización, la capacidad de prueba y el mantenimiento en comparación con enfoques tradicionales como constructores, fábricas y localizadores de servicios (p. ej., JNDI).
- **Jakarta Managed Beans:** Define un conjunto de servicios básicos para objetos administrados por contenedor con requisitos mínimos, también conocidos bajo el acrónimo *POJO* (Plain Old Java Objects).
- **Jakarta Persistence (JPA):** Define un estándar para la gestión de la persistencia y el mapeo de objetos/relaciones en entornos Java(R).
- **JAX-RS:** Proporciona soporte en la creación de servicios web de acuerdo con el estilo arquitectónico REST.

POJO (Plain Old Java Object) es una clase Java común que no sigue ninguna restricción o requisito específico impuesto por frameworks o bibliotecas externas. Se trata de objetos simples y ligeros que encapsulan datos y tienen métodos para acceder a ellos.

Un POJO se caracteriza por lo siguiente:

- Es una clase Java estándar, sin extender ninguna clase especial ni implementar interfaces específicas.

- Los atributos de un POJO suelen ser privados y se accede a ellos mediante métodos getters y setters.
- Los POJOs suelen tener métodos básicos, como constructores, *getters*, *setters* y, posiblemente, métodos auxiliares.
- Un POJO no depende de ninguna biblioteca o framework específico y no tiene anotaciones o configuraciones especiales.

Un Java Bean es una clase Java que sigue ciertas convenciones y patrones de diseño para permitir su fácil integración en entornos de desarrollo y frameworks específicos. Se utiliza para representar entidades o componentes en una aplicación y proporciona un conjunto predefinido de características y comportamientos.

Un Java Bean típicamente cumple con las siguientes características:

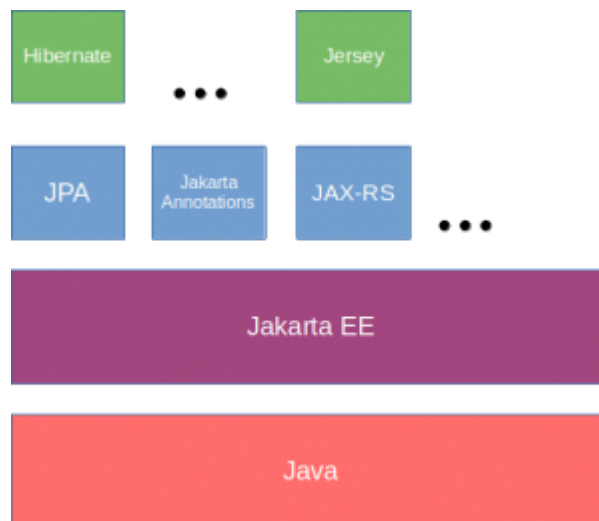
- Los atributos del bean suelen ser declarados como variables privadas.
- Los atributos se acceden mediante métodos *getter* y *setter* públicos.
- Debe tener un constructor público sin argumentos. Esto permite la creación de instancias del bean utilizando el operador *new()* sin necesidad de pasar argumentos.
- Un bean puede implementar interfaces opcionales, como *Serializable*, para permitir la serialización y deserialización del objeto.
- Los nombres de los métodos y atributos del bean siguen convenciones de nomenclatura estándar, como el prefijo "get" para los métodos *getter* y el prefijo *set* para los métodos *setter*.

Implementaciones

Jakarta EE, como hemos visto, es un conjunto de especificaciones para varias tecnologías. A partir de ahí, los fabricantes crean sus propias implementaciones siguiendo las reglas de las diferentes APIS, por ejemplo:

- **Hibernate:** Herramienta de mapeo objeto-relacional (ORM) que facilita el mapeo de atributos entre una base de datos relacional tradicional y el modelo de objetos de una aplicación, mediante archivos declarativos (XML) o anotaciones en los beans de las entidades que permiten establecer estas relaciones.
- **Jersey:** Framework para la creación de servicios REST en Java. Está basado en el estándar JAX-RS.

En realidad, cuando hacemos una aplicación Java utilizamos esas implementaciones (Hibernate, Jersey...), no las especificaciones de Jakarta.



Spring

Spring (<https://spring.io/>) es un framework de desarrollo de aplicaciones Java que ofrece un enfoque alternativo y más ligero para el desarrollo de aplicaciones empresariales en comparación con Jakarta EE. Aunque Spring se basó inicialmente en las tecnologías de Java EE, ha evolucionado de manera independiente y ha introducido sus propias abstracciones y conceptos.

Sin embargo, a partir de la versión 3.0, Spring adoptó un enfoque más modular y comenzó a alinearse más estrechamente con los estándares de Java EE y Jakarta EE. En lugar de depender completamente de la plataforma Java EE completa, Spring adoptó un enfoque más granular, permitiendo a los desarrolladores elegir las tecnologías específicas que desean utilizar en sus aplicaciones. Por ejemplo, Spring puede integrarse fácilmente con implementaciones de servlets y contenedores de aplicaciones compatibles con Jakarta EE.

Spring y Jakarta EE pueden coexistir en una aplicación y utilizarse de forma complementaria según las necesidades del proyecto. Spring proporciona muchas características y soluciones adicionales que no están disponibles en Jakarta EE, como el soporte para inyección de dependencias, programación declarativa, pruebas unitarias y soporte para integración con otros sistemas. Al mismo tiempo, Jakarta EE ofrece un conjunto sólido de tecnologías y estándares para el desarrollo de aplicaciones empresariales.

Spring Boot

Spring Boot (<https://spring.io/projects/spring-boot>) es un proyecto dentro del ecosistema de Spring Framework que simplifica y agiliza el desarrollo de aplicaciones Java. Se basa en los principios de Spring, pero proporciona características adicionales para facilitar la configuración y el despliegue de aplicaciones. Algunas de sus características son:

- Utiliza la convención sobre configuración para minimizar la necesidad de configuración manual. Proporciona valores predeterminados inteligentes y autoconfiguración basada en las dependencias presentes en el proyecto.

- Se integra perfectamente con las características y componentes de Spring Framework. Puede aprovechar todas las características de Spring, como la inyección de dependencias, la programación declarativa y la gestión transaccional.
- Incluye un servidor web embebido (por ejemplo, Tomcat, Jetty) para facilitar el despliegue de aplicaciones sin la necesidad de configurar y desplegar un servidor externo. Además, proporciona herramientas para generar artefactos ejecutables, como archivos JAR autocontenidos, que simplifican el despliegue y la distribución de la aplicación.
- Analiza el entorno de ejecución y automáticamente configura los componentes necesarios en función de las dependencias presentes en el proyecto. Esto permite que la aplicación se ejecute con una configuración predeterminada sin requerir una configuración manual extensa.
- Ofrece una forma sencilla de administrar las dependencias del proyecto a través de su herramienta de gestión de dependencias, como Maven o Gradle. Proporciona dependencias preconfiguradas para tecnologías populares, lo que simplifica la incorporación de nuevas funcionalidades a la aplicación.

En general, Spring Boot permite desarrollar aplicaciones Java de manera más rápida y sencilla al proporcionar una configuración inteligente, herramientas de despliegue simplificadas y una integración perfecta con Spring Framework. Facilita el desarrollo de aplicaciones robustas y escalables, al tiempo que reduce la carga de configuración manual y el tiempo de desarrollo.

Durante el curso, vamos a utilizar Spring Boot para crear nuestras aplicaciones web.

📄 clase/daw/dws/1eval/java.txt 📅 Última modificación: 2025/09/16 11:39 por cesguiro

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-Noncommercial-Share Alike 4.0 International

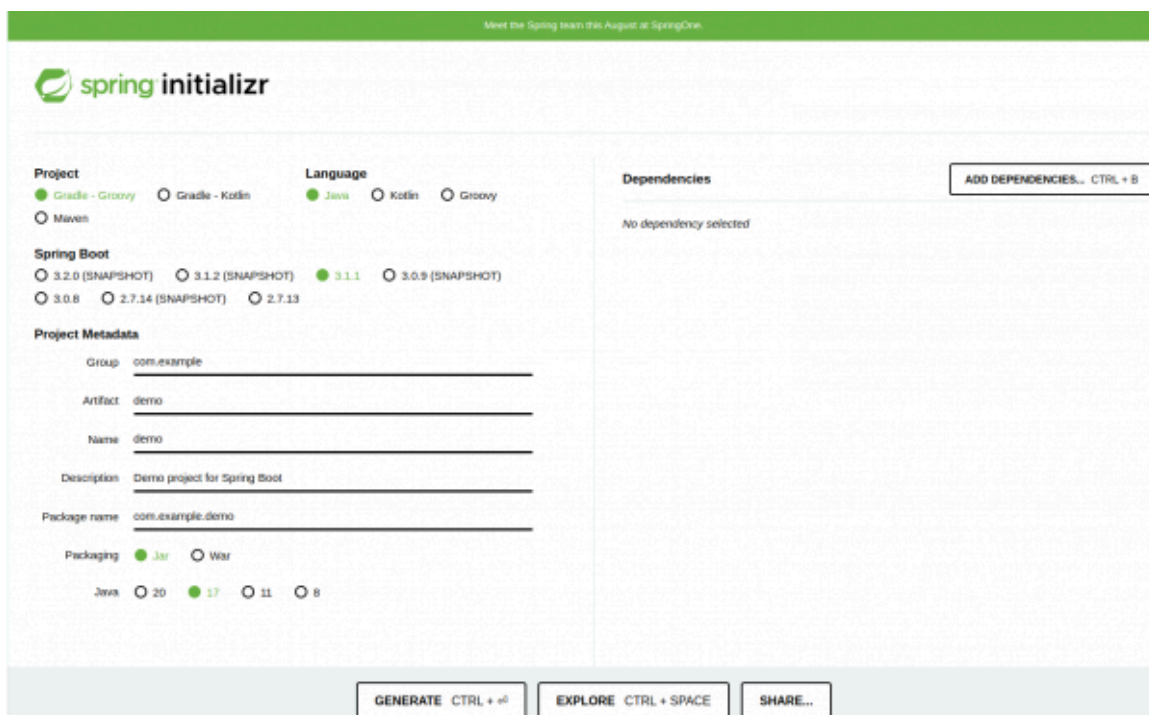
02 - Spring-Boot

Spring Boot es un framework de desarrollo de aplicaciones Java que simplifica el proceso de construcción y despliegue de aplicaciones. Proporciona un conjunto de características y herramientas que facilitan la configuración inicial, la gestión de dependencias y la implementación de aplicaciones de manera rápida y sencilla.

Spring Boot se basa en el framework Spring, aprovechando su potencia y modularidad, pero reduciendo la complejidad al proporcionar una configuración automática y por defecto. Esto permite a los desarrolladores centrarse en la lógica de negocio de sus aplicaciones en lugar de preocuparse por la configuración y la integración de diferentes componentes.

Crear un proyecto

Para crear un nuevo proyecto Spring Boot contamos con una herramienta muy útil que nos automatiza el proceso: Spring Initializr (<https://start.spring.io/>). Podemos crear el proyecto directamente desde su web, configurándolo a nuestra medida y añadiendo las dependencias necesarias. Una vez terminado el proceso, generamos el proyecto, lo que nos descargará un zip con el proyecto ya configurado.

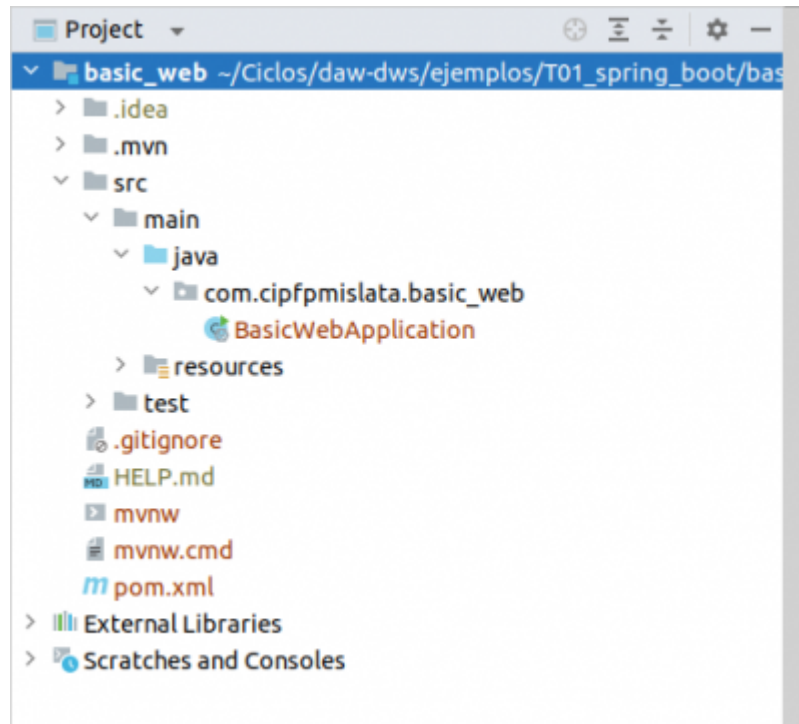


The screenshot shows the Spring Initializr web form. At the top, there is a green banner with the text "Meet the Spring team this August at SpringOne". Below the banner is the "spring initializr" logo. The form is divided into several sections: "Project" with radio buttons for "Gradle - Groovy" (selected), "Gradle - Kotlin", and "Maven"; "Language" with radio buttons for "Java" (selected), "Kotlin", and "Groovy"; "Spring Boot" with radio buttons for versions "3.2.0 (SNAPSHOT)", "3.1.2 (SNAPSHOT)", "3.1.1" (selected), "3.0.9 (SNAPSHOT)", "3.0.8", "2.7.14 (SNAPSHOT)", and "2.7.13"; "Project Metadata" with input fields for "Group" (com.example), "Artifact" (demo), "Name" (demo), "Description" (Demo project for Spring Boot), and "Package name" (com.example.demo); "Packaging" with radio buttons for "Jar" (selected) and "War"; and "Java" with radio buttons for versions "20", "17" (selected), "11", and "8". There is a "Dependencies" section on the right with a button "ADD DEPENDENCIES... CTRL + B" and the text "No dependency selected". At the bottom, there are three buttons: "GENERATE CTRL + G", "EXPLORE CTRL + SPACE", and "SHARE...".

Tendremos que elegir el tipo de proyecto (Gradle o Maven), el lenguaje (Java, Kotlin o Groovy) y la versión de Spring Boot. Además, añadiremos los metadatos (artefacto, grupo, descripción...), así como el tipo de empaquetado (que, en nuestro caso, será jar) y la versión de Java a utilizar.

En la sección de la derecha, podremos añadir las dependencias que queramos. Por ahora, vamos a añadir sólo la dependencia *Spring Web*, que nos permite crear webs (incluido Api Rest) y lleva un servidor Tomcat embebido.

Una vez lo tengamos todo, le damos al botón *Generate* y nos generará un archivo comprimido con el proyecto ya creado y configurado.

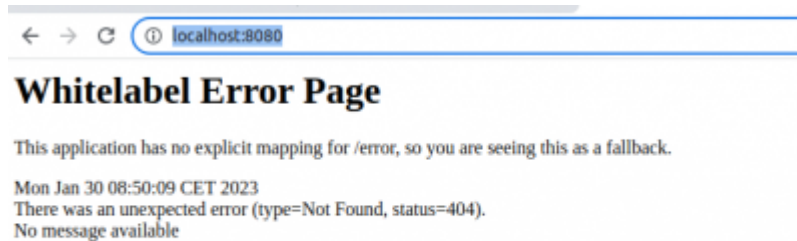


Iniciar el servidor

Al crear el proyecto, Spring Boot crea una clase que será la encargada de iniciarlo:

```
1 package com.cipfpmislata.basic_web;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class BasicWebApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(BasicWebApplication.class, args);
11     }
12
13 }
```

Si ejecutamos el proyecto, veremos que Spring Boot ha levantado el servidor Tomcat en el puerto por defecto (8080). Si accedes desde un navegador a <http://localhost:8080/> (<http://localhost:8080/>), deberías ver una página genérica de error (tranquilo, esto quiere decir que todo ha funcionado correctamente):



Cambiar puerto por defecto en Spring Boot

Para cambiar el puerto en el que escucha el servidor embebido de *Spring Boot*, la manera más rápida y fácil es añadiendo la siguiente línea al archivo *application.properties* de la carpeta *resources*:

```
1 | server.port=8081;
```

Si abres ahora el navegador, verás como podrás acceder a tu aplicación cargando la url <http://localhost:8081/> (<http://localhost:8081/>):



Beans

Dentro de la infraestructura de *Spring* tenemos algo llamado Spring Container (<https://docs.spring.io/spring-framework/docs/3.2.x/spring-framework-reference/html/beans.html>), que vendría siendo el núcleo del framework. El contenedor crea objetos, los une, los configura y administra su ciclo de vida completamente. Los Beans (<https://docs.spring.io/spring-framework/docs/current/reference/html/core.html#beans-definition>) son objetos que puede manejar el contenedor de *Spring*.

Para crear un *Bean Spring* utiliza anotaciones mediante el símbolo arroba (@). Por ejemplo, vamos a crear un controlador para responder a las rutas de nuestra aplicación. Para ello, crea la clase *MainController*:

```
1 | package com.cipfpmlata.basic_web;  
2 |  
3 | import org.springframework.web.bind.annotation.RestController  
4 |  
5 | @RestController  
6 | public class MainController {  
7 |  
8 |     public void index() {  
9 |         System.out.println("Método index del controlador N  
10 |     }  
11 | }
```

Fíjate que en la línea anterior a la definición de la clase hemos añadido la anotación **@RestController** (asegúrate de importar la librería **org.springframework.web.bind.annotation.RestController** para que funcione). Con esa notación, le indicamos a *Spring* que queremos crear un *Bean* de tipo controlador de Rest. Existen varios tipos de Beans en Spring Boot. Durante el curso, iremos usando algunos de ellos.

Rutas

Para que *Spring* sepa qué método ejecutar según la ruta que introduzca el usuario en el navegador, vamos a utilizar otro tipo de anotaciones delante de cada método: **@GetMapping**:

```
1 package com.cipfpmislata.basic_web;
2
3 import org.springframework.web.bind.annotation.RestController;
4 import org.springframework.web.bind.annotation.GetMapping;
5
6 @RestController
7 public class MainController {
8
9     @GetMapping("/")
10    public void index() {
11        System.out.println("Método index del controlador MainController ejecutándose");
12    }
13 }
```

Con ésto le estamos indicando a *Spring* que cada vez que se acceda a la ruta raíz de nuestra web (/) se ejecute el método *MainController* de nuestra aplicación. Si abrimos el navegador y accedemos a la ruta raíz (<http://localhost:8080/>) deberíamos ver en la terminal la frase “Método index de MainController ejecutándose”:



```
2023-07-18T09:18:41.771+02:00 INFO 60627 --- [main] o.s.b.w.s.BrowserSupport : Started BrowserSupport in 0.014 seconds (process)
2023-07-18T09:18:41.771+02:00 INFO 60627 --- [main] o.s.b.w.s.BrowserSupport : Initializing Spring DispatcherServlet 'dispatcherServlet'
2023-07-18T09:18:41.771+02:00 INFO 60627 --- [main] o.s.b.w.s.BrowserSupport : Initializing Servlet 'dispatcherServlet'
2023-07-18T09:18:41.771+02:00 INFO 60627 --- [main] o.s.b.w.s.BrowserSupport : Completed initialization in 1 ms
Método index del controlador MainController ejecutándose
```

Añadiendo más rutas

Vamos a añadir la ruta */about* con la información de nuestra organización. Lo único que tenemos que hacer es crear un nuevo método en nuestro controlador e indicarle la ruta a la que tiene que reaccionar:

```
1 package com.cipfpmislata.basic_web;
2
3 import org.springframework.web.bind.annotation.RestController
4 import org.springframework.web.bind.annotation.GetMapping;
5
6 @RestController
7 public class MainController {
8
9     @GetMapping("/")
10    public void index() {
11        System.out.println("Método index del controlador ")
12    }
13
14    @GetMapping("/about")
15    public void about() {
16        System.out.println("Método about de MainController")
17    }
18 }
```

Ten en cuenta que no tenemos que juntar todas las rutas en un mismo controlador. En una aplicación web, normalmente tenemos varios controladores, cada uno con su propias rutas. Gracias a *Spring Boot*, para que todo funcione correctamente solo tenemos que añadir anotaciones válidas (@RestController, @Controller...) a cada uno de nuestros controladores y definir las rutas para que se haga cargo el framework.

Si las rutas de nuestros controladores tienen todas la misma raíz (/categorias, /categorias/id_categoria, /categorias/id_categoria/productos...) podemos utilizar la anotación @RequestMapping delante de la clase para establecer esa ruta común:

```
1 @RequestMapping("/categorias")
2 @RestController
3 public class CategoryController {
4     ...
```

Por ahora, solo estamos utilizando el método *HTTP GET* mediante la anotación @GetMapping, que sirve para recuperar información. Si queremos añadir, modificar o borrar datos, también tenemos otras anotaciones como @PostMapping, @PutMapping, @DeleteMapping

Rutas con parámetros variables

¿Qué pasa si nuestras rutas son del estilo /productos/id_product? Obviamente, hacer una ruta por cada producto es inviable. Para eso, tenemos que definir una ruta con un parámetro variable (id_product). La manera de hacerlo mediante *Spring* es utilizando las llaves para definir parámetros variables en nuestras rutas y usando la notación @PathVariable en la cabecera de nuestro método para recoger ese parámetro.

Por ejemplo, crea el controlador *ProductController* y añade lo siguiente:

```

1  package com.cipfpmislata.basic_web;
2
3  import org.springframework.web.bind.annotation.RestController;
4  import org.springframework.web.bind.annotation.GetMapping;
5  import org.springframework.web.bind.annotation.PathVariable;
6
7  @RestController
8  public class ProductController {
9
10     @GetMapping("/products/{id}")
11     public void getById(@PathVariable("id") int id){
12         System.out.println("Ruta: /productos/" + id);
13     }
14 }

```

Si abrimos la ruta, por ejemplo, */products/2*, veremos como nuestro método recoge el valor de dicho *id*:

```

2023-07-19T09:19:06.818+02:00 INFO 60952 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
2023-07-19T09:19:06.819+02:00 INFO 60952 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
Ruta: /productos/2

```

Si accedes a la ruta */products/14*, verás como cambia el *id* del producto de manera correcta:

```

2023-07-19T09:19:06.81
2023-07-19T09:19:06.81
Ruta: /productos/2
Ruta: /productos/14

```

Si al parámetro lo llamas igual que la variable de entrada del método, te puedes ahorrar escribir su nombre entre paréntesis en la anotación `@PathVariable`:

```

1  @GetMapping("/productos/{id}")
2  public void getById(@PathVariable int id){
3      System.out.println("Ruta: /productos/" + id);
4  }

```

Inyección de dependencias automáticas

Vamos a ver otra funcionalidad muy útil de Spring: la inyección de dependencias automática.

La inyección de dependencias (DI) es un principio fundamental en el diseño de software y se refiere a la práctica de suministrar las dependencias de un objeto desde el exterior, en lugar de que el objeto las cree por sí mismo. En lugar de que un objeto se encargue de instanciar y gestionar sus propias dependencias, se le proporcionan desde fuera, lo que promueve la modularidad, la reutilización y la separación de responsabilidades en el código.

Spring lo hace mediante la inversión de control (IoC), donde un contenedor es responsable de administrar las dependencias y las proporciona automáticamente cuando se necesitan.

Vamos a ver un ejemplo para entenderlo mejor. Tenemos una interfaz llamada *ProductService* con un método *getAll()*:

```
1 public interface ProductService {
2
3     void getAll();
4 }
```

Además, implementamos la interfaz mediante la clase *ProductServiceImpl*. El método mostrará "Getting all products" en el log:

```
1 public class ProductServiceImpl implements ProductService{
2
3     @Override
4     public void getAll() {
5         LoggerFactory.getLogger(ProductService.class).warn(
6     }
7 }
```

Vamos a crear ahora un método en nuestro controlador de productos que llame al método *getAll()* del servicio recién creado. Este método se ejecutará cuando accedamos a la ruta *localhost:8080/products*:

```
1 @RequestMapping(ProductController.URL)
2 @RestController
3 public class ProductController {
4     public static final String URL = "/products";
5     private final ProductService productService = new Proc
6
7     @GetMapping
8     public ResponseEntity<Void> getAll() {
9         productService.getAll();
10        return ResponseEntity.ok()
11            .body("Hello World!");
12    }
13 }
```

Básicamente lo que hemos hecho es añadir un atributo de tipo *ProductService* e inicializarlo con una instancia de tipo *ProductServiceImpl* (Recuerda como funciona el polimorfismo). Después, ejecutamos su método *getAll()*.

Lo que devuelve el método es una respuesta http 200 (ok) con el contenido "Hello World!". Ya iremos viendo el formato y código de las respuestas en APIs más adelante.

Si accedemos a la ruta, deberíamos ver el mensaje "Getting all products" en el log.

Vamos a hacer ahora que sea el propio *Spring* el que se encargue de inyectar la dependencia y crear el objeto de forma automática. Para eso, utilizamos la etiqueta **@Autowired** en el atributo:

```

1  @RequestMapping(ProductController.URL)
2  @RestController
3  public class ProductController {
4      public static final String URL = "/products";
5      @Autowired
6      private final ProductService productService;
7
8      @GetMapping
9      public ResponseEntity<Void> getAll() {
10         productService.getAll();
11         return ResponseEntity.ok()
12             .body("Hello World!");
13     }
14 }

```

Si lo dejamos así, vemos que Spring nos muestra un error:

```

1  Field productService in es.cesguiro.ProductController requi

```

Ésto ocurre porque *Spring* sólo es capaz de manejar objetos que sean de tipo *Bean*, con lo que tenemos que convertir nuestro servicio en uno de ellos:

```

1  @Service
2  public class ProductServiceImpl implements ProductService{
3
4      @Override
5      public void getAll() {
6          LoggerFactory.getLogger(ProductService.class).warn(
7      }
8  }

```

Ahora ya no debería mostrarse el error y funcionar todo.

En este caso, hemos utilizado la anotación `@Service` en nuestro servicio. En realidad, podríamos utilizar cualquier anotación que convierta la clase en un *Bean* (`@Component`, `@Controller`, `@Repository`), aunque no es lo recomendable, ya que en un futuro podría cambiar la funcionalidad de cada etiqueta y sería mejor nombrar a cada *Bean* como lo qué es (controlador, servicio, repositorio...)

La inyección de dependencias la podemos hacer sobre atributos de clases directamente (como en el ejemplo anterior), en el constructor, en un setter... ¿Dónde es más adecuado hacerlo? Si te fijas, IntelliJ (o el editor que estés utilizando) se queja diciendo que la inyección en los atributos no es recomendable. Utilizar `@Autowired` en los atributos de clase no es recomendable principalmente por dos razones relacionadas con pruebas unitarias:

- **Dificultad para Mocking:** Cuando se utiliza `@Autowired` en los atributos de clase, Spring realiza la inyección de dependencias automáticamente durante la inicialización del bean. Esto puede dificultar la escritura de pruebas unitarias efectivas porque tus pruebas tendrían que depender de instancias reales de los beans inyectados.

- **Control Explícito en las Pruebas:** En cambio, al utilizar inyección de dependencias a través de constructores o métodos setters explícitos en lugar de `@Autowired` en los atributos, tienes un mayor control sobre lo que se inyecta en las pruebas. Puedes usar mocks o stubs específicos para simular el comportamiento de las dependencias y probar componentes de manera aislada.

Por lo tanto, vamos a modificar nuestro controlador para hacer la DI en el constructor:

```
1  @RequestMapping(ProductController.URL)
2  @RestController
3  public class ProductController {
4      public static final String URL = "/products";
5      private final ProductService productService;
6
7      @Autowired
8      public ProductController(ProductService productService) {
9          this.productService = productService;
10     }
11
12     @GetMapping
13     public ResponseEntity<Void> getAll() {
14         productService.getAll();
15         return ResponseEntity.ok()
16             .body("Hello World!");
17     }
18 }
```

Además, desde la versión 4.3 de Spring la anotación `@Autowired` ya no es necesaria explícitamente para la inyección de dependencias si se aplica únicamente en un constructor. Spring automáticamente detecta y utiliza el constructor para realizar la inyección de dependencias.

Por último, si utilizamos Lombok (<https://projectlombok.org/>), podremos incluso ahorrarnos escribir el código del constructor con la etiqueta `@RequiredArgsConstructor`:

```
1  @RequestMapping(ProductController.URL)
2  @RestController
3  @RequiredArgsConstructor
4  public class ProductController {
5      public static final String URL = "/products";
6      private final ProductService productService;
7
8      @GetMapping
9      public ResponseEntity<Void> getAll() {
10         productService.getAll();
11         return ResponseEntity.ok()
12             .body("Hello World!");
13     }
14 }
```

De esta forma, nuestro código queda más limpio y simple y podemos inyectar nuestras dependencias mockeadas en los tests sin problemas:

```
1  @WebMvcTest(ProductController.class)
2  class ProductControllerTest {
3
4      @Autowired
5      protected MockMvc mockMvc;
6
7      @MockBean
8      private ProductService productService;
9
10     @Test
11     void getAll() throws Exception {
12         doNothing().when(productService).getAll();
13         mockMvc.perform(get("/products"))
14             .andExpect(status().isOk())
15             .andExpect(content().string("Hello World!"));
16     }
17 }
```

¿Qué pasa si tenemos más de una implementación de una interfaz? En ese caso, podemos utilizar la etiqueta `Qualifier` (<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/beans/factory/annotation/Qualifier.html>) para indicar qué implementación concreta queremos inyectar.

Spring se puede configurar (creación de *Beans*, inyección de dependencias...) de varias formas. Las 2 formas habituales son mediante anotaciones, como haremos nosotros durante el curso, o documentos XML.

2023/07/19 08:15(.) · cesguiro

📁 clase/daw/dws/1eval/spring.txt 🕒 Última modificación: 2024/12/16 21:13 por 217.113.194.21

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-NonCommercial-Share Alike 4.0 International

03 - API REST

Cualquier aplicación web se basa en una arquitectura cliente-servidor, donde un servidor queda a la espera de conexiones de clientes, y los clientes se conectan a los servidores para solicitar ciertos recursos.

Estas comunicaciones se realizan mediante un protocolo llamado **HTTP** (o **HTTPS**, en el caso de comunicaciones seguras). En ambos casos, cliente y servidor se envían cierta información estándar, en cada mensaje:

- Los clientes envían al servidor los datos del recurso que solicitan, junto con cierta información adicional, como por ejemplo las cabeceras de petición (información relativa al tipo de cliente o navegador, contenido que acepta, etc), y parámetros adicionales llamados normalmente datos del formulario.
- Por lo que respecta a los servidores, aceptan estas peticiones, las procesan y envían de vuelta algunos datos relevantes, como un código de estado (indicando si la petición pudo ser atendida satisfactoriamente o no), cabeceras de respuesta (indicando el tipo de contenido enviado, tamaño, idioma, etc), y el recurso solicitado propiamente dicho, si todo ha ido correctamente.

Para solicitar los recursos, los clientes se conectan o solicitan una determinada **URL** (). (siglas en inglés de *localización uniforme de recursos*, **Uniform Resource Location**). Esta **URL** () consiste en una línea de texto con tres secciones diferenciadas:

- El **protocolo** empleado (HTTP o HTTPS)
- El **nombre de dominio**, que identifica al servidor y lo localiza en la red.
- La **ruta** hacia el recurso solicitado, dentro del propio servidor. Esta última parte también suele denominarse **URI** () (identificador uniforme de recurso, o en inglés, Uniform Resource Identifier). Esta **URI** () identifica unívocamente el recurso buscado entre todos los demás recursos que pueda albergar el servidor.

Por ejemplo, la siguiente podría ser una **URL** () válida:

`http://miservidor.com/books?id=123` (`http://miservidor.com/books?id=123`)

El protocolo empleado es *http*, y el nombre de dominio es *miservidor.com*. Finalmente, la ruta o **URI** () es *books?id=123*, y el texto tras el interrogante '?' es la información adicional llamada datos del formulario. Esta información permite aportar algo más de información sobre el recurso solicitado. En este caso, podría hacer referencia al código del libro que estamos buscando.

Dependiendo de cómo se haya implementado el servidor, también podríamos reescribir esta **URI** () de este otro modo, con el mismo significado:

<http://miservidor.com/books/123> (<http://miservidor.com/books/123>)

Los servicios REST

REST son las siglas de **REpresentational State Transfer**, y designa un estilo de arquitectura de aplicaciones distribuidas, como las aplicaciones web. En un sistema *REST*, identificamos cada recurso a solicitar con una **URL** (identificador uniforme de recurso), y definimos un conjunto delimitado de comandos o métodos a realizar, que típicamente son:

- **GET**: para obtener resultados de algún tipo (listados completos o filtrados por alguna condición)
- **POST**: para realizar inserciones o añadir elementos en un conjunto de datos
- **PUT**: para realizar modificaciones o actualizaciones del conjunto de datos
- **DELETE**: para realizar borrados del conjunto de datos

Existen otros tipos de comandos o métodos, como por ejemplo **PATCH** (similar a *PUT*, pero para cambios parciales), **HEAD** (para consultar sólo el encabezado de la respuesta obtenida), etc. Nos centraremos, no obstante, en los cuatro métodos principales anteriores.

Por lo tanto, identificando el recurso a solicitar y el comando a aplicarle, el servidor que ofrece esta *API* *REST* proporciona una respuesta a esa petición. Esta respuesta típicamente viene dada por un mensaje en formato **JSON** o **XML** (aunque éste último cada vez está más en desuso).

Durante la realización de la mayoría de las prácticas del curso, usaremos una aplicación llamada Postman (<https://www.postman.com/>), que permite construir peticiones de diferentes tipos, empleando distintos tipos de comandos, cabeceras y contenidos, enviarlas al servidor que indiquemos y examinar la respuesta proporcionada por éste.

JSON

JSON son las siglas de **JavaScript Object Notation**, una sintaxis propia de Javascript para poder representar objetos como cadenas de texto, y poder así serializar y enviar información de objetos a través de flujos de datos (archivos de texto, comunicaciones cliente-servidor, etc).

Un objeto Javascript se define mediante una serie de propiedades y valores. Por ejemplo, los datos de una persona (como nombre y edad) podríamos almacenarlos así:

```
1 let persona = {  
2   nombre: "Nacho",  
3   edad: 39  
4 };
```

Este mismo objeto, convertido a JSON, formaría una cadena de texto con este contenido:

```
1  {  
2      "nombre": "Nacho",  
3      "edad": 39  
4  }
```

Del mismo modo, si tenemos una colección (vector) de objetos como ésta:

```
1  let personas = [  
2      {  
3          nombre: "Nacho",  
4          edad: 39  
5      },  
6      {  
7          nombre: "Mario",  
8          edad: 4  
9      },  
10     {  
11         nombre: "Laura",  
12         edad: 2  
13     },  
14     {  
15         nombre: "Nora",  
16         edad: 10  
17     }  
18 ];
```

Transformada a JSON sigue la misma sintaxis, pero entre corchetes:

```
1  [  
2      {  
3          "nombre": "Nacho",  
4          "edad": 39  
5      },  
6      {  
7          "nombre": "Mario",  
8          "edad": 4  
9      },  
10     {  
11         "nombre": "Laura",  
12         "edad": 2  
13     },  
14     {  
15         "nombre": "Nora",  
16         "edad": 10  
17     }  
18 ]
```

Códigos de estado

Los códigos de estado de la respuesta, depende del resultado de la operación que se haya realizado. Se catalogan en cinco grupos:

- **Códigos 1XX:** representan información sobre una petición normalmente incompleta. No son muy habituales, pero se pueden emplear cuando la petición es muy larga, y se envía antes una cabecera para comprobar si se puede procesar dicha petición.
- **Códigos 2xx:** representan peticiones que se han podido atender satisfactoriamente. El código más habitual es el 200, respuesta estándar para las peticiones que son correctas. Existen otras variantes, como el

código 201, que se envía cuando se ha insertado o creado un nuevo recurso en el servidor (una inserción en una base de datos, por ejemplo), o el código 204, que indica que la petición se ha atendido bien, pero no se ha devuelto nada como respuesta.

- **Códigos 3xx:** son códigos de redirección, que indican que de algún modo la petición original se ha redirigido a otro recurso del servidor. Por ejemplo, el código 301 indica que el recurso solicitado se ha movido permanentemente a otra [URL\(\)](#). El código 304 indica que el recurso solicitado no ha cambiado desde la última vez que se solicitó, por si se quiere recuperar de la caché local en ese caso.
- **Códigos 4xx:** indican un error por parte del cliente. El más típico es el error 404, que indica que estamos solicitando una [URL\(\)](#) o recurso que no existe. Pero también hay otros habituales, como el 401 (cliente no autorizado), o 400 (los datos de la petición no son correctos, por ejemplo, porque los campos del formulario no sean válidos).
- **Códigos 5xx:** indican un error por parte del servidor. Por ejemplo, el error 500 indica un error interno del servidor, o el 504, que es un error de timeout por tiempo excesivo en emitir la respuesta.

En este enlace (<https://www.ibm.com/docs/es/odm/8.5.1?topic=api-rest-response-codes-error-messages>) puedes ver los diferentes código de estado y su explicación.

Haremos uso de estos códigos de estado en nuestros servicios REST para informar al cliente del tipo de error que se haya producido, o del estado en que se ha podido atender su petición.

Endpoints

un **endpoint** se refiere a una [URL\(\)](#) específica a la que se puede realizar una solicitud HTTP (GET, POST, PUT, DELETE...) para interactuar con un recurso o conjunto de recursos. Cada endpoint está asociado con una operación y puede devolver o manipular datos en formato JSON, XML u otro formato, según lo definido por la [API\(\)](#).

Hay una serie de buenas prácticas a la hora de crear tus endpoints. Algunas de ellas son:

- Deberían siempre ser sustantivos, nunca verbos. El tipo de acción (el verbo) viene determinado por el método HTTP (GET, PUT, POST, DELETE...).
- Los recursos tienen que estar en plural, aunque se devuelva uno sólo.
- Para relacionar recursos, se van anidando recursos específicos en la [URL\(\)](#).

Vamos a ver varios ejemplos para clarificar lo anterior:

- GET /books - Devuelve el listado de todos los libros.
- GET /books/12 - Devuelve el libro con id 12.
- POST /books - Crea un nuevo libro
- PUT /books/12 - Actualiza el libro con id 12.

- DELETE /books/12 - Elimina el libro con id 12
- GET /books/12/authors - Devuelve el listado de autores del libro con id 12.

Más adelante ya veremos como enviar los datos que tenemos que insertar o borrar.

Además, los endpoints pueden llevar parámetros opcionales. Por ejemplo, el siguiente endpoint devolvería la página 3 del listado de libros:

GET /books?page=3

📄 clase/daw/dws/1eval/api.txt 📅 Última modificación: 2025/09/16 11:39 por cesguiro

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-Noncommercial-Share Alike 4.0 International

04 - Arquitectura de software

La arquitectura de software es uno de los pilares fundamentales en el desarrollo de aplicaciones y sistemas complejos. Representa la estructura organizacional de un sistema, describiendo cómo sus componentes interactúan entre sí y con su entorno. La arquitectura no solo define el diseño técnico, sino que también influye en decisiones clave que afectan la escalabilidad, el rendimiento, la mantenibilidad y la flexibilidad a lo largo del ciclo de vida del software.

Con el avance de la tecnología y la evolución de las necesidades empresariales, la arquitectura de software ha adoptado múltiples enfoques, como el monolítico, microservicios o arquitecturas orientadas a eventos. Cada una de estas soluciones responde a diferentes desafíos y ofrece ventajas específicas, dependiendo del contexto y los objetivos del proyecto.

A lo largo de este tema, exploraremos los conceptos clave, los principios fundamentales y los tipos más comunes de arquitecturas, proporcionando una base sólida para entender cómo estructurar un sistema de software de manera eficiente y escalable.

Arquitectura de software

La arquitectura de software se refiere al conjunto de decisiones fundamentales sobre la estructura y el comportamiento de un sistema de software. Estas decisiones incluyen la organización de los componentes del sistema, cómo interactúan entre sí, y las restricciones y directrices que afectan dichas interacciones. En términos más simples, la arquitectura de software es el diseño general de alto nivel que establece el marco para el desarrollo de una aplicación.

Los Componentes claves de una arquitectura incluyen:

- **Componentes:** Son las piezas funcionales que forman la aplicación, como módulos, servicios o bases de datos.
- **Conectores:** Definen cómo los componentes se comunican entre sí, utilizando protocolos como HTTP, `API()`, llamadas a procedimientos, etc.
- **Relaciones:** Determinan las interacciones entre los componentes y los conectores, estableciendo flujos de datos y dependencias.

La arquitectura de software es crucial porque permite gestionar la complejidad de los sistemas modernos dividiendo el sistema en partes más manejables. Facilita la toma de decisiones técnicas que guían aspectos como las tecnologías y lenguajes a utilizar, asegurando que el sistema sea escalable y adaptable a futuras necesidades. Además, influye directamente en la calidad del software, mejorando su rendimiento, mantenibilidad, flexibilidad y seguridad.

Objetivos

El principal objetivo de la arquitectura de software es proporcionar una estructura sólida y organizada para el desarrollo de sistemas, de modo que se optimice la eficiencia y la calidad del producto final. A través de una planificación cuidadosa, se asegura que el sistema pueda escalar, evolucionar y mantenerse de manera efectiva a lo largo del tiempo.

Los objetivos específicos de una buena arquitectura incluyen:

- **Gestión de la complejidad:** La arquitectura permite dividir el sistema en módulos o componentes más pequeños, lo que facilita su comprensión, desarrollo y mantenimiento. Esta segmentación ayuda a reducir la complejidad inherente de sistemas grandes y distribuidos.
- **Escalabilidad:** Un sistema bien estructurado puede crecer en tamaño o capacidad sin perder eficiencia ni comprometer su rendimiento. La arquitectura debe permitir agregar nuevos componentes o mejorar los existentes sin necesidad de rehacer todo el sistema.
- **Mantenibilidad:** A medida que el software evoluciona, la arquitectura facilita la incorporación de cambios y nuevas funcionalidades sin generar errores o comprometer el rendimiento. Un diseño modular bien estructurado es clave para mantener la calidad del software a lo largo de su ciclo de vida.
- **Flexibilidad:** Los sistemas deben poder adaptarse a cambios futuros, ya sea en los requisitos funcionales o en las tecnologías utilizadas. Una arquitectura flexible permite reemplazar componentes o integrarse con nuevos sistemas sin afectar otras partes del software.

Además de estos objetivos, la arquitectura debe tener en cuenta aspectos como la seguridad, el rendimiento y la reutilización de componentes, lo que garantiza que el sistema cumpla con los requisitos actuales y futuros de los usuarios y las empresas.

Principios clave de una buena arquitectura

Una arquitectura de software eficaz se fundamenta en una serie de principios que aseguran la calidad, flexibilidad y mantenibilidad del sistema a lo largo del tiempo. Estos principios sirven como guía para tomar decisiones técnicas que beneficien tanto el desarrollo inicial como la evolución futura del software.

Entre los principios clave destacan:

- **Separación de responsabilidades:** Cada componente o módulo del sistema debe tener una única responsabilidad clara y bien definida. Esto facilita la comprensión del sistema, permite el aislamiento de fallos y hace más sencillo el desarrollo y mantenimiento. Aplicar este principio reduce el

acoplamiento entre componentes, lo que permite que los cambios en una parte del sistema no afecten negativamente a otras.

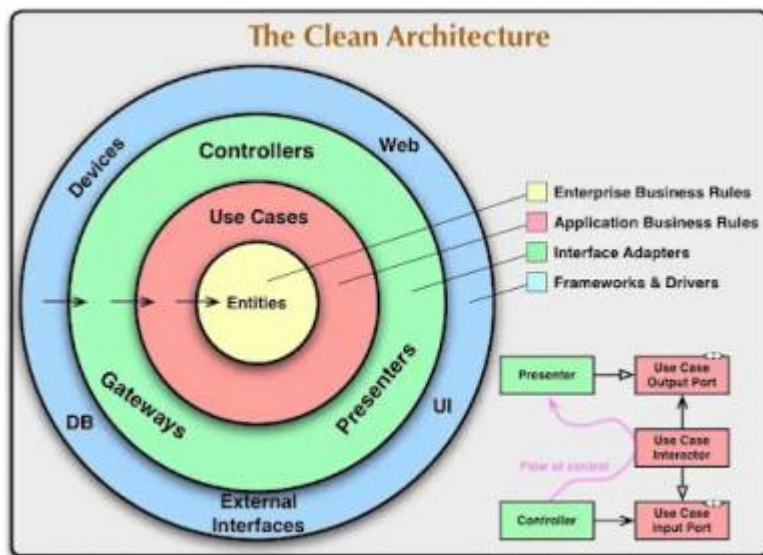
- **Independencia tecnológica:** La arquitectura debe estar diseñada de manera que los componentes centrales del sistema no dependan de una tecnología específica. Esto permite reemplazar tecnologías o plataformas (como bases de datos o frameworks) sin necesidad de modificar la lógica de negocio, lo que otorga flexibilidad a largo plazo.
- **Modularidad:** Dividir el sistema en módulos independientes mejora su organización y facilita el trabajo en equipo. Cada módulo puede desarrollarse, probarse y desplegarse por separado, lo que mejora la escalabilidad y el tiempo de respuesta ante cambios o problemas.
- **Acoplamiento bajo y alta cohesión:** El acoplamiento bajo implica que los componentes del sistema tienen pocas dependencias entre sí, lo que reduce el riesgo de que un cambio en un componente afecte a otros. Por otro lado, la alta cohesión significa que los componentes están organizados en torno a una responsabilidad específica, lo que mejora la claridad del código y facilita su mantenimiento.
- **Escalabilidad y rendimiento:** Desde su diseño inicial, la arquitectura debe considerar cómo manejará un incremento en el número de usuarios, datos o interacciones. Esto implica planificar para optimizar el rendimiento del sistema a medida que crece, utilizando principios como el balanceo de carga, la replicación de datos y el diseño orientado a la concurrencia.

Al seguir estos principios, se asegura que la arquitectura no solo sea funcional en su etapa inicial, sino que también pueda evolucionar, adaptarse a cambios y mantener una alta calidad a lo largo de su ciclo de vida.

Arquitectura limpia

La arquitectura limpia es un enfoque que se centra en mantener la independencia de los componentes de un sistema de software. Propuesta por Robert C. Martin (Uncle Bob) (<https://blog.cleancoder.com/>), este modelo organiza el software en capas que separan las reglas de negocio de los detalles de implementación, como la interfaz de usuario o la base de datos.

El principio básico de la arquitectura limpia es que el núcleo del sistema debe estar compuesto por la lógica de negocio pura, es decir, por las entidades y casos de uso. Los detalles tecnológicos, como bases de datos, frameworks o interfaces, son considerados periféricos y deben poder cambiarse sin afectar el núcleo.



Este enfoque tiene varios beneficios:

- **Independencia del framework:** El sistema no depende de un framework específico, por lo que puede evolucionar y adaptarse fácilmente a nuevas tecnologías.
- **Independencia de la UI:** Las interfaces de usuario, como aplicaciones web o móviles, pueden cambiar sin modificar la lógica central.
- **Independencia de la base de datos:** La lógica de negocio no está atada a una base de datos concreta, lo que facilita cambiar el sistema de almacenamiento sin grandes ajustes.
- **Facilidad para realizar pruebas:** Al estar desacoplados los detalles externos de la lógica de negocio, las pruebas unitarias son más sencillas y confiables.

A partir de este enfoque, se han desarrollado varias arquitecturas derivadas que comparten principios similares, como la arquitectura por capas, la arquitectura hexagonal y la arquitectura de cebolla.

Tipos de arquitecturas

Existen varios tipos de arquitecturas de software, cada una con características y ventajas específicas que se ajustan a diferentes escenarios y necesidades de desarrollo:

- **Arquitectura monolítica:** Es una de las arquitecturas más tradicionales, donde todo el sistema se desarrolla como un único bloque o aplicación. Si bien es sencilla de implementar al inicio, puede volverse difícil de mantener y escalar a medida que el sistema crece.
- **Arquitectura por capas:** Este tipo de arquitectura organiza el sistema en diferentes capas, donde cada capa tiene una responsabilidad específica. Normalmente incluye capas de presentación, lógica de negocio y acceso a datos, entre otras. Aunque es muy popular, puede generar dependencias rígidas entre capas si no se gestiona adecuadamente.

- **Arquitectura de microservicios:** En lugar de construir un sistema como una única unidad, los microservicios dividen el sistema en pequeños servicios independientes que se comunican entre sí. Cada servicio tiene su propio ciclo de vida y puede desarrollarse, desplegarse y escalarse de forma independiente.
- **Arquitectura hexagonal (Ports and Adapters):** Fomenta la separación de la lógica de negocio del mundo externo, como interfaces de usuario o bases de datos, utilizando “puertos” (interfaces) y “adaptadores” (implementaciones).
- **Arquitectura de cebolla:** Similar a la arquitectura limpia, organiza el sistema en capas concéntricas, donde el núcleo es la lógica de negocio, y las capas externas manejan detalles tecnológicos.
- **Arquitectura orientada a eventos:** Se centra en la producción, detección, consumo y reacción a eventos dentro de un sistema. Es útil para sistemas distribuidos y altamente escalables.
- **Serverless:** En esta arquitectura, el desarrollador solo se preocupa por escribir código, mientras el proveedor de la nube gestiona el escalado y la infraestructura. Es útil para aplicaciones de alta disponibilidad y bajo costo de operación.

2024/09/23 09:39() · cesguiro

A lo largo de este curso, nos centraremos en desarrollar una arquitectura limpia por capas, aplicando los principios fundamentales de separación de responsabilidades, independencia tecnológica y modularidad. Este enfoque permitirá construir sistemas que no solo sean mantenibles y escalables, sino también flexibles ante cambios tecnológicos y de negocio. Exploraremos en detalle cómo estructurar cada capa, desde la lógica de negocio hasta las interfaces con los sistemas externos, siguiendo las mejores prácticas y patrones de diseño arquitectónico.

📄 [clase/daw/dws/1eval/arquitectura.txt](#) 📅 Última modificación: 2025/09/16 11:39 por cesguiro

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-Noncommercial-Share Alike 4.0 International

05 - Arquitectura por capas

La arquitectura de capas es un patrón de diseño de software que organiza el sistema en diferentes niveles o capas. Cada capa tiene una responsabilidad clara y definida, y las capas interactúan entre sí de manera jerárquica. Las capas superiores utilizan los servicios ofrecidos por las capas inferiores, pero las capas inferiores no dependen de las superiores.

Se caracteriza por la separación de responsabilidades, donde cada capa está encargada de una funcionalidad específica del sistema. Esto permite una mejor organización del código y facilita la mantenibilidad y escalabilidad del mismo.

Además, presenta ventajas como la reutilización de código y la posibilidad de realizar pruebas de manera más efectiva. Sin embargo, también tiene desventajas, como una posible sobrecarga en la comunicación entre capas y una complejidad adicional en la gestión de las interacciones.

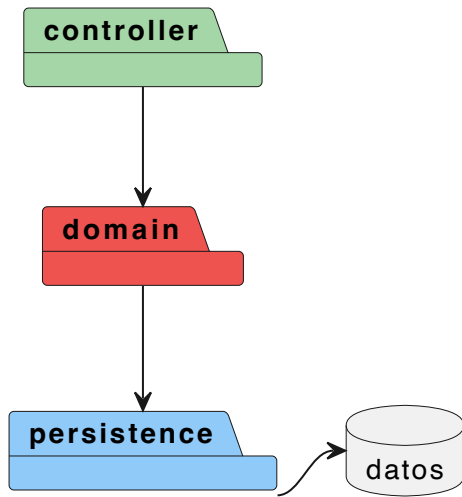
Capas

Las capas básicas de la arquitectura de la aplicación web son la capa de **presentación**, la capa de **dominio** y la capa de **persistencia**.

La capa de **presentación** (en nuestro caso le llamaremos **controller**, ya que es dónde estarán nuestros controladores) es responsable de interactuar con el usuario. Aquí es donde se manejan las solicitudes y respuestas, y se presenta la información de una manera comprensible. En esta capa, se implementan los controladores que gestionan la lógica de la interfaz y coordinan la interacción entre el usuario y el sistema.

La capa de **dominio** contiene la lógica de negocio de la aplicación. Está compuesta por los servicios que procesan la información y las entidades del modelo que representan los datos fundamentales. Esta capa se encarga de las reglas de negocio y la validación de los datos antes de que sean enviados a la capa de persistencia.

La capa de **persistencia** se encarga de la gestión del almacenamiento de datos. Aquí se implementan los repositorios que facilitan la interacción con los datos, permitiendo realizar operaciones de creación, lectura, actualización y eliminación (CRUD) sobre las entidades del modelo.



Además de estas capas básicas, es posible incorporar más capas según las necesidades de la aplicación. Por ejemplo, se pueden incluir capas de servicio adicional para la integración con servicios externos, capas de presentación para distintos tipos de interfaces (como APL() REST y aplicaciones web), o capas de configuración y seguridad. Esta flexibilidad permite adaptar la arquitectura a los requisitos específicos de cada proyecto.

En el tema anterior, cuando hablamos de las arquitecturas limpias vimos que el núcleo del sistema debería estar compuesto por los modelos y las reglas de negocio y que debería ser independiente de los detalles tecnológicos, como bases de datos, frameworks...

En el esquema anterior, ésto no se cumple, ya que, en realidad, todo depende de la última capa, la de persistencia. A lo largo de curso, ya iremos viendo mecanismos para solucionar esta situación

Capa de presentación

La capa de presentación es responsable de interactuar con el cliente y gestionar las solicitudes y respuestas de la aplicación.

Los controladores reciben las solicitudes HTTP, procesan los datos y devuelven respuestas adecuadas. Utilizan los servicios de la capa de dominio para llevar a cabo la lógica de negocio necesaria.

En *Spring* es habitual tener en esta capa nuestros controladores y el sistema de enrutamiento. Es común utilizar anotaciones en esta capa, como:

- **@RestController**: Esta anotación indica que la clase es un controlador y que las respuestas de los métodos se serializan automáticamente en formato JSON o XML.
- **@RequestMapping**: Se utiliza para mapear las solicitudes HTTP a los métodos del controlador.
- **@GetMapping, @PostMapping, @PutMapping, @DeleteMapping**: Estas anotaciones específicas permiten definir los métodos que manejarán las

solicitudes HTTP correspondientes a las operaciones de lectura, creación, actualización y eliminación, respectivamente.

```
1  @RestController
2  @RequestMapping(BookController.URL)
3  @RequiredArgsConstructor
4  public class BookController {
5
6      public static final String URL = "/books";
7
8      private final BookService bookService;
9
10     @GetMapping
11     public List<Book> getAll() {
12         return bookService.getAll();
13     }
14
15     @GetMapping("/{isbn}")
16     public Book findByIsbn(@PathVariable String isbn) {
17         return bookService.findByIsbn(isbn);
18     }
19 }
```

Por ejemplo, los métodos *getAll()* y *findByIsbn()* manejan las solicitudes para obtener todos los libros y para encontrar un libro específico por su ISBN, respectivamente. Ambos métodos utilizan el servicio *BookService* para realizar la lógica de negocio, manteniendo así una clara separación entre la presentación y la lógica del dominio.

Capa de dominio

Es el componente principal de nuestra aplicación, donde estarán nuestras modelos y reglas de negocio.

Esta capa no debería tener ninguna dependencia con componentes externos, con lo que deberemos buscar mecanismos para evitar introducir referencias a Spring, bbdd, JSON...

```
1 public class Book {
2 package es.cesguiro.model;
3
4 import java.math.BigDecimal;
5 import java.math.RoundingMode;
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public class Book {
10
11     private String isbn;
12     private String titleEs;
13     private String titleEn;
14     private String synopsisEs;
15     private String synopsisEn;
16     private BigDecimal basePrice;
17     private double discountPercentage;
18     private BigDecimal price;
19     private String cover;
20     private LocalDate publicationDate;
21     private Publisher publisher;
22     private List<Author> authors;
23
24     // Constructores, getters, setters, lógica de negocio,
25 }

```

```
1 package es.cesguiro.service;
2
3 import java.util.List;
4 import java.util.Optional;
5
6 public interface BookService {
7
8     List<Book> getAll(int page, int size);
9
10    //...
11
12 }

```

```
1 package es.cesguiro.service.impl;
2
3 import es.cesguiro.repository.BookRepository;
4 import es.cesguiro.service.BookService;
5
6 import java.util.List;
7 import java.util.Optional;
8
9 public class BookServiceImpl implements BookService {
10
11     private final BookRepository bookRepository;
12
13     public BookServiceImpl(BookRepository bookRepository)
14         this.bookRepository = bookRepository;
15     }
16
17     @Override
18     public List<Book> getAll(int page, int size) {
19         return bookRepository.findAll(page, size);
20     }
21
22     // ...
23 }

```

Capa de persistencia

La capa de persistencia es responsable de gestionar la interacción con el origen de datos (base de datos, ficheros, APIs...), permitiendo el almacenamiento y recuperación de los mismos.

```
1 public interface BookRepository {  
2     List<Book> getAll();  
3  
4 }  
  
1 public BookRepositoryImpl {  
2  
3     List<Book> getAll() {  
4         // Conectar con el origen de datos de nuestra aplicac  
5         // Recuperar y devolver listado de libros  
6     }  
7 }
```

En realidad, como veremos más adelante, los repositorios (concretamente las interfaces de los repositorios) pertenecen a la capa de dominio, ya que utilizaremos **inversión de dependencias** para que sea la capa de persistencia la que dependa de dominio

📄 clase/daw/dws/1eval/arquitectura_capas.txt 🕒 Última modificación: 2025/09/16 12:31 por cesguiro

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-Noncommercial-Share Alike 4.0 International

06 - Capa de dominio

En una arquitectura limpia, la capa de dominio es el corazón de la aplicación. Aquí es donde se encuentran las reglas de negocio y la lógica fundamental que define el comportamiento del sistema. Esta capa debe estar **completamente aislada de cualquier detalle de implementación o infraestructura, como bases de datos, frameworks o herramientas de persistencia**. Esto permite que los cambios en estos detalles no afecten la lógica central del negocio.

Para nuestra biblioteca online, vamos a crear un proyecto Maven que será nuestra capa de dominio. Las únicas dependencias que tendremos serán las de JUnit y Mockito para los tests:


```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-insta
4         xsi:schemaLocation="http://maven.apache.org/POM/4
5         <modelVersion>4.0.0</modelVersion>
6
7         <groupId>es.cesguiro</groupId>
8         <artifactId>daw2-bookstore-domain</artifactId>
9         <version>1.0-SNAPSHOT</version>
10        <packaging>jar</packaging>
11        <name>${project.groupId}.${project.artifactId}</name>
12        <description>Demo project for domain layer</descripti
13        <url>https://github.com/cesguiro/${project.artifactId}
14
15        <licenses>
16            <license>
17                <name>MIT License</name>
18                <url>http://www.opensource.org/licenses/mit-li
19            </license>
20        </licenses>
21
22        <developers>
23            <developer>
24                <name>César Guijarro</name>
25                <id>cesguiro</id>
26                <email>cesguirol@gmail.com</email>
27                <organization>cesguiro</organization>
28                <roles>
29                    <role>Architect</role>
30                    <role>Developer</role>
31                </roles>
32                <timezone>+1</timezone>
33                <url>https://cesguiro.es</url>
34            </developer>
35        </developers>
36
37        <properties>
38            <maven.compiler.source>21</maven.compiler.source>
39            <maven.compiler.target>21</maven.compiler.target>
40            <project.build.sourceEncoding>UTF-8</project.build
41            <org.junit.version>5.10.3</org.junit.version>
42            <org.mockito.version>5.12.0</org.mockito.version>
43            <org.apache.maven.plugins.version>3.5.2</org.apach
44        </properties>
45
46        <dependencies>
47            <dependency>
48                <groupId>org.junit.jupiter</groupId>
49                <artifactId>junit-jupiter</artifactId>
50                <version>${org.junit.version}</version>
51                <scope>test</scope>
52            </dependency>
53            <dependency>
54                <groupId>org.mockito</groupId>
55                <artifactId>mockito-junit-jupiter</artifactId>
56                <version>${org.mockito.version}</version>
57                <scope>test</scope>
58            </dependency>
59        </dependencies>
60
61        <build>
62            <plugins>
63                <!-- Plugin Surefire para ejecutar pruebas cor

```

```
64         <plugin>
65             <groupId>org.apache.maven.plugins</groupId>
66             <artifactId>maven-surefire-plugin</artifactId>
67             <version>${org.apache.maven.plugins.version}</version>
68         </plugin>
69     </plugins>
70 </build>
71
72 </project>
```

Inyección de dependencias en la capa de dominio

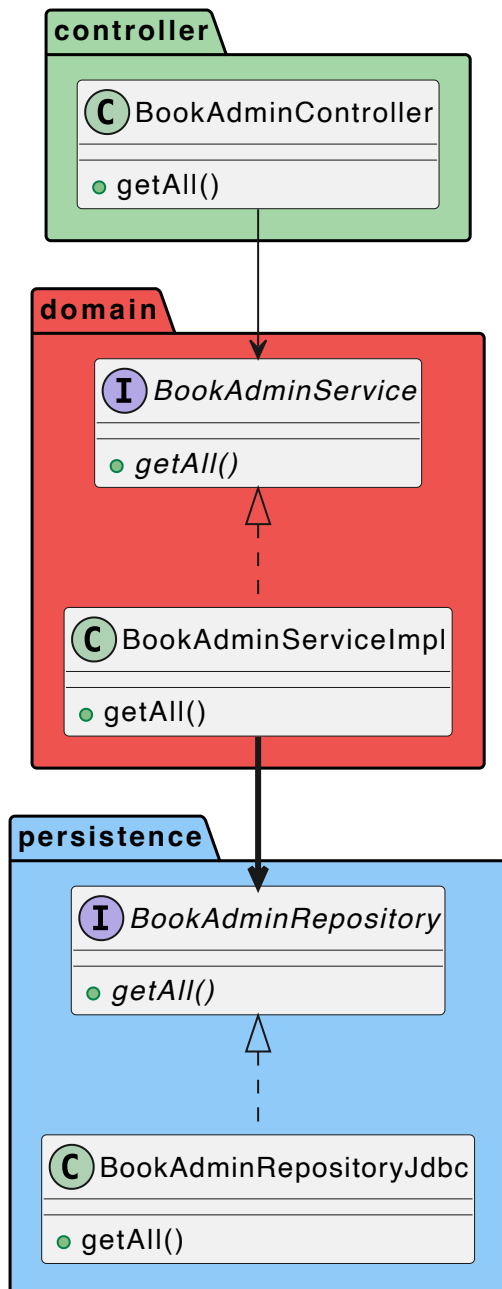
Para mantener la independencia del framework en la capa de dominio, es importante evitar depender directamente de herramientas o bibliotecas como Spring, incluso cuando este facilita mucho la inyección de dependencias con anotaciones como **@Service**. Si deseamos respetar los principios de una arquitectura limpia, debemos asegurarnos de que la capa de dominio no esté acoplada a ninguna tecnología externa.

La mayoría de frameworks (como Spring) tiene mecanismos para configurar y utilizar DI sin necesidad de añadir elementos externos en la capa de dominio. Cuando hagamos la integración de las capas, veremos como podemos manejar ésto con Spring.

Como hemos dicho, en esta capa (en teoría) no debería haber ningún *import* en ningún componente que no forme parte de la biblioteca básica de Java. En cualquier caso, existen librerías de terceros (como Lombok) que están muy extendidas y muchas veces se utilizan en esta capa. De ti depende decidir si vale la pena o no, teniendo siempre en cuenta que tú no tienes control sobre esas librerías.

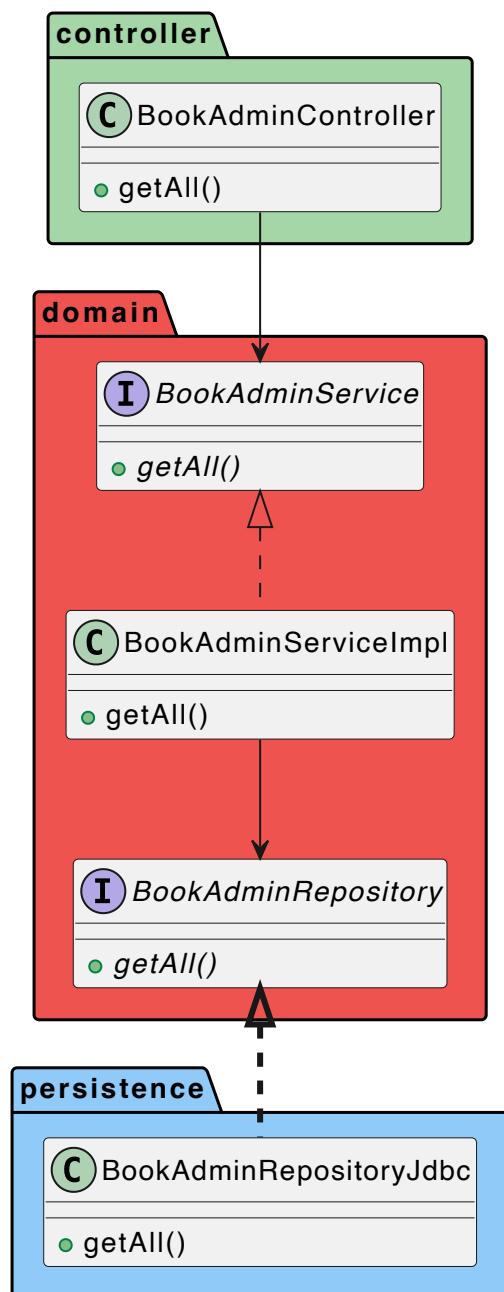
Inversión de dependencias

En el tema anterior vimos las relaciones entre capas. Al final, la capa más importante era la de persistencia, ya que todas dependían de ella. Si hiciésemos algún cambio en esa capa, el resto es probable que se viera afectado. Por ejemplo, un escenario típico sería:

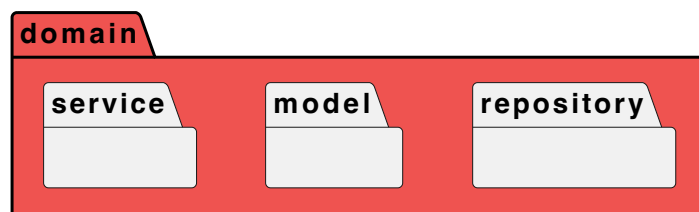


En una arquitectura bien estructurada, el flujo de dependencias debería ser inverso, es decir, la capa de persistencia debe depender de la capa de dominio, y no al revés.

Para abordar este problema, aplicamos el principio de **inversión de dependencias**. Esto implica mover las interfaces de los repositorios a la capa de dominio, permitiendo que la lógica de negocio dependa únicamente de estas interfaces. La implementación concreta de estas interfaces se mantendrá en la capa de persistencia. De esta manera, logramos desacoplar las capas y facilitar la adaptación a futuros cambios en la implementación de la persistencia sin afectar la lógica de dominio.



De esta forma, nuestra capa de dominio estará formada (por ahora) por los packages:



Modelos

Una de las primeras cosas que debemos hacer al crear una aplicación es definir los modelos. Por ejemplo, en nuestra biblioteca online podríamos definir los siguientes modelos:

```
1 public class Book {
2
3     private Long id;
4     private String isbn;
5     private String titleEs;
6     private String titleEn;
7     private String synopsisEs;
8     private String synopsisEn;
9     private BigDecimal basePrice;
10    private double discountPercentage;
11    private BigDecimal price;
12    private String cover;
13    private LocalDate publicationDate;
14    private Publisher publisher;
15    private List<Author> authors;
16
17    // Constructores, getters, setters

```

```
1 public class Author {
2
3     private Long id;
4     private String name;
5     private String nationality;
6     private String biographyEs;
7     private String biographyEn;
8     private int birthYear;
9     private Integer deathYear;
10    private String slug;
11
12    // Constructores, getters, setters

```

```
1 public class Publisher {
2
3     private Long id;
4     private String name;
5     private String slug;
6
7    // Constructores, getters, setters

```

Fíjate que nuestros modelos pueden tener campos que no existan en la bbdd, como **price** en *Book*, que será el precio final aplicando el descuento.

Transporte de datos entre capas

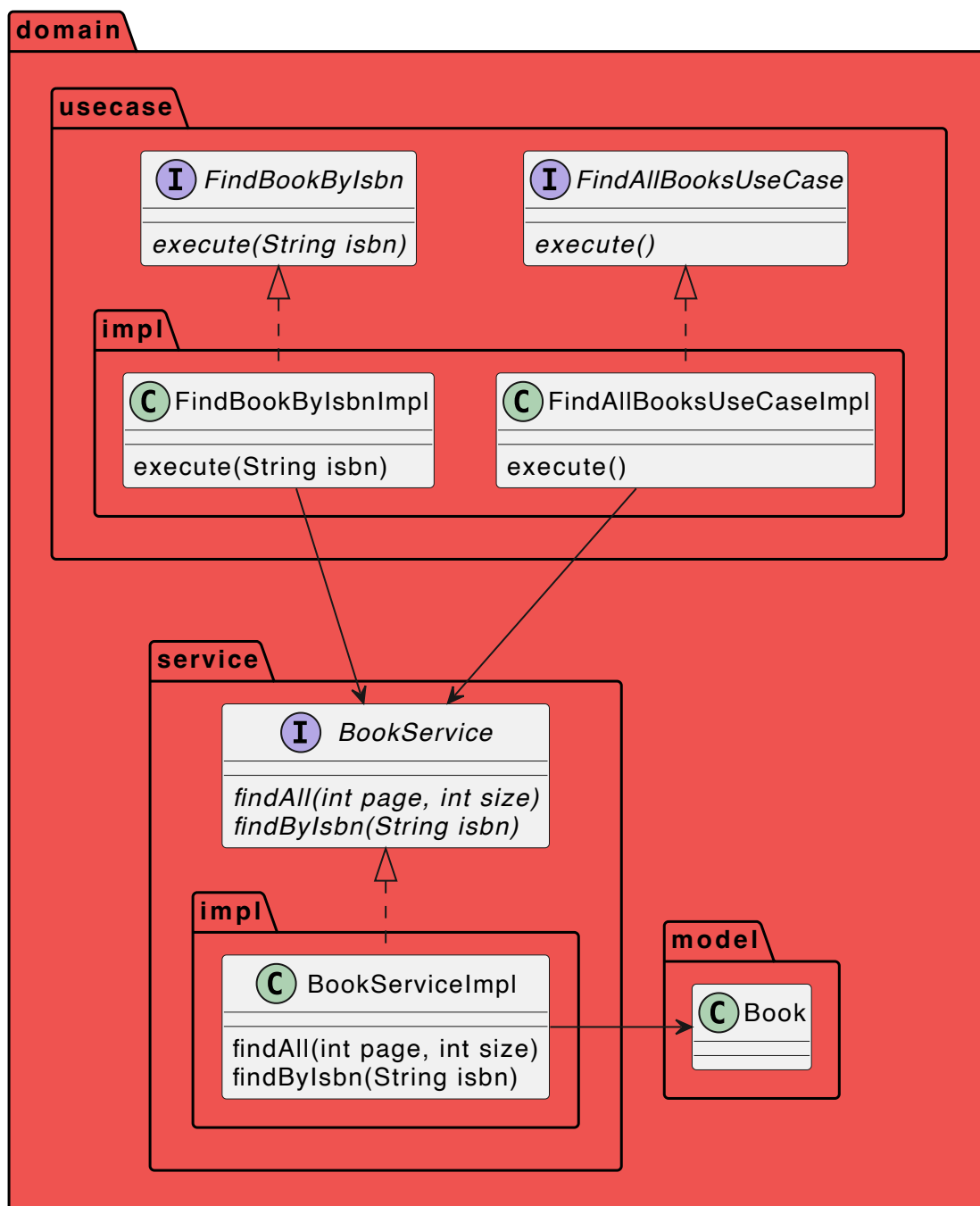
Otro punto importante es cómo transferimos información entre capas. Aquí surgen diferentes posibilidades, aunque podemos resumirlas en dos: usar modelos de negocio anémicos o ricos (o enriquecidos).

Modelos anémicos

Son objetos de dominio que no tienen lógica de negocio, sólo getters y setters. Están considerando un antipatrón por muchos autores.

El horror fundamental de este antipatrón es que contradice por completo la idea básica del diseño orientado a objetos, que consiste en combinar datos y procesos. El modelo de dominio anémico es en realidad un diseño de estilo procedimental, justo el tipo de cosa contra la que los fanáticos de los objetos como Eric y yo hemos luchado desde nuestros inicios en Smalltalk. Lo que es peor, mucha gente cree que los objetos anémicos son objetos reales y, por lo tanto, no comprenden en absoluto la esencia del diseño orientado a objetos. *Martin Fowler*
(<https://martinfowler.com/bliki/AnemicDomainModel.html>)

Con este tipo de modelos, metemos la lógica de negocio en los servicios. Incluso podemos usar casos de uso individuales, que haría uso de estos servicios:



La ventaja de este patrón es que compartimos el modelo de datos entre las capas, con lo que no tendremos que hacer mapeos entre ellas. Además, si queremos modificar el modelo, sólo debemos hacerlo en una clase.

Entre las desventajas, además de las señaladas por Martin Fowler y demás autores, está que, al compartir el mismo modelo entre capas, no podemos tener atributos diferentes entre ellas.

Estos modelos se convierten en simples DTOs.

Un **Data Transfer Object** es un objeto plano (POJO) con atributos, getters, setters y ninguna lógica de negocio. Se usan para transportar datos entre capas. Además, deben ser serializables, ya que también se utilizan, por ejemplo, para enviar datos del servidor al cliente. Por ejemplo, en Java, campos como `Date` o `Calendar` no

tienen una forma estándar para serializarse en REST, con lo que deberemos ocuparnos nosotros.

Modelos ricos o enriquecidos

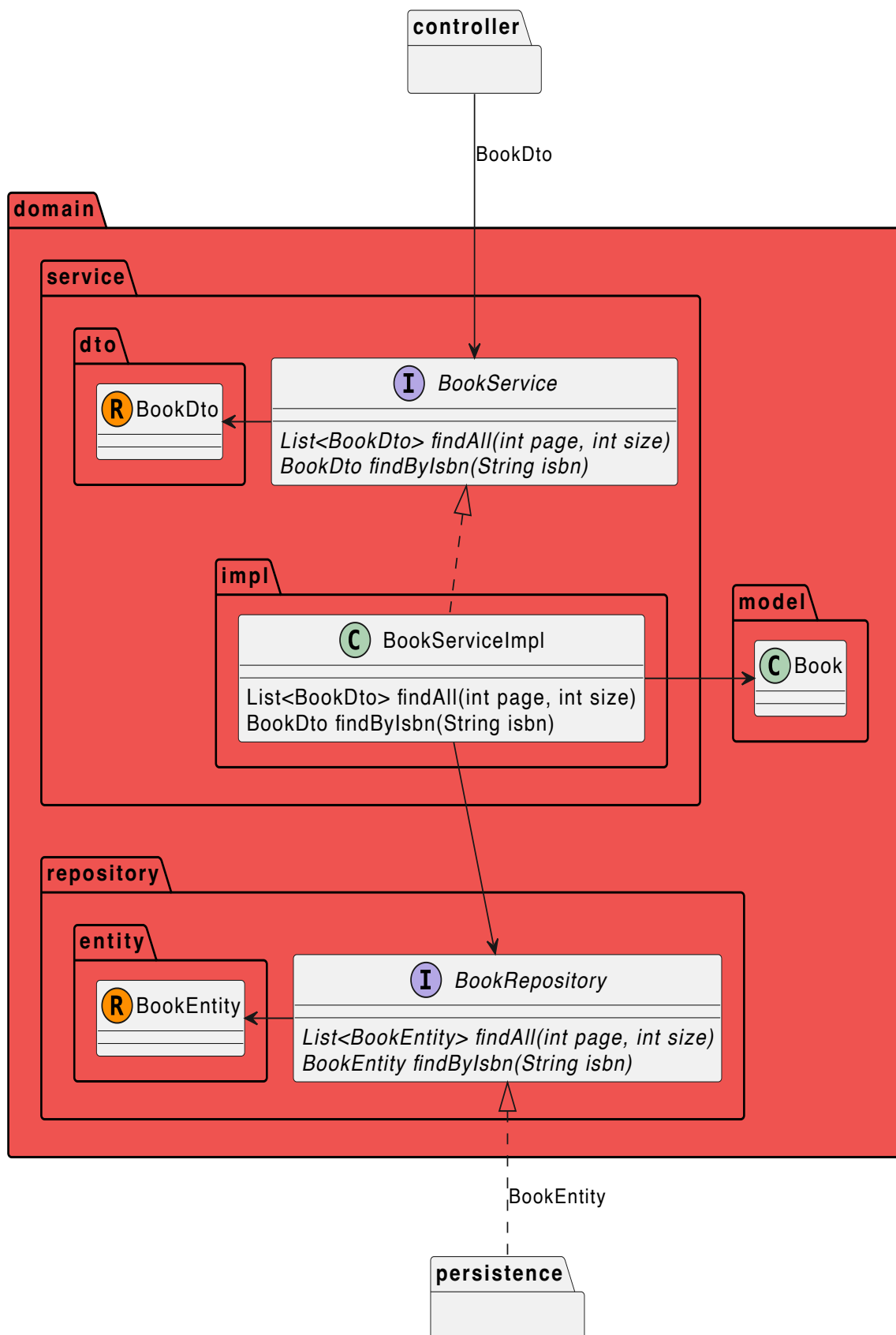
Al contrario que en los anémicos, estos modelos contienen lógica de negocio, con lo que son clases propiamente dichas. El problema fundamental es que, al contener lógica de negocio, no queremos exponer estos modelos fuera de los servicios, que son los únicos que deberían tratar con ellos, por lo tanto, necesitamos crear DTOs para transportar los datos entre los diferentes componentes.

Podríamos crear un DTO por modelo y usarlo para transportar datos entre dominio - presentación, dominio - persistencia y viceversa.

Otra opción, sería crear DTOs diferentes para transportar datos entre dominio - presentación y dominio - persistencia. De hecho, en nuestro caso es lo que vamos a hacer. Para ello, usaremos la clase Record (<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Record.html>) de Java.

La clase Record se introdujo de forma previa en JDK 14, y de forma definitiva en JDK 17. Básicamente son una implementación del patrón DTO. En general, son una forma de almacenar valores de forma inmutable y que, además, permiten la creación de objetos de forma sencilla, dado que sólo necesitamos especificar los atributos y el compilador se encarga de crear los gettes, constructor con todos los atributos, además de los métodos equals, hashCode y toString

Como hemos dicho, en nuestro caso crearemos dos conjuntos de DTOs, uno para comunicarnos con la capa de presentación (al que llamaremos modeloDTO) y otro con la de persistencia (modeloEntity).



Por ejemplo, en el caso de Book, tendríamos:

```
1  public class Book {
2
3      private Long id;
4      private String isbn;
5      private String titleEs;
6      private String titleEn;
7      private String synopsisEs;
8      private String synopsisEn;
9      private BigDecimal basePrice;
10     private double discountPercentage;
11     private BigDecimal price;
12     private String cover;
13     private LocalDate publicationDate;
14     private Publisher publisher;
15     private List<Author> authors;
16
17     public Book(
18         Long id,
19         String isbn,
20         String titleEs,
21         String titleEn,
22         String synopsisEs,
23         String synopsisEn,
24         BigDecimal basePrice,
25         double discountPercentage,
26         String cover,
27         LocalDate publicationDate,
28         Publisher publisher,
29         List<Author> authors
30     ) {
31         this.id = id;
32         this.isbn = isbn;
33         this.titleEs = titleEs;
34         this.titleEn = titleEn;
35         this.synopsisEs = synopsisEs;
36         this.synopsisEn = synopsisEn;
37         this.basePrice = basePrice;
38         this.discountPercentage = discountPercentage;
39         this.price = calculateFinalPrice();
40         this.cover = cover;
41         this.publicationDate = publicationDate;
42         this.publisher = publisher;
43         this.authors = authors;
44     }
45
46     // getters y setters
47
48     public BigDecimal calculateFinalPrice() {
49         BigDecimal discount = basePrice
50             .multiply(BigDecimal.valueOf(discountPerce
51             .divide(BigDecimal.valueOf(100), 2, Roundi
52
53         return basePrice.subtract(discount).setScale(2, Rc
54     }
55
56     public void addAuthor(Author author) {
57         this.authors.add(author);
58     }
59
60 }
```

Fíjate que hemos creado dos métodos, uno para calcular el precio final y otro para añadir un autor al libro

```
1 public record BookDto(  
2     Long id,  
3     String isbn,  
4     String titleEs,  
5     String titleEn,  
6     String synopsisEs,  
7     String synopsisEn,  
8     BigDecimal basePrice,  
9     double discountPercentage,  
10    BigDecimal price,  
11    String cover,  
12    LocalDate publicationDate,  
13    PublisherDto publisher,  
14    List<AuthorDto> authors  
15 ) {  
16 }
```

```
1 public record BookEntity(  
2     Long id,  
3     String isbn,  
4     String titleEs,  
5     String titleEn,  
6     String synopsisEs,  
7     String synopsisEn,  
8     BigDecimal basePrice,  
9     double discountPercentage,  
10    String cover,  
11    LocalDate publicationDate,  
12    PublisherEntity publisher,  
13    List<AuthorEntity> authors  
14 ) {  
15 }
```

Como puedes ver, los DTOs no son exactamente iguales. BookDto tiene el precio final (que calculamos cuando construimos el objeto Book) mientras que BookEntity no lo tiene.

Mapeadores

En nuestro caso, hemos optado por usar modelos enriquecidos y DTOs. Obviamente, tenemos que crear mecanismos para transformar los modelos entre ellos. Estos componentes son los **mapeadores**. Aunque existen librerías de terceros como MapStruct (<https://mapstruct.org/>) que nos facilitan la tarea, recuerda que estamos en la capa de dominio, con lo que no queremos añadir dependencias externas. En cualquier caso, aunque pueden ser engorrosos, el código no es nada complicado. Por ejemplo, podemos crear el mapeador de Book de la siguiente manera:

```
1  public class BookMapper {
2
3      private static BookMapper INSTANCE;
4
5      private BookMapper() {
6      }
7
8      public static BookMapper getInstance() {
9          if (INSTANCE == null) {
10              INSTANCE = new BookMapper();
11          }
12          return INSTANCE;
13      }
14
15      public Book fromBookEntityToBook(BookEntity bookEntity)
16          if (bookEntity == null) {
17              return null;
18          }
19          return new Book(
20              bookEntity.id(),
21              bookEntity.isbn(),
22              bookEntity.titleEs(),
23              bookEntity.titleEn(),
24              bookEntity.synopsisEs(),
25              bookEntity.synopsisEn(),
26              bookEntity.basePrice(),
27              bookEntity.discountPercentage(),
28              bookEntity.cover(),
29              bookEntity.publicationDate(),
30              PublisherMapper.getInstance().fromPublisher(bookEntity.publisher()),
31              bookEntity.authors().stream().map(AuthorMapper::fromAuthorEntity).collect(Collectors.toList());
32      );
33  }
34
35      public BookEntity fromBookToBookEntity(Book book) {
36          if (book == null) {
37              return null;
38          }
39          return new BookEntity(
40              book.getId(),
41              book.getIsbn(),
42              book.getTitleEs(),
43              book.getTitleEn(),
44              book.getSynopsisEs(),
45              book.getSynopsisEn(),
46              book.getBasePrice(),
47              book.getDiscountPercentage(),
48              book.getCover(),
49              book.getPublicationDate(),
50              PublisherMapper.getInstance().fromPublisher(book.getPublisher()),
51              book.getAuthors().stream().map(AuthorMapper::fromAuthorEntity).collect(Collectors.toList());
52      );
53  }
54
55      public BookDto fromBookToBookDto(Book book) {
56          if (book == null) {
57              return null;
58          }
59          return new BookDto(
60              book.getId(),
61              book.getIsbn(),
62              book.getTitleEs(),
63              book.getTitleEn(),
```

```

64         book.getSynopsisEs(),
65         book.getSynopsisEn(),
66         book.getBasePrice(),
67         book.getDiscountPercentage(),
68         book.calculateFinalPrice(),
69         book.getCover(),
70         book.getPublicationDate(),
71         PublisherMapper.getInstance().fromPublisher(book.getAuthors().stream().map(AuthorMapper::toAuthor)),
72     );
73 }
74
75
76
77     public Book fromBookDtoToBook(BookDto bookDto) {
78         if (bookDto == null) {
79             return null;
80         }
81         return new Book(
82             bookDto.id(),
83             bookDto.isbn(),
84             bookDto.titleEs(),
85             bookDto.titleEn(),
86             bookDto.synopsisEs(),
87             bookDto.synopsisEn(),
88             bookDto.basePrice(),
89             bookDto.discountPercentage(),
90             bookDto.cover(),
91             bookDto.publicationDate(),
92             PublisherMapper.getInstance().fromPublisher(bookDto.authors().stream().map(AuthorMapper::toAuthor)),
93         );
94     }
95 }
96

```

La idea es crearlo con el patrón *Singleton* para no tener que instanciar la clase cada vez que queramos utilizarlo (se encarga el método `getInstance()` de hacerlo por nosotros). De esta forma, podemos mapear objetos de forma sencilla:

```

1     public BookDto getByIsbn(String isbn) {
2         return bookRepository
3             .findByIsbn(isbn)
4             .map(BookMapper.getInstance()::fromBookEntityToBookDto)
5             .map(BookMapper.getInstance()::fromBookToBookDto)
6             .orElseThrow(() -> new BusinessException("Book not found"));
7     }

```

Servicios

Como siempre, usaremos interfaces para definir nuestros servicios, que serán los componentes con los que trabajará la capa de presentación (recuerda que devolvemos DTOs a la capa de presentación):

```
1 public interface BookService {  
2  
3     List<BookDto> getAll(int page, int size);  
4  
5     BookDto getByIsbn(String isbn);  
6  
7     Optional<BookDto> findByIsbn(String isbn);  
8  
9     BookDto create(BookDto bookDto);  
10  
11    BookDto update(BookDto bookDto);  
12  
13    void delete(String isbn);  
14  
15 }
```

Observa que tenemos dos métodos aparentemente iguales: *findByIsbn* y *getByIsbn*. Cuando lleguemos al punto de las excepciones, entenderás el porqué.

La implementación usará el repositorio para leer/escribir datos. Usaremos inyección de dependencias para inyectarle la implementación concreta del repositorio en el momento de creación del servicio:

```
1  public class BookServiceImpl implements BookService {
2
3      private final BookRepository bookRepository;
4
5      public BookServiceImpl(BookRepository bookRepository)
6          this.bookRepository = bookRepository;
7      }
8
9      @Override
10     public List<BookDto> getAll(int page, int size) {
11         return bookRepository
12             .findAll(page, size)
13             .stream()
14             .map(BookMapper.getInstance()::fromBoc
15             .map(BookMapper.getInstance()::fromBoc
16             .toList();
17     }
18
19     @Override
20     public BookDto getByIsbn(String isbn) {
21         return bookRepository
22             .findByIsbn(isbn)
23             .map(BookMapper.getInstance()::fromBookEnt
24             .map(BookMapper.getInstance()::fromBookToE
25             .orElseThrow(() -> new ResourceNotFoundExc
26     }
27
28     @Override
29     public Optional<BookDto> findByIsbn(String isbn) {
30         return bookRepository.findByIsbn(isbn)
31             .map(BookMapper.getInstance()::fromBookEnt
32             .map(BookMapper.getInstance()::fromBookToE
33     }
34
35     @Override
36     public BookDto create(BookDto bookDto) {
37         return null;
38     }
39
40     @Override
41     public BookDto update(BookDto bookDto) {
42         return null;
43     }
44
45     @Override
46     public void delete(String isbn) {
47
48     }
49 }
```

Puede que te estés preguntando cómo haremos para inyectar la implementación del repositorio, pero eso no es problema del servicio. Cuando montemos la aplicación completa verás como usamos Spring para tal fin.

Fíjate que mapeamos de BookEntity - Book - BookDto. Podrías pensar que ahorraríamos trabajo si mapeáramos directamente BookEntity - BookDto. El problema es que habrá validaciones o

campos calculados (como *price* en Book) que estarán en la lógica de Book. Si mapeáramos directamente BookEntity - BookDto perderíamos esas funcionalidades.

Repositorios

En este paquete, definiremos las interfaces de nuestros repositorios. Como en el caso de los servicios, no trabajaremos directamente con los modelos de dominio, sino con los Entity:

```
1 public interface BookRepository {  
2  
3     List<BookEntity> findAll(int page, int size);  
4  
5     Optional<BookEntity> findById(String isbn);  
6 }
```

¿Y las implementaciones? Eso le corresponde a la capa de persistencia, con lo que no nos preocuparemos por ahora.

Excepciones

El último paso será añadir excepciones a nuestros métodos.

Lo primero que tenemos que definir es qué es una excepción, dónde se lanzan y quién las trata. Una excepción es un mecanismo mediante el cual podemos controlar los errores producidos en tiempo de ejecución.

Por ejemplo, volvamos a los dos métodos similares de nuestro servicio:

```
1 @Override  
2 public BookDto getByIsbn(String isbn) {  
3     return bookRepository  
4         .findById(isbn)  
5         .map(BookMapper.getInstance()::fromBookEntityI  
6         .map(BookMapper.getInstance()::fromBookToBookI  
7         .orElseThrow(() -> new ResourceNotFoundException  
8     }  
9  
10 @Override  
11 public Optional<BookDto> findById(String isbn) {  
12     return bookRepository.findById(isbn)  
13         .map(BookMapper.getInstance()::fromBookEntityI  
14         .map(BookMapper.getInstance()::fromBookToBookI  
15 }
```

¿Por qué el primero lanza una excepción y el segundo no? La razón es sencilla; el primero lo utilizaremos en el caso de uso de recuperar los datos de un libro que asumimos que existe. Por ejemplo, cuando recibimos la petición GET /api/books/123456789. En ese caso, nuestra aplicación devolverá un 404 indicando que ese libro no existe.

Por el contrario, el segundo método lo podemos usar, por ejemplo, para hacer un buscador por diferentes campos, como el ISBN. Si no existe ningún libro con el ISBN pasado, no significa que es un error, simplemente no hay resultados.

En nuestro caso, por ahora crearemos dos tipos de excepciones:

```
1 public class BusinessException extends RuntimeException {  
2     public BusinessException(String message) {  
3         super(message);  
4     }  
5 }  
  
1 public class ResourceNotFoundException extends RuntimeException {  
2     public ResourceNotFoundException(String message) {  
3         super(message);  
4     }  
5 }
```

Aunque podríamos crear más tipos de excepciones (y puede que creemos más a lo largo del curso), simplifcaremos la tarea y usaremos la primera para cualquier error de la capa de negocio y la segunda para cuando no encontremos un recurso.

Repositorio

<https://github.com/cesguiro/daw2-bookstore-domain>
(<https://github.com/cesguiro/daw2-bookstore-domain>)

📄 clase/daw/dws/1eval/domain.txt 🕒 Última modificación: 2025/10/20 09:02 por cesguiro

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-Noncommercial-Share Alike 4.0 International

06_2 - Capa dominio: Test

Para comprobar que todo funciona correctamente, tenemos que usar tests automáticos obligatoriamente, ya que no podemos arrancar la aplicación (en realidad, da igual que tengamos un método main(), siempre deberíamos usar test para comprobar nuestros componentes).

Tests de los modelos

Nuestros modelos, a la larga, tendrán lógica de negocio que necesitamos probar. Por ejemplo, en nuestro modelo Book tenemos un test que calcula el precio final en función del precio base y el descuento:

```
1 public BigDecimal calculateFinalPrice() {
2     BigDecimal discount = basePrice
3         .multiply(BigDecimal.valueOf(discountPercentage
4         .divide(BigDecimal.valueOf(100), 2, RoundingMoc
5
6     return basePrice.subtract(discount).setScale(2, Roundir
7 }
```

Los test tienen que probar situaciones, no métodos. Es decir, tenemos que pensar diferentes situaciones que se puedan dar a la hora de calcular el precio final. Para ello, usaremos tests parametrizados para probar diferentes escenarios:

```
1 @ParameterizedTest(name = "{index} => basePrice={0}, discc
2 @DisplayName("Calculate final price with various discounts
3 @CsvSource({
4     "100.00, 15.0, 85.00",
5     "50.00, 0.0, 50.00",
6     "75.00, 100.0, 0.00",
7     "60.00, -10.0, 60.00"
8 })
9 void calculateFinalPrice(String basePrice, double discount
10     Book book = new Book(
11         1L,
12         "9999999999999999",
13         "Título en Español",
14         "Title in English",
15         "Sinopsis en Español",
16         "Synopsis in English",
17         new BigDecimal(basePrice),
18         discountPercentage,
19         "cover.jpg",
20         LocalDate.of(2023, 1, 1),
21         null,
22         null
23     );
24     BigDecimal expected = new BigDecimal(expectedPrice).se
25     assertEquals(expected, book.getPrice());
26 }
```

En este caso, vemos que el último test falla, cuando el descuento es negativo. Se supone que si el descuento es negativo, asumimos que es un error y que debería ser cero. De ahí la importancia de probar situaciones diferentes y TDD. Primero pensamos posibles escenarios y después vamos refactorizando el código para que todo funcione correctamente.

Otra funcionalidad que debemos probar es la de añadir autores a un libro:

```
1 public void addAuthor(Author author) {  
2     this.authors.add(author);  
3 }
```

¿Qué pasa si añadimos un autor que ya existe? ¿Y si añadimos un autor a un libro que no tiene ningún autor todavía? Como ves, tenemos que pensar en diferentes situaciones y crear tests adecuados a cada una de ellas. De esta forma, iremos depurando el código con la funcionalidad que necesitamos.

Los tests anteriores son sólo algunos ejemplos básicos. Siempre que añadamos lógica de negocio a nuestros modelos, deberíamos empezar por crear los tests probando diferentes posibilidades, ya que, a la larga, nos ayudarán a ser más rápidos y eficientes

Tests de los mapeadores

En cuanto a los mapeadores, tenemos que probar situaciones como que nos pasen un objeto null para mapear, ¿debería devolver null o dar una excepción? Dependiendo de cómo queramos que funcione nuestra aplicación, refactorizaremos nuestro código en función de los tests.

Además, tenemos que probar los mapeos entre diferentes clases, con listados de objetos... Algunos ejemplos de tests pueden ser:

```

1  @Test
2  @DisplayName("Test map Book to BookDto")
3  void toBookDto() {
4      // Arrange
5      var book = new Book(
6          "978-84-376-0494-7",
7          "TitleEs",
8          "TitleEn",
9          "SynopsisEs",
10         "SynopsisEn",
11         new BigDecimal("20.00"),
12         10,
13         "cover.jpg",
14         LocalDate.of(2020, 1, 1),
15         null,
16         null
17     );
18     book.setPublisher(new Publisher("Publisher Name", "pu
19     book.setAuthors(List.of(
20         new Author("Author One", "nationality1", "Bic
21         new Author("Author Two", "nationality2", "Bic
22     ));
23
24     // Act
25     var bookDto = BookMapper.getInstance().fromBookToBook
26
27     // Assert
28     assertEquals(book.getIsbn(), bookDto.is
29     () -> assertEquals(book.getTitleEs(), bookDtc
30     () -> assertEquals(book.getTitleEn(), bookDtc
31     () -> assertEquals(book.getSynopsisEs(), book
32     () -> assertEquals(book.getSynopsisEn(), book
33     () -> assertEquals(book.getBasePrice(), bookI
34     () -> assertEquals(book.getDiscountPercentage
35     () -> assertEquals(book.getCover(), bookDto.c
36     () -> assertEquals(book.getPublicationDate(),
37     () -> assertEquals(book.getPublisher().getNar
38     () -> assertEquals(book.getPublisher().getSlu
39     () -> assertEquals(book.getPublisher().getSlu
40     () -> assertEquals(2, bookDto.authors().size(
41     () -> assertEquals("Author One", bookDto.auth
42     () -> assertEquals("Author Two", bookDto.auth
43     );
44 }
45
46 @Test
47 @DisplayName("Test map null Book to BookDto throws Busine
48 void toBookDto_NullBook_ThrowsBusinessException() {
49     // Arrange
50     Book book = null;
51     // Act & Assert
52     assertEquals(book, null);
53     assertEquals(book, null);
54     assertEquals(book, null);
55 }
56
57 @Test
58 @DisplayName("Test map BookEntity to Book")
59 void toBook() {
60     // Arrange
61     var bookEntity = new BookEntity(
62         "978-84-376-0494-7",
63         "TitleEs",

```

```

64         "TitleEn",
65         "SynopsisEs",
66         "SynopsisEn",
67         new BigDecimal("20.00"),
68         10,
69         "cover.jpg",
70         LocalDate.of(2020, 1, 1),
71         new PublisherEntity("Publisher Name", "publicis"),
72         List.of(
73             new AuthorEntity("Author One", "national geographic"),
74             new AuthorEntity("Author Two", "national geographic")
75         )
76     );
77
78     // Act
79     var book = BookMapper.getInstance().fromBookEntityToBook(bookEntity);
80
81     // Assert
82     assertThat(book, allOf(
83         equalTo(bookEntity.isbn()),
84         equalTo(bookEntity.titleEs()),
85         equalTo(bookEntity.titleEn()),
86         equalTo(bookEntity.synopsisEs()),
87         equalTo(bookEntity.synopsisEn()),
88         equalTo(bookEntity.basePrice()),
89         equalTo(new BigDecimal("18.00")),
90         equalTo(bookEntity.discountPercent()),
91         equalTo(bookEntity.cover()),
92         equalTo(bookEntity.publicationDate()),
93         notNullValue(book.getPublisher()),
94         equalTo(bookEntity.publisher().name()),
95         equalTo(bookEntity.publisher().slug()),
96         notNullValue(book.getAuthors()),
97         equalTo(2, book.getAuthors().size()),
98         equalTo("Author One", book.getAuthors().get(0).name()),
99         equalTo("Author Two", book.getAuthors().get(1).name())
100     ));
101 }

```

Test de los servicios

En este caso, tendremos que usar Mockito para mockear nuestras dependencias. Por ejemplo, a nuestro servicio de libros necesitamos inyectarle la dependencia con el repositorio:

```

1 public class BookServiceImpl implements BookService {
2
3     private final BookRepository bookRepository;
4
5     public BookServiceImpl(BookRepository bookRepository) {
6         this.bookRepository = bookRepository;
7     }

```

Por lo tanto, lo primero que haremos en nuestro test es mockear ese repositorio:

```
1  @ExtendWith(MockitoExtension.class)
2  class BookServiceImplTest {
3
4      @Mock
5      private BookRepository bookRepository;
6
7      @InjectMocks
8      private BookServiceImpl bookServiceImpl;
```

A partir de ahí, como hemos hecho en los anteriores tests, iremos probando diferentes situaciones:

```
1  @Test
2  @DisplayName("getAll should return list of books")
3  void getAll_ShouldReturnListOfBooks() {
4      // Arrange
5      int page = 0;
6      int size = 10;
7
8      // Mock books
9      BookEntity bookEntity1 = new BookEntity(
10         "123",
11         "TitleEs1",
12         "TitleEn1",
13         "SynopsisEs1",
14         "SynopsisEn1",
15         new BigDecimal("10.00"),
16         5,
17         "cover1.jpg", LocalDate.of(2020, 1, 1),
18         null,
19         null
20     );
21     BookEntity bookEntity2 = new BookEntity(
22         "456",
23         "TitleEs2",
24         "TitleEn2",
25         "SynopsisEs2",
26         "SynopsisEn2",
27         new BigDecimal("15.00"),
28         10,
29         "cover2.jpg", LocalDate.of(2021, 6, 15),
30         null,
31         null
32     );
33     List<BookEntity> bookEntities = List.of(bookEntity1, bookEntity2);
34     when(bookRepository.findAll(page, size)).thenReturn(bookEntities);
35
36     // Act
37     List<BookDto> result = bookServiceImpl.getAll(page, size);
38
39     // Assert
40     assertEquals(2, result.size(), "Result should have 2 books");
41     assertEquals("123", result.get(0).isbn(), "First book isbn");
42     assertEquals("456", result.get(1).isbn(), "Second book isbn");
43
44     // Verify interaction with mock
45     Mockito.verify(bookRepository).findAll(page, size);
46 }
47
48 // test getByIsbn when book exists
49
50 // test getByIsbn when book does not exist
51
52 // test findByIsbn when book exists
53
54 // test findByIsbn when book does not exist
55
56 // test create book
57
58 // test create book with existing isbn
```

```
64  
65 // test create book with invalid data  
66  
67 // test create book with non-existing authors  
68  
69 // .....
```

Ten en cuenta que éstos son sólo algunos ejemplos de tests. Deberíamos crear nuevos tests por cada funcionalidad o lógica de negocio que añadiéramos al proyecto, como añadir autores, editoriales, sacar el listado de autores de un libro...

📄 clase/daw/dws/1eval/domain2.txt 📅 Última modificación: 2025/10/20 09:08 por cesguiro

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-Noncommercial-Share Alike 4.0 International

06_3 - Validación

Uno de los aspectos más importantes es la validación de datos. Tenemos que tener claro qué validamos o dónde validamos los datos y reglas de negocio.

Lo primero, ¿dónde validamos? Podríamos pensar que cuánto antes mejor, y, en parte, es correcto ¿Significa eso que tenemos que validar los datos en la capa de presentación? ¿Qué pasa si cambiamos los controladores o si tenemos varios controladores que reciben los mismos datos? Eso nos obligaría a repetir el código de validación en varios controladores, con lo que no parece la mejor opción.

La mayoría de validaciones deberían estar en nuestra capa de dominio, de forma que si cambiamos presentación o persistencia sigan siendo válidas.

En general, podemos distinguir dos tipos de validaciones: validaciones de **datos de entrada** y de **lógica de negocio**. La diferencia fundamental es que, mientras en las primeras no necesitamos acceder al estado del objeto para hacer las validaciones, en las segundas sí.

Por ejemplo, que un descuento no pueda ser negativo se puede validar accediendo al valor del campo únicamente, mientras que comprobar que un autor no esté repetido al añadirlo a un libro necesita acceder a los autores asociados a ese libro.

Validaciones de datos de entrada

Como hemos dicho, estas validaciones comprueban que los datos de entrada sean correctos, independientemente del estado del objeto.

Antes de ver dónde validamos, vamos a crearnos una excepción para lanzar en caso que nuestras validaciones fallen:

```
1 public class ValidationException extends RuntimeException {  
2     public ValidationException(String message) {  
3         super(message);  
4     }  
5 }
```

Veamos ahora dónde validar nuestros datos. Por ejemplo, queremos que el ISBN de los libros no sea nulo. Tenemos, básicamente, dos opciones: validarlo en el modelo de negocio (Book) o en el DTO de entrada (BookDto).

En nuestro caso, validaremos los datos en los DTOs, que son la frontera de entrada al dominio. Así obligamos a que cualquier capa externa (presentación, persistencia, A.P.I., etc.) entregue datos consistentes. De esta forma, cuando se construya un Book, podremos asumir que los datos básicos ya son válidos.

Si validáramos solo en el modelo (Book), podrían colarse datos inválidos desde fuera hasta que se cree la entidad. En cambio, al validar en el BookDto, nos aseguramos de que cualquier capa externa entregue datos consistentes desde el primer momento.

Por ejemplo, podríamos definir varias validaciones a la hora de crear el DTO:

```
1  public record BookDto(  
2      Long id,  
3      String isbn,  
4      String titleEs,  
5      String titleEn,  
6      String synopsisEs,  
7      String synopsisEn,  
8      BigDecimal basePrice,  
9      double discountPercentage,  
10     BigDecimal price,  
11     String cover,  
12     LocalDate publicationDate,  
13     PublisherDto publisher,  
14     List<AuthorDto> authors  
15 ) {  
16  
17     public BookDto(  
18         Long id,  
19         String isbn,  
20         String titleEs,  
21         String titleEn,  
22         String synopsisEs,  
23         String synopsisEn,  
24         BigDecimal basePrice,  
25         double discountPercentage,  
26         BigDecimal price,  
27         String cover,  
28         LocalDate publicationDate,  
29         PublisherDto publisher,  
30         List<AuthorDto> authors  
31     ) {  
32         this.id = id;  
33  
34         // === ISBN ===  
35         if (isbn == null || isbn.isBlank()) {  
36             throw new ValidationException("ISBN es obligat  
37         }  
38         this.isbn = isbn;  
39  
40         // === Títulos ===  
41         if ((titleEs == null || titleEs.isBlank()) &&  
42             (titleEn == null || titleEn.isBlank())) {  
43             throw new ValidationException("Debe existir al  
44         }  
45         this.titleEs = titleEs;  
46         this.titleEn = titleEn;  
47  
48         this.synopsisEs = synopsisEs;  
49         this.synopsisEn = synopsisEn;  
50  
51         // === Precio base ===  
52         if (basePrice == null || basePrice.compareTo(BigDe  
53             throw new ValidationException("El precio base  
54         }  
55         this.basePrice = basePrice;  
56  
57         // === Descuento ===  
58         if (discountPercentage < 0 || discountPercentage >  
59             throw new ValidationException("El descuento de  
60         }  
61         this.discountPercentage = discountPercentage;  
62  
63         this.price = price;
```

```
64
65         this.cover = cover;
66
67         // === Fecha publicación ===
68         if (publicationDate != null && publicationDate.isP
69             throw new ValidationException("La fecha de puk
70     }
71     this.publicationDate = publicationDate;
72
73     this.publisher = publisher;
74
75     this.authors = List.copyOf(authors);
76 }
77 }
```

Usamos **List.copyOf(authors)** para evitar **aliasing**, es decir, que alguien modifique desde fuera la lista original y cambie el estado interno del DTO sin pasar por las validaciones.

En este punto tenemos DTOs con validaciones manuales que garantizan datos consistentes al entrar en el dominio.

Sin embargo, este enfoque presenta dos problemas: mucho código repetitivo y validaciones dispersas dentro de cada DTO. La solución pasa por centralizar la lógica de validación y hacerla más declarativa.

Antes de recurrir a un framework externo como **Jakarta Bean Validation**, vamos a implementar nuestro propio mini-sistema de validación. La idea es disponer de una clase base **Validator<T>** que nos proporcione utilidades genéricas (como *notNull*, *notEmpty*, *pattern*, etc.), y a partir de ella crear validadores concretos para cada DTO, como *BookValidator*.

De esta forma entendemos la mecánica que hay detrás: recorrer los campos, comprobar reglas y acumular errores. Una vez tengamos claro este patrón, daremos el salto a *Jakarta Bean Validation*, que no es más que una implementación estándar y mucho más completa de la misma idea.

Validaciones con un mini-framework propio

Hemos visto que validar directamente dentro de cada DTO funciona, pero también implica repetir mucho código y mezclar responsabilidades. Una alternativa es separar la lógica de validación en clases dedicadas. Esto es, de hecho, lo que hacen frameworks como **Jakarta Bean Validation**: ofrecen una forma estándar y declarativa de definir validaciones.

Antes de introducir Jakarta, vamos a construir nuestro propio mini-framework de validación. La idea es tener una clase abstracta **Validator<T>** que proporcione métodos genéricos para comprobar reglas (*notNull*, *notEmpty*, *pattern*, etc.) y acumular errores. Luego, cada DTO tendrá su propio validador específico, por ejemplo, **BookValidator**.

Veamos un ejemplo inicial con la validación `notNull` aplicada al campo ISBN de nuestro *BookDto*:

```

1  public abstract class Validator<T> {
2
3      private List<String> errors = new ArrayList<>();
4
5      public abstract List<String> validate(T t);
6
7      protected void addError(String error) {
8          errors.add(error);
9      }
10
11     public boolean isValid() {
12         return errors.isEmpty();
13     }
14
15     public List<String> getErrors() {
16         return errors;
17     }
18
19     protected void notNull(T instance, Field field) throws
20         field.setAccessible(true);
21         Object value = field.get(instance);
22         if (value == null) {
23             addError(field.getName() + " must not be null"
24         }
25     }
26
27     // Aquí podríamos añadir más validaciones: notEmpty, n
28 }

1  public class BookValidator extends Validator<BookDto> {
2
3      @Override
4      public List<String> validate(BookDto bookDto) {
5          try {
6              Field isbnField = BookDto.class.getDeclaredFie
7              this.notNull(bookDto, isbnField);
8          } catch (Exception e) {
9              throw new ValidationException("ISBN is require
10          }
11          return getErrors();
12      }
13  }

```

Con esto tenemos una primera versión muy básica de un sistema de validaciones centralizado. Lo importante aquí no es la potencia, sino entender el **patrón**: tener validadores genéricos que recorren los objetos y aplican reglas, acumulando errores.

Una vez definido nuestro validador, podemos crear un `BookDto` y comprobar si es válido:

```

1 | BookDto bookDto = new BookDto(
2 |     1L,
3 |     null, // ISBN nulo para probar la validación
4 |     "Fundación",
5 |     "Foundation",
6 |     "Sinopsis en español",
7 |     "Synopsis in English",
8 |     BigDecimal.valueOf(10.0),
9 |     10,
10 |     null,
11 |     "cover.jpg",
12 |     LocalDate.of(1951, 6, 1),
13 |     new PublisherDto(1L, "Gnome Press", "gnome"),
14 |     List.of(new AuthorDto(1L, "Isaac Asimov", "USA", r
15 | );
16 |
17 | // Creamos el validador
18 | BookValidator validator = new BookValidator();
19 | List<String> errors = validator.validate(bookDto);
20 |
21 | if (!errors.isEmpty()) {
22 |     System.out.println("Errores de validación:");
23 |     errors.forEach(System.out::println);
24 | } else {
25 |     System.out.println("El BookDto es válido");
26 | }

```

Aunque este enfoque funciona, sigue siendo manual y limitado:

- Cada vez que añadimos un nuevo campo o regla debemos modificar el validador.
- Solo tenemos las reglas que implementamos nosotros (notNull, notEmpty, etc.).
- Seguimos gestionando manualmente la reflexión para acceder a los campos.

Para resolver estas limitaciones, existen frameworks como **Jakarta Bean Validation (JBV)**.

JBV nos permite declarar las reglas directamente sobre los campos mediante **anotaciones** (`@NotNull`, `@Size`, `@PastOrPresent`, etc.) y delegar en un motor de validación que recorre automáticamente los objetos y reporta los errores.

De esta forma:

1. Reducimos el código repetitivo.
1. Obtenemos validaciones consistentes y reutilizables.
1. Nos acercamos a la forma en que funcionan frameworks maduros como Hibernate Validator, que implementa JBV.

En la siguiente sección veremos cómo transformar nuestros DTOs y reglas manuales en **validaciones declarativas con JBV**.

Jakarta Validation

Jakarta Bean Validation (<https://beanvalidation.org/>) (**JBV**) es una especificación de Java que define un conjunto de APIs y anotaciones para la validación de objetos en aplicaciones Java. Proporciona un marco estándar para la validación de datos, permitiendo a los desarrolladores especificar reglas de validación directamente en las clases de dominio de sus aplicaciones.

La especificación *Jakarta Bean Validation* se centra en la validación de datos de manera declarativa mediante el uso de anotaciones en los campos de las clases. Algunas de las anotaciones comunes incluyen **@NotNull** para garantizar que un valor no sea nulo, **@Size** para especificar restricciones sobre la longitud de una cadena, **@Min** y **@Max** para establecer límites numéricos, entre otras.

Diferentes implementaciones, como Hibernate Validator (<https://hibernate.org/validator/>), Apache BVal (<https://bval.apache.org/>), y Apache Geronimo Validator (<https://geronimo.apache.org/apidocs/2.0.1/org/apache/geronimo/validator/package-summary.html>), proporcionan el soporte concreto para la especificación *Jakarta Bean Validation*, permitiendo a los desarrolladores elegir la implementación que mejor se adapte a sus necesidades.

Para usar JVB, primero debemos añadir la dependencia:

```
1 <dependency>
2   <groupId>jakarta.validation</groupId>
3   <artifactId>jakarta.validation-api</artifactId>
4   <version>${jakarta.validation.version}</version>
5 </dependency>
```

Después, deberíamos implementar las diferentes anotaciones que queramos usar (podemos crear anotaciones nuevas). Por ejemplo, vamos a implementar la anotación **@NotNull**:

```
1 @Target({ElementType.FIELD, ElementType.METHOD})
2 @Retention(RetentionPolicy.RUNTIME)
3 @Constraint(validatedBy = NotNullValidator.class)
4 public @interface NotNull {
5
6     String message() default "must not be null";
7 }
```

La anotación **@Constraint(validatedBy = NotNullValidator.class)** indica cuál es la clase encargada de validar esa anotación. En nuestro caso, podemos crear algo similar a:

```
1 public class NotNullValidator implements ConstraintValidator
2
3     @Override
4     public boolean isValid(Object value, ConstraintValidator
5         return value != null;
6     }
7 }
```

De esta forma, ya podemos usar esas anotaciones en nuestros DTOs:

```

1  public record BookDto (
2      Long id,
3      @NotNull
4      String isbn,
5      String titleEs,
6      String titleEn,
7      String synopsisEs,
8      String synopsisEn,
9      BigDecimal basePrice,
10     double discountPercentage,
11     BigDecimal price,
12     String cover,
13     LocalDate publicationDate,
14     PublisherDto publisher,
15     List<AuthorDto> authors
16 ) {
17 }

```

Siempre que queramos validar, deberemos usar un **Validator** de JBV:

```

1  BookDto bookDto = new BookDto(null, "Fundación", "Foundati
2
3  ValidatorFactory factory = Validation.buildDefaultValidatc
4  jakarta.validation.Validator validator = factory.getValide
5
6  Set<ConstraintViolation<BookDto>> violations = validator.v
7
8  if (!violations.isEmpty()) {
9      System.out.println("Errores de validación:");
10     violations.forEach(v -> System.out.println(v.getProper
11 } else {
12     System.out.println("El BookDto es válido");
13 }

```

Cuando usamos JBV, primero creamos la fábrica de validadores mediante **Validation.buildDefaultValidatorFactory()**. Esta fábrica (ValidatorFactory) es la encargada de generar instancias de **Validator**, que son los objetos que realmente realizan la validación de los DTOs. Con el Validator, podemos llamar a métodos como **validate(object)** para validar todo el objeto, o **validateProperty(object, propertyName)** para validar únicamente un campo concreto.

Cada vez que una regla de validación falla, JBV genera un objeto **ConstraintViolation** que contiene información detallada: el nombre del campo que no cumplió la regla (**getPropertyPath()**), el mensaje de error (**getMessage()**) y el valor que provocó la violación (**getInvalidValue()**). Así, al llamar a **validator.validate(bookDto)**, obtenemos un conjunto de todas las violaciones detectadas.

En resumen, el flujo consiste en crear el DTO a validar, obtener la fábrica y el validador, ejecutar la validación y procesar las violaciones. Todo esto se hace de manera automática, sin necesidad de recorrer los campos manualmente ni implementar múltiples comprobaciones, lo que es la principal ventaja de JBV frente a las validaciones manuales o nuestros mini-frameworks propios.

Al usar Jakarta Bean Validation directamente en nuestros DTOs de dominio estamos introduciendo una dependencia externa en la

capa de dominio, lo que rompe la pureza de la arquitectura limpia.

Esto puede provocar acoplamientos indeseados, dificultar el testing unitario puro del dominio y complicar cambios de framework en el futuro. Sin embargo, en la práctica es una concesión común y aceptada debido a la productividad y consistencia que ofrece.

Spring Boot Validation

Spring Boot Validation (SPV) es la implementación de Jakarta Bean Validation de Spring. Está basado en Hibernate Validator (<https://hibernate.org/validator/>). Nos permite usar todas las anotaciones de JBV de manera completa y además proporciona validadores avanzados y reglas adicionales.

SPV hace que nuestras validaciones sean automáticas y consistentes, integrándose fácilmente con frameworks de persistencia, APIs REST o formularios, y eliminando la necesidad de escribir lógica de validación manual o mini-frameworks caseros.

La ventaja de SPB es que además de respetar todas las anotaciones estándar de JBV, permite crear validadores personalizados complejos y reutilizables, y soporta mensajes internacionales y interpolación avanzada.

Como siempre, primero debemos añadir la dependencia:

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-validation</artifactId>
4   <version>${spring.boot.validation.version}</version>
5   <scope>test</scope>
6 </dependency>
```

Fíjate que añadimos la dependencia para que esté disponible sólo en los tests. De esta forma, podemos cambiar la implementación concreta de JBV sin que nuestra capa de dominio se vea afectada.

Ahora ya podemos usar sus anotaciones en nuestros DTOs:

```
1 public record PublisherDto(
2     Long id,
3     @NotNull(message = "Nombre no puede ser nulo")
4     String name,
5     @NotNull(message = "Slug no puede ser nulo")
6     @Pattern(regexp = "[a-z0-9]+(?:-[a-z0-9]+)*$", mes
7     String slug
8 ) {
9 }
```

Aquí usamos las anotaciones estándar de JBV:

- **@NotNull** para campos obligatorios.
- **@Pattern** para validar que el slug cumpla formato `URL()-friendly`.

Para centralizar la validación de cualquier DTO, creamos la clase **DtoValidator**. Esta clase inicializa un Validator de Hibernate Validator de manera estática, usando ValidatorFactory. Luego, mediante el método validate(T dto), podemos pasar cualquier DTO y automáticamente se verifican todas las anotaciones de validación. Si se detectan violaciones, se lanza una ValidationException que ahora acepta el conjunto de ConstraintViolation, permitiendo acceder a todos los errores de manera estructurada.

```

1  public class DtoValidator {
2
3      private static final Validator validator;
4
5      static {
6          ValidatorFactory factory = Validation.byDefaultPrc
7              .configure()
8              .buildValidatorFactory();
9          validator = factory.getValidator();
10     }
11
12     public static <T> void validate(T dto) {
13         Set<ConstraintViolation<T>> violations = validator
14             .if (!violations.isEmpty()) {
15                 throw new ValidationException(violations);
16             }
17     }
18 }

```

Para que esto funcione, modificamos nuestra ValidationException para aceptar directamente el conjunto de ConstraintViolation. Esto permite no solo almacenar los errores, sino también construir un mensaje completo concatenando cada violación y, al mismo tiempo, mantener acceso al detalle de cada una mediante el método getViolations().

```

1  public class ValidationException extends RuntimeException
2
3      private final Set<ConstraintViolation<?>> violations;
4
5      public ValidationException(String message) {
6          super(message);
7          this.violations = Set.of();
8      }
9
10     public ValidationException(Set<? extends ConstraintVic
11         super("Errores de validación detectados: " + viol
12         this.violations = Set.copyOf(violations);
13     }
14
15     public Set<ConstraintViolation<?>> getViolations() {
16         return violations;
17     }
18
19     @Override
20     public String getMessage() {
21         return violations.stream()
22             .map(v -> v.getPropertyPath() + ": " + v.c
23             .collect(Collectors.joining(", "));
24     }
25 }

```

Con esta estructura, validar un PublisherDto es tan sencillo como llamar a `DtoValidator.validate(publisherDto)`. Si existen errores, se lanza la excepción con todos los detalles de las violaciones, y de esta forma mantenemos centralizada la lógica de validación, aprovechando las ventajas de Hibernate Validator sin repetir código en cada DTO.

```

1  class PublisherDtoTest {
2
3
4
5      @ParameterizedTest
6      @DisplayName("Create publisherDto with invalid data sh
7      @CsvSource({
8          "1L, '', 'valid-slug'",
9          "1L, 'Valid Name', '',
10         "1L, '', '',
11         "1L, Valid Name, invalid slug",
12         "1L, Valid Name, 'invalid_slug!'"
13     })
14     void createPublisherDto_WithNullName_ShouldThrowExcept
15         PublisherDto publisherDto = new PublisherDto(1L, r
16         assertThrows(ValidationException.class, () -> DtoV
17
18     }
19
20 }
```

En muchas aplicaciones necesitamos validar que ciertos campos cumplan un formato específico. Por ejemplo, en nuestro caso, un *slug* debe ser `URL()-friendly`, es decir, contener solo minúsculas, números y guiones. Para no repetir la lógica en varios DTOs, podemos crear nuestra propia anotación de validación, aprovechando la infraestructura de Jakarta Bean Validation.

Primero, definimos la anotación **@Slug**. Esta anotación permite especificar un mensaje de error personalizado y es reconocida por JBV mediante la propiedad **@Constraint**, que indica la clase que implementará la validación:

```

1  @Target({ ElementType.FIELD, ElementType.METHOD })
2  @Retention(RetentionPolicy.RUNTIME)
3  @Constraint(validatedBy = SlugValidator.class)
4  public @interface Slug {
5      String message() default "Valor inválido, debe ser URI
6
7      Class<?>[] groups() default {};
8
9      Class<? extends Payload>[] payload() default {};
10 }
```

Cuando definimos nuestra anotación **@Slug** aparecen dos elementos: **Class<?>[] groups()** y **Class<? extends Payload>[] payload()**.

- **Class<?>[] groups() default {}:** este atributo se usa para organizar las validaciones en grupos de validación. Esto permite que un mismo DTO pueda validarse de diferentes maneras según el contexto. Por ejemplo, podríamos tener un grupo de validación para creación y otro para actualización, aplicando diferentes restricciones en cada caso. Si no necesitamos grupos, simplemente lo dejamos vacío.

- **Class<? extends Payload>[] payload() default {}:** este atributo permite asociar información adicional a la anotación, que luego puede ser utilizada por frameworks o interceptores. Por ejemplo, podríamos usarlo para categorizar errores de validación, adjuntar metadatos o incluir un código de error específico. En la mayoría de los casos, si solo queremos mostrar un mensaje, se deja vacío.

En resumen, ambos son requeridos por la especificación de Jakarta Bean Validation, incluso si no los usamos de manera activa. Nos permiten mantener la anotación compatible con la infraestructura de validación y con futuras extensiones, asegurando que nuestro @Slug pueda integrarse correctamente con cualquier Validator de JBV.

A continuación, implementamos la lógica de validación en **SlugValidator**. Este validador comprueba que el valor no sea nulo ni vacío y que cumpla la expresión regular correspondiente al formato `URL()-friendly`:

```

1 public class SlugValidator implements ConstraintValidator<
2
3     private static final String SLUG_PATTERN = "^[a-z0-9]+
4
5     @Override
6     public boolean isValid(String value, jakarta.validation.
7         if (value == null || value.isEmpty()) {
8             return false;
9         }
10        return value.matches(SLUG_PATTERN);
11    }
12 }
```

Una vez creada la anotación y el validador, podemos usar @Slug en cualquier DTO que necesite un campo `URL()-friendly`. Por ejemplo, tanto PublisherDto como AuthorDto pueden usarla en sus campos slug:

```

1 public record PublisherDto(
2     Long id,
3     @NotNull(message = "Nombre no puede ser nulo")
4     String name,
5     @Slug
6     String slug
7 ) {
8 }
```

De esta manera, centralizamos la lógica de validación de slugs y evitamos duplicación de código. Cuando llamamos a `DtoValidator.validate(dto)`, el validador comprobará automáticamente la anotación @Slug, junto con otras anotaciones estándar como @NotNull o @Pattern. Esto permite mantener consistencia en toda la aplicación y simplifica la gestión de errores de validación.

Validaciones de lógica de negocio

Mientras que las validaciones de entrada se ocupan de que los datos sean sintácticamente correctos (ej. ISBN de 13 dígitos, precio no negativo, fecha no futura), las reglas de negocio dependen del estado del dominio o de relaciones entre entidades. Estas validaciones suelen ir en la entidad o en servicios de dominio, no en los DTOs.

Las diferencias claves con respecto a las validaciones de datos de entrada son:

- **Contexto:** Requiere conocer el estado de la entidad o incluso otras entidades del dominio (ej. autores asociados a un libro).
- **Momento de ejecución:** Se realiza después de construir las entidades con datos válidos, normalmente en métodos de negocio o servicios.
- **Excepciones:** Se suelen lanzar **BusinessException** o excepciones específicas del dominio, no **ValidationException**.

Por ejemplo, en la entidad **Book**, podríamos tener una validación que asegura que un autor no se agregue dos veces a un libro:

```
1 public void addAuthor(Author author) {
2     if (this.authors.contains(author)) {
3         throw new BusinessException("Author already exists");
4     }
5     this.authors.add(author);
6 }
```

Este tipo de validación es interna a la entidad y depende del estado actual del libro (**this.authors**). Se asegura de que la regla de negocio —un libro no puede tener autores duplicados— se cumpla en cualquier momento que se manipulen los autores del libro.

En cambio, en el servicio **BookServiceImpl**, podríamos tener una validación al crear un libro:

```
1 @Override
2 public BookDto create(BookDto bookDto) {
3     Optional<BookDto> existingBookDto = findByIsbn(bookDto.getIsbn());
4
5     if (existingBookDto.isPresent()) {
6         throw new BusinessException("Book with isbn " + bookDto.getIsbn() + " already exists");
7     }
8
9     BookEntity newBookEntity = BookMapper.getInstance().fromBookDtoToBook(bookDto);
10    BookMapper.getInstance().fromBookEntityToBookDto(newBookEntity, bookDto);
11    ;
12
13    if (bookDto.publisher() != null &&
14        publisherRepository.findById(bookDto.publisherId()).isEmpty()) {
15        throw new ResourceNotFoundException("Publisher with id " + bookDto.publisherId() + " not found");
16    }
17    return BookMapper.getInstance().fromBookToBookDto(
18        BookMapper.getInstance().fromBookEntityToBook(
19            bookRepository.save(newBookEntity)
20        ),
21        bookDto
22    );
23 }
```

Aquí, la validación ocurre en un servicio de dominio y no solo depende del estado de un objeto, sino también de información externa, como:

- Existencia previa de un libro con el mismo ISBN.
- Existencia de un publisher en el repositorio.

Si alguna de estas condiciones no se cumple, se lanzan excepciones de negocio (**BusinessException** o **ResourceNotFoundException**), que reflejan reglas del sistema más amplias que solo afectan a la entidad Book.

Las diferencias claves entre ambas validaciones son:

- **Ámbito:**
 - **Entidad:** valida solo su propio estado interno y sus invariantes.
 - **Servicio:** puede validar reglas que involucren varios objetos o recursos externos.
- **Dependencias:**
 - **Entidad:** no debería depender de repositorios ni otros servicios.
 - **Servicio:** puede acceder a repositorios, otros servicios o APIs externas.
- **Momento de ejecución:**
 - **Entidad:** se ejecuta cada vez que se modifica la entidad de manera crítica (ej. añadir un autor).
 - **Servicio:** se ejecuta en operaciones de alto nivel, como crear o actualizar un recurso completo.
- **Tipo de excepción:**
 - **Entidad:** suele lanzar BusinessException para proteger las invariantes.
 - **Servicio:** puede lanzar BusinessException, ResourceNotFoundException o incluso combinaciones que reflejen reglas de negocio más complejas.

📄 clase/daw/dws/1eval/validacion.txt 📅 Última modificación: 2025/10/04 10:37 por cesguiro

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-Noncommercial-Share Alike 4.0 International

07 - Capa de persistencia

En una arquitectura limpia, los repositorios tienen la responsabilidad de gestionar el acceso a los datos desde la perspectiva del dominio. Esto significa que trabajan exclusivamente con modelos de dominio, abstrayendo por completo los detalles de cómo y dónde se almacenan los datos. Su objetivo principal es proporcionar una interfaz coherente que permita a la capa de dominio interactuar con los datos sin preocuparse por las particularidades de la persistencia.

Para lograr este aislamiento, los repositorios no deberían implementar directamente las operaciones específicas de almacenamiento o recuperación. Aquí es donde entra en juego la introducción de los **DAOs**, que encapsulan los detalles de persistencia. Por ejemplo, en un sistema que combine una base de datos y un sistema de caché, los repositorios podrían delegar en un DAO el acceso a la caché para recuperar datos rápidamente o almacenar temporalmente resultados, manteniendo así su enfoque en los modelos de dominio.

En este tema, vamos a crear una aplicación que representará la capa de persistencia de nuestra aplicación de gestión de libros.

Dependencia con dominio

Nuestra capa de persistencia tiene que implementar los repositorios que están en la capa de dominio, por lo que tendremos que añadir esa dependencia en nuestro pom:

```
<dependency>
  <groupId>es.cesguiro</groupId>
  <artifactId>daw2-bookstore-domain</artifactId>
  <version>${es.cesguiro.daw2-bookstore-domain.version}</
version>
</dependency>
```

DAO (Data Access Object)

El patrón DAO (Data Access Object) se utiliza para separar y encapsular el acceso a los datos, proporcionando una capa de abstracción que aísla la lógica de acceso a datos del resto de la aplicación. Esto permite que el resto de la aplicación, y en particular la capa de dominio, interactúe con los datos sin preocuparse por los detalles específicos de persistencia. Los DAOs facilitan la delegación de las operaciones de acceso a los datos, de modo que los repositorios se puedan enfocar únicamente en trabajar con los modelos de dominio.

Por ejemplo, supongamos que queremos implementar en nuestra aplicación un sistema de caché con Redis (<https://redis.io/>) para recuperar libros por su ISBN. En este caso, podemos hacer que el repositorio `BookRepositoryImpl` utilice dos DAOs: **BookDao**, que maneja el acceso a la base de datos, y **BookRedisDao**, que gestiona el almacenamiento en caché. De esta manera, el método `findByIsbn` intenta primero recuperar el libro de la caché a través de `BookRedisDao`. Si no se encuentra en la caché, `BookDao` realiza la consulta en la base de datos, y si el libro es encontrado, se guarda en la caché para futuras solicitudes.

Redis (REmote DIctionary Server) es un almacenamiento en memoria de datos tipo clave-valor, extremadamente rápido, que se utiliza principalmente para caching, colas de mensajes, sesiones de usuario y otros escenarios donde la velocidad de acceso es crítica.

Los datos se guardan en memoria RAM, por lo que la lectura y escritura es mucho más rápida que en una base de datos tradicional.

Redis soporta distintos tipos de estructuras de datos: strings, hashes, listas, conjuntos y sorted sets, lo que lo hace muy versátil.

Aunque es principalmente un almacenamiento en memoria, Redis permite persistir los datos en disco para mantenerlos entre reinicios si es necesario.

Es común usar Redis como nivel de cache entre la aplicación y la base de datos (patrón cache-aside), lo que reduce la carga sobre la base de datos y mejora el rendimiento de lectura.

En el contexto de nuestra aplicación, `BookRedisDao` actúa como un DAO específico de Redis: guarda y recupera libros usando su ISBN como clave, proporcionando un acceso mucho más rápido que la consulta directa a la base de datos.

Comenzaremos creando una interfaz genérica que servirá como base para nuestros DAOs: **GenericDaoDb**. Esta interfaz define los métodos básicos para interactuar con una base de datos.

Clases Genéricas

En Java, las **clases genéricas** nos permiten escribir código flexible y reutilizable. Definir una clase o interfaz genérica significa que podemos trabajar con distintos tipos de datos sin necesidad de escribir código duplicado para cada tipo de entidad. En el caso de los DAOs, esto es útil porque muchas operaciones, como obtener todos los registros o buscar por ID, son comunes para todas las entidades.

Por ejemplo, consideremos el método *findAll* en la interfaz *GenericDao*. Este método devuelve una lista de entidades, pero al ser genérico, no podemos saber, a priori, de qué tipo. En lugar de definir un método que solo funcione con, por ejemplo, libros (*Page<Book>*), usamos un parámetro genérico (*Page<T>*) para que pueda adaptarse a cualquier tipo de entidad, ya sea *Book*, *Author* o *Publisher*:

```
1 public interface GenericDao<T> {
2
3     Page<T> findAll(int page, int size);
4     Optional<T> findById(Long id);
5     T insert(T entity);
6     T update(T entity);
7     void deleteById(Long id);
8     long count();
9 }
```

Al utilizar **T**, le estamos indicando al compilador que el tipo de la entidad que se va a utilizar será determinado cuando implementemos o instanciamos la interfaz. Esto hace que la interfaz sea flexible y reutilizable para cualquier entidad.

Cuando necesitamos operaciones específicas para ciertas entidades, podemos crear interfaces que extienden *GenericDao* y agregar los métodos adicionales que correspondan a esas entidades. Por ejemplo, *PublisherDao* puede extender *GenericDao<Publisher>* y sólo tendríamos que añadir el método **findBySlug**, ya que no será común a todos nuestros Daos.

```
1 public interface PublisherDao extends GenericDao<PublisherEntity> {
2
3     Optional<PublisherEntity> findBySlug(String slug);
4 }
```

Cuando extendemos *GenericDao*, necesitamos indicar el tipo de entidad con el que trabajará la interfaz. Esto se hace especificando el tipo entre los corchetes angulares (<>) al momento de extender la interfaz genérica.

De esta forma, podemos crear el resto de DAOs:

```
1 public interface BookDao extends GenericDao<BookEntity> {
2
3     Optional<BookEntity> findByIsbn(String isbn);
4     void deleteByIsbn(String isbn);
5 }

1 public interface AuthorDao extends GenericDao<AuthorEntity> {
2
3     Optional<AuthorEntity> findBySlug(String slug);
4 }
```

Jakarta Persistence

La persistencia de datos en aplicaciones es un aspecto fundamental para el desarrollo de sistemas robustos y escalables. En el entorno de Java, Jakarta Persistence (<https://jakarta.ee/specifications/persistence/>) ha surgido como la

evolución moderna de **Java Persistence API (JPA)**, ofreciendo una solución estándar para el mapeo objeto-relacional (ORM). Jakarta Persistence permite a los desarrolladores interactuar con bases de datos relacionales utilizando objetos Java de manera eficiente y coherente.

Originalmente, JPA se definía bajo el paquete **javax.persistence**. Tras la migración de Java EE a la Eclipse Foundation, la especificación se renombró a Jakarta Persistence y los paquetes pasaron a ser **jakarta.persistence**. La funcionalidad es prácticamente la misma: definir entidades, relaciones y operaciones CRUD de manera orientada a objetos, pero con el namespace actualizado. Esta transición es importante para la compatibilidad con versiones modernas de Spring Boot y Hibernate.

Jakarta Persistence facilita la representación de entidades de persistencia como objetos Java, permitiendo su almacenamiento y recuperación en bases de datos de manera transparente. Al abstraer la lógica de persistencia, simplifica el manejo de la capa de acceso a datos, promoviendo un código más limpio, mantenible y portable entre diferentes proveedores de bases de datos.

En estos apuntes utilizaremos el término JPA por comodidad y porque es más familiar para muchos desarrolladores, pero en realidad nos referimos a Jakarta Persistence, la versión moderna y actualizada de JPA (jakarta.persistence).

JPA define la gestión de datos en aplicaciones mediante la representación de objetos en una base de datos relacional. JPA proporciona un conjunto de interfaces y clases abstractas que permiten a los desarrolladores interactuar con la base de datos de una manera orientada a objetos, sin tener que preocuparse por detalles específicos de la implementación subyacente de la base de datos.

Existen diferentes implementaciones de JPA, entre las que destacan:

- **Hibernate** (<https://hibernate.org/>): Es una de las implementaciones JPA más populares. Ofrece funcionalidades avanzadas y flexibilidad en el mapeo objeto-relacional, además de herramientas adicionales que simplifican el desarrollo y la gestión de la base de datos.
- **EclipseLink** (<https://eclipse.dev/eclipselink/>): Otra implementación JPA robusta, potente y de alto rendimiento. Ofrece características de mapeo avanzadas, soporte para estándares JPA y herramientas de persistencia útiles.
- **Apache OpenJPA** (<https://openjpa.apache.org/>): Una implementación JPA que proviene del proyecto OpenJPA de Apache. Proporciona funcionalidades completas de JPA, cumpliendo con los estándares de la API.

Estas implementaciones ofrecen funcionalidades similares en términos de mapeo objeto-relacional y operaciones CRUD, pero pueden diferir en sus características específicas, herramientas adicionales y rendimiento en ciertos contextos.

Al utilizar JPA, los desarrolladores pueden escribir código independiente de la base de datos subyacente, lo que les permite cambiar de proveedor de bases de datos sin tener que modificar considerablemente el código de la aplicación, lo que resulta en sistemas más flexibles y mantenibles a largo plazo.

En Spring Boot tenemos opciones para trabajar con JPA. Si bien Spring Data JPA (<https://spring.io/projects/spring-data-jpa>) es una forma común y conveniente de comenzar, podemos personalizar el enfoque dependiendo de las necesidades específicas del proyecto.

Por ejemplo, además de Spring Data JPA, podríamos optar por configurar JPA manualmente agregando las dependencias necesarias de forma individual. Esto nos ofrece un mayor control sobre cada componente que utilizamos.

Además, podemos elegir implementaciones específicas de JPA según nuestras preferencias o requisitos del proyecto. Aunque *Hibernate* es la implementación JPA predeterminada en Spring Boot, podemos optar por otras implementaciones como *EclipseLink* o *Apache OpenJPA*.

Añadiendo la dependencia y configurando la conexión

Como siempre, añadimos jakarta.persistence a nuestro pom:

```
1 <dependency>
2   <groupId>jakarta.persistence</groupId>
3   <artifactId>jakarta.persistence-api</artifactId>
4   <version>${jakarta.persistence-api.version}</version>
5 </dependency>
```

Igual que con las validaciones, tenemos que indicar en algún lugar qué implementación concreta vamos a usar en nuestro proyecto. Por ahora, definiremos esa dependencia para que esté disponible en los tests.

En nuestro caso, vamos a utilizar la configuración común de JPA en un proyecto Spring. Al incluir *Spring Data JPA* en un proyecto Spring Boot, obtenemos una configuración predeterminada que facilita la interacción con bases de datos utilizando JPA.

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-data-jpa</artifactId>
4   <version>${org.springframework.boot.version}</version>
5   <scope>test</scope>
6 </dependency>
```

Este starter no solo proporciona *Spring Data JPA*, sino que también configura automáticamente otros componentes esenciales como *Spring JDBC*, *Spring Transactions*, *Spring AOP* y *Spring Aspects*, si se requieren para el contexto de persistencia de datos.

Además, añadiremos otra dependencia de Spring en los tests que lleva incorporado JUnit y Mockito, entre otras cosas:

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-test</artifactId>
4   <version>${org.springframework.boot.version}</version>
5   <scope>test</scope>
6 </dependency>
```

En nuestro archivo *application.properties* de test tendremos que configurar de la conexión a la bbdd. Además, vamos a añadir un par de opciones que nos serán útiles:

```
1 spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;
2 spring.jpa.properties.hibernate.dialect=org.hibernate.diale
3 spring.datasource.username=root
4 spring.datasource.password=root
5 spring.jpa.show-sql=true
```

En los tests vamos a usar una bbdd embebida en memoria, H2 (<https://www.h2database.com/html/main.html>). Cuando veamos el tema de los tests en la capa de persistencia ya veremos como funciona.

En producción, deberemos usar el driver adecuado a nuestra bbdd

La propiedad **spring.jpa.hibernate.ddl-auto** es una configuración en aplicaciones Spring que determina cómo Hibernate maneja la creación y actualización de la estructura de la base de datos. Tiene varios valores que permiten definir cómo se realiza esta gestión:

- **none:** Este valor desactiva cualquier acción automática de creación o actualización de la base de datos por parte de Hibernate. Con este modo, Hibernate no hace ningún cambio en la estructura de la base de datos existente.
- **create:** Al utilizar este valor, Hibernate creará la estructura de la base de datos si no existe. Si la base de datos ya está presente, Hibernate la eliminará y la volverá a crear, lo que conlleva la pérdida de datos existentes.
- **create-drop:** Similar a create, Hibernate crea la estructura de la base de datos si no existe. Sin embargo, al cerrar la sesión de Hibernate o detener la aplicación, Hibernate eliminará la base de datos, lo que puede ser útil para entornos de desarrollo o pruebas.
- **update:** Este valor indica a Hibernate que actualice la estructura de la base de datos según los cambios en las entidades. No elimina los datos existentes, pero puede modificar o eliminar columnas, tablas, etc., para reflejar los cambios en el modelo de datos.

A veces, tu configuración del SGBD no permite crear la bbdd de forma automática. Una solución sencilla es crear la bbdd desde

fuera y ejecutar Spring para crear las tablas.

La última línea (**spring.jpa.show-sql=true**) es una configuración que se utiliza en aplicaciones Spring con Hibernate como implementación JPA. Cuando se establece a *true*, esta propiedad le indica a Hibernate que imprima las consultas SQL generadas por la aplicación en la consola, lo que nos puede ser útil mientras estamos en desarrollo.

Entidades

JPA utiliza sus propias entidades

(<https://jakarta.ee/specifications/persistence/3.2/jakarta-persistence-spec-3.2#entities>) que mapean una tabla de la bbdd. Para ello, debemos añadir la anotación **@Entity** a nuestras entidades.

```
1 | @Entity
2 | public class PublisherJpaEntity implements Serializable {
```

Una entidad de JPA debe tener un constructor vacío. Si no defines ningún otro constructor, el compilador de Java generará automáticamente uno vacío, por lo que JPA podrá instanciar la entidad sin problema.

Sin embargo, si añades un constructor con parámetros y no defines explícitamente un constructor vacío, JPA dará error al intentar instanciar la entidad.

JPA intentará conectar la entidad con la tabla correspondiente. El problema es que nuestras entidades llevan el sufijo *JpaEntity*, con lo que no encontrará ninguna tabla llamada *PublisherJpaEntity*. Por suerte, la solución es muy sencilla: indicarle el nombre de la tabla mediante la anotación **@Table**:

```
1 | @Entity
2 | @Table(name = "publishers")
3 | public class PublisherJpaEntity implements Serializable {
```

Una entidad debe tener definida una clave primaria. La forma más sencilla de definir la clave primaria es mediante la anotación **@Id**.

Además, la anotación **@GeneratedValue** en JPA, junto con **@Id**, se utiliza para especificar cómo se generan los valores para una clave primaria en una entidad persistente. En particular, *@GeneratedValue* se usa para indicar la estrategia que se utilizará para generar los valores de las claves primarias de manera automática por parte de la base de datos.

La estrategia **GenerationType.AUTO** especifica que JPA deja que el persistence provider (por ejemplo, Hibernate) elija automáticamente la mejor estrategia según la base de datos que se esté usando. :

```

1  @Entity
2  @Table(name = "publishers")
3  @RedisHash("Publisher")
4  public class PublisherJpaEntity implements Serializable {
5
6      @Id
7      @GeneratedValue(strategy = GenerationType.AUTO)
8      private Long id;
9      private String name;
10     private String slug;
11
12     //constructores, getters y setters

```

¿Qué pasa si los nombres de los campos de la tabla no coinciden con los de los atributos? Por ejemplo, en BookJpaEntity tenemos los campos titleEs, titleEn... mientras que en nuestra tabla los nombres son title_es, title_en... JPA no es capaz de asociar el atributo a un campo de la tabla. Por suerte, contamos con la anotación **@Column** para indicarle el nombre de la columna en la bbdd:

```

1  @Entity
2  @Table(name = "books")
3  public class BookJpaEntity implements Serializable {
4
5      @Id
6      @GeneratedValue(strategy = GenerationType.IDENTITY)
7      private Long id;
8      private String isbn;
9      @Column(name = "title_es")
10     private String titleEs;
11     ...

```

Relaciones entre entidades

En JPA podemos representar relaciones entre entidades para facilitar el tratamiento de datos. Para hacerlo, contamos con una serie de anotaciones según el tipo de relación que queramos representar:

- **@OneToOne**: Define una relación uno a uno entre dos entidades.
- **@OneToMany**: Define una relación de uno a muchos, donde una entidad tiene una asociación con múltiples entidades de otro tipo.
- **@ManyToOne**: Establece una relación muchos a uno, indicando que varias entidades de una clase están relacionadas con una única entidad de otra clase.
- **@ManyToMany**: Define una relación muchos a muchos entre dos entidades, lo que implica que una entidad puede estar asociada con múltiples entidades de otro tipo y viceversa.

Las relaciones pueden ser bidireccionales o unidireccionales. Una relación bidireccional tiene dos lados:

- un lado propietario (owning side), y
- un lado inverso o no propietario (inverse / non-owning side).

En cambio, una relación unidireccional tiene solo el lado propietario.

El lado propietario de una relación es el que controla las actualizaciones que se reflejan en la base de datos.

Por ejemplo, BookJpaEntity tiene dos relaciones con PublisherJpaEntity y AuthorJpaEntity:

```

1  @Entity
2  @Table(name = "books")
3  public class BookJpaEntity implements Serializable {
4
5      @Id
6      @GeneratedValue(strategy = GenerationType.IDENTITY)
7      private Long id;
8      ...
9      @ManyToOne(fetch = FetchType.LAZY)
10     private PublisherJpaEntity publisher;
11     @ManyToMany
12     @JoinTable(
13         name = "books_authors",
14         joinColumns = @JoinColumn(name = "book_id"),
15         inverseJoinColumns = @JoinColumn(name = "author_id")
16     )
17     private List<AuthorJpaEntity> authors;
18     ...

```

El primer tipo de relación (con PublisherJpaEntity) será **ManyToOne**. El atributo **fetch** indica cómo queremos recuperar los datos asociados. En este caso, indicamos que queremos Lazy Loading (https://es.wikipedia.org/wiki/Lazy_loading) (carga perezosa).

El segundo tipo sería @ManyToMany. En este tipo de relaciones tenemos que indicarle la tabla de enganche y las claves ajenas de dicha tabla. Para eso, utilizamos la anotación **@JoinTable**.

Aunque podríamos dejarlo así, vamos a romper la relación ManyToMany y convertirla en una relación OneToMany/ManyToOne con la entidad intermedia **BookAuthorJpaEntity**. Este enfoque presenta varias ventajas frente al uso de una relación ManyToMany directa:

- **Mayor flexibilidad:** permite añadir información adicional sobre la relación, como el rol del autor, el orden en que aparece, o cualquier otro metadato relevante.
- **Control total sobre la persistencia:** al ser una entidad propia, podemos gestionar su ciclo de vida, aplicar cascade operations, y eliminar registros huérfanos de forma más controlada.
- **Mejor legibilidad y mantenibilidad:** el modelo refleja de manera más fiel la estructura real de la base de datos, donde la relación muchos a muchos siempre se materializa mediante una tabla intermedia.
- **Facilidad para realizar consultas específicas:** se pueden definir consultas directamente sobre la entidad intermedia, lo que permite filtrar, ordenar o agrupar resultados de forma más precisa.

Convertir la relación ManyToMany en una estructura OneToMany/ManyToOne aporta una mayor robustez y flexibilidad al modelo de datos, lo que resulta especialmente útil en proyectos que pueden evolucionar o requerir información

adicional en las asociaciones:

```

1  @Entity
2  @Table(name = "books")
3  public class BookJpaEntity implements Serializable {
4
5      @Id
6      @GeneratedValue(strategy = GenerationType.IDENTITY)
7      private Long id;
8      private String isbn;
9      @Column(name = "title_es")
10     private String titleEs;
11     @Column(name = "title_en")
12     private String titleEn;
13     @Column(name = "synopsis_es", length = 2000)
14     private String synopsisEs;
15     @Column(name = "synopsis_en", length = 2000)
16     private String synopsisEn;
17     @Column(name = "base_price")
18     private BigDecimal basePrice;
19     @Column(name = "discount_percentage")
20     private Double discountPercentage;
21     private String cover;
22     @Column(name = "publication_date")
23     private String publicationDate;
24     @ManyToOne(fetch = FetchType.LAZY)
25     @JoinColumn(name = "publisher_id")
26     private PublisherJpaEntity publisher;
27
28     @OneToMany(mappedBy = "book", cascade = CascadeType.ALL)
29     private List<BookAuthorJpaEntity> bookAuthors = new ArrayList<>();

```

```

1  @Entity
2  @Table(name = "book_author")
3  public class BookAuthorJpaEntity {
4
5      @Id
6      @GeneratedValue(strategy = GenerationType.IDENTITY)
7      private Long id;
8
9      @ManyToOne(fetch = FetchType.LAZY)
10     @JoinColumn(name = "book_id")
11     private BookJpaEntity book;
12
13     @ManyToOne(fetch = FetchType.LAZY)
14     @JoinColumn(name = "author_id")
15     private AuthorJpaEntity author;

```

Además, podemos crear dos métodos en BookJpaEntity para leer/escribir los autores:


```

1  public List<AuthorJpaEntity> getAuthors() {
2      return bookAuthors.stream().map(BookAuthorJpaEntity::c
3  }
4
5  public void setAuthors(List<AuthorJpaEntity> authors) {
6      this.bookAuthors.clear();
7      for (AuthorJpaEntity author : authors) {
8          BookAuthorJpaEntity bookAuthor = new BookAuthorJpa
9          this.bookAuthors.add(bookAuthor);
10     }
11
12 }

```

Mapeadores

Como es obvio, necesitamos poder convertir los DTOs que nos llegan de dominio (BookEntity) a entidades de JPA (BookJpaEntity). Podríamos hacer los mapeadores a mano, como hicimos en dominio, pero vamos a utilizar una librería que nos ayudará en la tarea: MapStruct (<https://mapstruct.org/>).

MapStruct es una librería de mapeo de Java que te permite convertir automáticamente entre distintos tipos de objetos (DTOs, entidades, JPA entities, etc.) sin escribir manualmente los métodos de copia de propiedades.

Ventajas principales:

- Generación automática de código en tiempo de compilación → no hay reflexión en tiempo de ejecución.
- Alto rendimiento comparado con librerías de mapeo dinámico como ModelMapper.
- Puedes personalizar las conversiones con anotaciones como @Mapping, @Mappings y métodos auxiliares.
- Compatible con Spring (componentModel = "spring") para inyección automática de beans mappers.

Como siempre, añadimos la dependencia correspondiente:

```

1  <dependency>
2      <groupId>org.mapstruct</groupId>
3      <artifactId>mapstruct</artifactId>
4      <version>${org.mapstruct.version}</version>
5  </dependency>

```

Además, en el plugin de compilación de Maven debemos indicar el procesador de anotaciones:

```

1  <plugin>
2    <groupId>org.apache.maven.plugins</groupId>
3    <artifactId>maven-compiler-plugin</artifactId>
4    <version>${org.apache.maven.compiler.plugin.version}</
5    <configuration>
6      <source>${maven.compiler.source}</source>
7      <target>${maven.compiler.target}</target>
8      <parameters>true</parameters>
9      <annotationProcessorPaths>
10         <path>
11           <groupId>org.mapstruct</groupId>
12           <artifactId>mapstruct-processor</artifactId>
13           <version>${org.mapstruct.version}</version>
14         </path>
15       </annotationProcessorPaths>
16     </configuration>
17 </plugin>

```

Con ésto, ya podemos crear mapeadores con MapStruct, usando la anotación **@Mapper**:

```

1  @Mapper
2  public interface PublisherMapper {
3
4      PublisherMapper INSTANCE = Mappers.getMapper(Publisher
5
6      PublisherJpaEntity publisherEntityToPublisherJpaEntity
7
8      PublisherEntity publisherJpaEntityToPublisherEntity(Pu
9
10 }

```

Si los campos de las dos clases involucradas (en este caso PublisherEntity y PublisherJpaEntity) son del mismo tipo y tienen el mismo nombre, no hace falta añadir nada más. MapStruct creará la implementación de forma automática.

Gracias a los métodos añadidos en BookJpaEntity (**getAuthors** y **setAuthors**), el mapeador funcionará sin necesidad de añadir más código, ya que, en realidad, utiliza los getters y setters de cada entidad para hacer el mapeo.

DAOs JPA

En general, un DAO JPA tendrá:

- Una referencia a EntityManager (<https://jakarta.ee/specifications/persistence/3.1/jakarta-persistence-spec-3.1#a1062>), que es el componente central de JPA para gestionar la persistencia.
- Implementaciones de los métodos definidos en GenericDao, como findAll, findById, insert, update o deleteById.
- Métodos específicos para la entidad concreta, como findByIsbn en BookDao.

EntityManager actúa como el puente entre nuestras entidades Java y la base de datos. Permite realizar operaciones CRUD (crear, leer, actualizar, borrar) de manera transparente, sin tener que escribir SQL manualmente, y también gestionar el ciclo de vida de las entidades dentro del contexto de persistencia.

Con EntityManager podemos:

Operación	Método	Descripción
Crear	<code>entityManager.persist(entity)</code>	Inserta un nuevo objeto en la base de datos (cuando se sincronice el contexto).
Leer	<code>entityManager.find(BookJpaEntity.class, id)</code>	Recupera la entidad con la clave primaria indicada.
Actualizar	<code>entityManager.merge(entity)</code>	Sincroniza los cambios de un objeto con la base de datos.
Borrar	<code>entityManager.remove(entity)</code>	Elimina la entidad de la base de datos.
Consultar	<code>entityManager.createQuery("SELECT b FROM BookJpaEntity b", BookJpaEntity.class)</code>	Ejecuta una consulta JPQL sobre entidades
Sincronizar	<code>entityManager.flush()</code>	Fuerza la escritura inmediata de los cambios pendientes en la base de datos.

Contexto de persistencia

EntityManager mantiene un **persistence context**, es decir, un conjunto de entidades gestionadas en memoria.

Si obtenemos la misma entidad varias veces, JPA nos devolverá **la misma instancia en memoria**, y cualquier cambio realizado se sincronizará automáticamente con la base de datos cuando se complete la transacción.

En Jakarta Persistence (JPA), la anotación **@PersistenceContext** es la forma estándar de inyectar un EntityManager gestionado por el contenedor.

Esto significa que el contenedor se encarga del ciclo de vida del EntityManager: abrirlo, cerrarlo y sincronizarlo con las transacciones activas.

```

1 public class PublisherDaoJpa implements PublisherDao {
2
3     @PersistenceContext
4     private EntityManager entityManager;
```

De esta manera, cada DAO tiene un control completo sobre cómo se interactúa con la base de datos, permitiendo personalizar consultas, optimizar el rendimiento y combinar operaciones con otros sistemas como Redis, siguiendo el patrón de repositorios que hemos definido en la capa de dominio.

Acceso a datos

Con JPA disponemos de tres formas principales de consultar y operar con la base de datos:

JPQL / HQL (JPA Query Language)

JPQL (Java Persistence Query Language) es un lenguaje de consultas orientado a entidades, no a tablas. Permite escribir consultas similares a SQL, pero utilizando los nombres de las clases y sus atributos, en lugar de tablas y columnas.

```
1 List<BookJpaEntity> books = entityManager
2     .createQuery("SELECT b FROM BookJpaEntity b WHERE b.aut
3     .setParameter("author", "Isaac Asimov")
4     .getResultList();
```

Ventajas

Más legible y cercano al modelo de dominio.

Independiente del motor de base de datos.

Permite aprovechar relaciones (joins, fetch) entre entidades.

Inconvenientes

No soporta funciones específicas de cada motor SQL.

Native SQL

Permite ejecutar consultas SQL crudas directamente sobre la base de datos, usando `createNativeQuery`. Se utiliza cuando JPQL no es suficiente o se necesita aprovechar funciones específicas del motor SQL.

```
1 List<Object[]> result = entityManager
2     .createNativeQuery("SELECT id, title FROM books WHERE a
3     .setParameter(1, "Isaac Asimov")
4     .getResultList();
```

También se pueden mapear los resultados a entidades o DTOs:

```
1 List<BookJpaEntity> books = entityManager
2     .createNativeQuery("SELECT * FROM books", BookJpaEntity
3     .getResultList();
```

Ventajas

Control total sobre la consulta.

Permite usar funciones y optimizaciones específicas del motor SQL.

Inconvenientes

Dependiente del dialecto de la base de datos.

No se aprovecha la abstracción del modelo de entidades.

Más propenso a errores de mantenimiento.

Criteria API (CriteriaBuilder / CriteriaQuery)

Es un query builder tipado y programático que permite construir consultas dinámicamente mediante código Java, sin concatenar strings. Ideal para filtros opcionales o búsquedas avanzadas.

```
1 CriteriaBuilder cb = entityManager.getCriteriaBuilder();
2 CriteriaQuery<BookJpaEntity> cq = cb.createQuery(BookJpaEnt
3 Root<BookJpaEntity> book = cq.from(BookJpaEntity.class);
4
5 cq.select(book)
6     .where(cb.equal(book.get("author"), "Isaac Asimov"));
7
8 List<BookJpaEntity> results = entityManager.createQuery(cq)
```

Ventajas**Inconvenientes**

Totalmente tipado y seguro en tiempo de compilación.

Verboso y menos legible que JPQL.

Muy útil para construir consultas dinámicas.

Más complejo para consultas simples.

Evita errores de concatenación o typos en nombres de campos.

Por ejemplo, en nuestro BookJpaDaoImpl, podemos añadir los métodos necesarios para recuperar datos:

```

1  public class BookJpaDaoJpaImpl implements BookJpaDao {
2
3      @PersistenceContext
4      private EntityManager entityManager;
5
6      @Override
7      public long count() {
8          return entityManager.createQuery("SELECT COUNT(b)
9              .getSingleResult();
10     }
11
12     @Override
13     public List<BookJpaEntity> findAll(int page, int size)
14         int pageIndex = Math.max(page - 1, 0);
15
16         String sql = "SELECT b FROM BookJpaEntity b ORDER
17         TypedQuery<BookJpaEntity> bookJpaEntityPage = enti
18             .createQuery(sql, BookJpaEntity.class)
19             .setFirstResult(pageIndex * size)
20             .setMaxResults(size);
21
22         return bookJpaEntityPage.getResultList();
23     }
24
25     @Override
26     public Optional<BookJpaEntity> findById(Long id) {
27         return Optional.ofNullable(entityManager.find(Book
28     }
29
30     @Override
31     public Optional<BookJpaEntity> findByIsbn(String isbn)
32         String sql = "SELECT b FROM BookJpaEntity b WHERE
33         try {
34             return Optional.of(entityManager.createQuery(s
35                 .setParameter("isbn", isbn)
36                 .getSingleResult());
37         } catch (Exception e) {
38             return Optional.empty();
39         }
40     }

```

Y nuestro repositorio:

```

1  public class BookRepositoryImpl implements BookRepository
2
3      private final BookJpaDao bookJpaDao;
4
5      public BookRepositoryImpl(BookJpaDao bookJpaDao) {
6          this.bookJpaDao = bookJpaDao;
7      }
8
9      @Override
10     public Page<BookEntity> findAll(int page, int size) {
11         List<BookEntity> content = bookJpaDao.findAll(page
12             .map(BookMapper.INSTANCE::fromBookJpaEntity)
13             .toList());
14         long totalElements = bookJpaDao.count();
15         return new Page<>(content, page, size, totalElements);
16     }
17
18     @Override
19     public Optional<BookEntity> findById(Long id) {
20         return bookJpaDao.findById(id)
21             .map(BookMapper.INSTANCE::fromBookJpaEntity);
22     }
23
24     @Override
25     public Optional<BookEntity> findByIsbn(String isbn) {
26         return bookJpaDao.findByIsbn(isbn)
27             .map(BookMapper.INSTANCE::fromBookJpaEntity);
28     }
29
30

```

Actualizaciones, inserciones y borrados

Para las inserciones y actualizaciones, podemos crear el método **save()** en el repositorio, y saber si es una actualización o inserción según si tiene o no **id**:

```

1  @Override
2  public BookEntity save(BookEntity bookEntity) {
3      BookJpaEntity bookJpaEntity = BookMapper.INSTANCE.fromEntity(bookEntity);
4      if(bookEntity.id() == null) {
5          return BookMapper.INSTANCE.fromBookJpaEntityToBookEntity(bookJpaEntity);
6      }
7      return BookMapper.INSTANCE.fromBookJpaEntityToBookEntity(bookJpaDao.save(bookJpaEntity));
8  }

```

En cuanto al borrado, lo hacemos por isbn:

```

1  @Override
2  public void deleteByIsbn(String isbn) {
3      bookJpaDao.deleteByIsbn(isbn);
4  }

```

En nuestro DAO, los métodos **insert** y **deleteByIsbn** son sencillos:

```
1  @Override
2  public BookJpaEntity insert(BookJpaEntity bookJpaEntity) {
3      entityManager.persist(bookJpaEntity);
4      return bookJpaEntity;
5  }
6
7  @Override
8  public void deleteById(Long id) {
9      entityManager.remove(entityManager.find(BookJpaEntity.
10 }
```

En cuanto al **update**, al hacer mapeos entre BookJpaEntity y BookEntity y haber roto la relación ManyToMany con Authors, primero debemos asegurarnos que eliminamos los autores anteriores:

```
1  @Override
2  public BookJpaEntity update(BookJpaEntity bookJpaEntity) {
3      BookJpaEntity managed = entityManager.find(BookJpaEnti
4      if (managed == null) {
5          throw new ResourceNotFoundException("Book with id
6      }
7      managed.getBookAuthors().clear();
8      entityManager.flush();
9      return entityManager.merge(bookJpaEntity);
10 }
```

Usamos el método **flush** para ejecutar el borrado antes que intente insertar de nuevo los autores.

Repositorio

<https://github.com/cesguiro/daw2-bookstore-persistence>
(<https://github.com/cesguiro/daw2-bookstore-persistence>)

📄 clase/daw/dws/1eval/persistence.txt 📅 Última modificación: 2025/10/15 13:33 por cesguiro

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-NonCommercial-Share Alike 4.0 International

07_2 - Capa persistencia: Test

Para los test de la capa de persistencia usaremos las siguientes dependencias:

- H2 (<https://www.h2database.com/html/main.html>): Base de datos embebida en memoria
- Flywaydb (<https://www.red-gate.com/products/flyway/>): Herramienta de migración de base de datos
- Spring Data Jpa (<https://spring.io/projects/spring-data-jpa>): Implementación de Jakarta Persistence de Spring

```
1  <dependency>
2      <groupId>org.springframework.boot</groupId>
3      <artifactId>spring-boot-starter-data-jpa</artifactId>
4      <version>${org.springframework.boot.version}</version>
5      <scope>test</scope>
6  </dependency>
7  <dependency>
8      <groupId>org.springframework.boot</groupId>
9      <artifactId>spring-boot-starter-test</artifactId>
10     <version>${org.springframework.boot.version}</version>
11     <scope>test</scope>
12 </dependency>
13 <dependency>
14     <groupId>org.apache.commons</groupId>
15     <artifactId>commons-csv</artifactId>
16     <version>${org.apache.commons.csv.version}</version>
17     <scope>test</scope>
18 </dependency>
19 <dependency>
20     <groupId>com.h2database</groupId>
21     <artifactId>h2</artifactId>
22     <version>${com.h2database.version}</version>
23     <scope>test</scope>
24 </dependency>
25 <dependency>
26     <groupId>org.flywaydb</groupId>
27     <artifactId>flyway-core</artifactId>
28     <version>${org.flywaydb.version}</version>
29     <scope>test</scope>
30 </dependency>
```

Los tests de los mapeadores y repositorios los haremos como los de la capa de dominio, usando Mockito cuando sea necesario (para mockear los DAO).

Test DAO

Para testear los DAO, necesitamos levantar una base de datos temporal en memoria. H2 es ideal para esto porque se ejecuta embebida, sin necesidad de un servidor externo, y se reinicia desde cero en cada ejecución de test.

Configurando H2

En el fichero **src/test/resources/application-test.properties**, añadimos la configuración de la base de datos en memoria:

```
1 spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;
2 spring.jpa.properties.hibernate.dialect=org.hibernate.diale
3 spring.datasource.username=root
4 spring.datasource.password=root
5 spring.jpa.show-sql=true
6 spring.jpa.hibernate.ddl-auto=none
```

Para poblar la bbdd de pruebas, usaremos Flyway.

Migraciones con Flyway

Cuando se ejecutan los tests de persistencia, necesitamos que la base de datos (H2 en memoria) esté correctamente inicializada con las tablas y datos que usarán los DAO o repositorios. Ahí es donde entra Flyway: ejecuta automáticamente los scripts de migración SQL antes de que empiecen los tests.

Por convención, Flyway busca los scripts SQL dentro de **src/main/resources/db/migration**. Cada archivo debe tener un nombre con el siguiente formato:

```
1 V1__init.sql
2 V2__insert_data.sql
3 V3__add_constraints.sql
4 ...
```

El prefijo **V** indica una versión (migración incremental). El doble guion bajo __ separa el número de versión del nombre descriptivo.

Si tus migraciones están en otra carpeta, puedes configurarlo en **application.properties**:

```
1 spring.flyway.locations=classpath:/migraciones
```

@DataJpaTest levanta un contexto mínimo de Spring con JPA, H2 y transacciones configuradas automáticamente. Al terminar cada test, la transacción se revierte, dejando la BD limpia.

```
1 @DataJpaTest
2 class PublisherDaoJpaImplTest {
```

Además, tendremos que crear los Bean de Spring para que funcione la inyección de dependencias. Para eso, creamos una clase de configuración:

```
1  @Configuration
2  @EnableJpaRepositories(basePackages = "es.cesguiro.persist
3  @EntityScan(basePackages = "es.cesguiro.persistence.dao.jp
4  public class TestConfig {
5      @Bean
6      public PublisherJpaDao publisherJpaDao(EntityManager e
7          return new PublisherJpaDaoImpl();
8      }
9
10     @Bean
11     public BookJpaDao bookJpaDao(EntityManager entityManag
12         return new BookJpaDaoImpl();
13     }
14
15     @Bean
16     public AuthorJpaDao authorJpaDao(EntityManager entityM
17         return new AuthorJpaDaoImpl();
18     }
19
20 }
```

De esta forma, cuando tenga que inyectar, por ejemplo, BookJpaDao, Spring buscará en su contenedor de Beans si existe y será capaz de hacer la inyección de forma automática.

```

1  @DataJpaTest
2  @ContextConfiguration(classes = TestConfig.class)
3  @AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase
4  class BookJpaDaoImplTest {
5
6      @PersistenceContext
7      private EntityManager entityManager;
8
9      @Autowired
10     private BookJpaDao bookJpaDao;
11
12     @Autowired
13     private PublisherJpaDao publisherJpaDao;
14
15     @Autowired
16     private AuthorJpaDao authorJpaDao;
17
18
19     @Test
20     @DisplayName("Test insert method persists BookJpaEntity")
21     void testInsert() {
22         PublisherJpaEntity publisherJpaEntity = new PublisherJpaEntity();
23         publisherJpaEntity.setName("Editorial X");
24         publisherJpaEntity.setSlug("editorial-x");
25         entityManager.persist(publisherJpaEntity);
26
27         AuthorJpaEntity authorJpaEntity1 = new AuthorJpaEntity();
28         authorJpaEntity1.setName("Author One");
29         entityManager.persist(authorJpaEntity1);
30         AuthorJpaEntity authorJpaEntity2 = new AuthorJpaEntity();
31         authorJpaEntity2.setName("Author Two");
32         entityManager.persist(authorJpaEntity2);
33
34
35         BookJpaEntity newBook = new BookJpaEntity(
36             null,
37             "66666666666666",
38             "New Book Title ES",
39             "New Book Title EN",
40             "New Book Synopsis ES",
41             "New Book Synopsis EN",
42             BigDecimal.valueOf(29.99),
43             10.0,
44             "new_book_cover.jpg",
45             LocalDate.of(2024, 1, 1).toString(),
46             publisherJpaEntity, // Assuming the first
47             List.of(authorJpaEntity1, authorJpaEntity2)
48         );
49
50         String sql = "SELECT COUNT(b) FROM BookJpaEntity k";
51         long countBefore = entityManager.createQuery(sql, Long.class)
52             .getSingleResult();
53
54         BookJpaEntity result = bookJpaDao.insert(newBook);
55
56         long countAfter = entityManager.createQuery(sql, Long.class)
57             .getSingleResult();
58
59         long lastId = entityManager.createQuery("SELECT MAX(id) FROM BookJpaEntity")
60             .getSingleResult();
61
62         Set<Long> expectedAuthorIds = newBook.getAuthors()
63             .map(AuthorJpaEntity::getId)

```

```
64         .collect(Collectors.toSet());
65     Set<Long> resultAuthorIds = result.getAuthors().st
66         .map(AuthorJpaEntity::getId)
67         .collect(Collectors.toSet());
68
69     assertAll(
70         () -> assertNotNull(result),
71         () -> assertEquals(lastId, result.getId())
72         () -> assertEquals(newBook.getIsbn(), resu
73         () -> assertEquals(newBook.getTitleEs(), r
74         () -> assertEquals(newBook.getTitleEn(), r
75         () -> assertEquals(newBook.getSynopsisEs()
76         () -> assertEquals(newBook.getSynopsisEn()
77         () -> assertEquals(newBook.getBasePrice(),
78         () -> assertEquals(newBook.getDiscountPerc
79         () -> assertEquals(newBook.getCover(), res
80         () -> assertEquals(newBook.getPublicationI
81         () -> assertEquals(newBook.getPublisher().
82         () -> assertEquals(newBook.getAuthors().si
83         () -> assertEquals(expectedAuthorIds, resu
84         () -> assertEquals(countBefore + 1, countA
85     );
86 }
```

📄 clase/daw/dws/1eval/persistence2.txt 📅 Última modificación: 2025/10/18 10:52 por cesguiro

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-Noncommercial-Share Alike 4.0 International

08 - Capa presentación

La capa de presentación en una arquitectura por capas se encarga de gestionar la interacción del usuario con la aplicación. Su principal función es recibir las solicitudes del usuario (normalmente a través de controladores REST o MVC en el caso de aplicaciones web), procesarlas y devolver una respuesta adecuada. Aquí se implementan los controladores que definen los endpoints de la **APL()** y, opcionalmente, la lógica de validación básica de las solicitudes.

Esta capa debe ser lo más independiente posible de la lógica de negocio y la persistencia. Se comunica con la capa de dominio o de aplicación para ejecutar los casos de uso requeridos, devolviendo al usuario las respuestas en el formato adecuado (como JSON o **HTML()**). También puede encargarse del manejo de excepciones y la validación básica de los datos de entrada.

endpoints

En el desarrollo de APIs, es una buena práctica organizar y estructurar los endpoints para que sean intuitivos y fáciles de mantener. Un cambio común es añadir un prefijo como **/api** a las rutas, lo que ayuda a distinguir los recursos de la **APL()** del resto de las rutas que puede tener la aplicación. En nuestro caso, pasaremos de `localhost:8080/books` a `localhost:8080/api/books`, lo que mejora la claridad al indicar que estas rutas pertenecen a la **APL()** de la aplicación. Este enfoque es útil para proyectos que combinan **APL()** y otros tipos de recursos, como páginas web o servicios estáticos, ya que permite tener una estructura más limpia y comprensible.

Además, este tipo de organización facilita el mantenimiento de la aplicación a medida que crece. Los equipos de desarrollo pueden añadir fácilmente nuevas funcionalidades sin que se mezcle la lógica de negocio con otros aspectos de la aplicación.

```
1 @RestController
2 @RequestMapping(BookController.URL)
3 @RequiredArgsConstructor
4 public class BookController {
5
6     public static final String URL = "/api/books";
```

También es común añadir la versión de la **APL()** en los endpoints, como `localhost:8080/api/v1/books`. Esto permite mantener múltiples versiones de la **APL...** en producción, ofreciendo flexibilidad para hacer cambios sin romper las integraciones con los clientes existentes. En este caso, no hemos optado por versionar la **APL()**, ya que no es necesario en nuestra aplicación actual.

Modelos de la capa de presentación

En la capa de presentación, a veces es conveniente utilizar modelos diferentes a los de la capa de dominio para adaptarse a las necesidades de la interfaz y la experiencia del usuario. Esto permite reducir la cantidad de información enviada en ciertas situaciones y evitar exponer detalles innecesarios.

Por ejemplo, para el método *findAllBooks()* del controlador, que devuelve un listado de libros, podríamos devolver sólo algunos atributos, como *isbn*, *titleEs*, *titleEn*, *basePrice*, *discountPercentage*, *price* y *cover*. Aquí podríamos crear un modelo de presentación simplificado que contenga solo estos campos.

Por otro lado, para el detalle de un libro (*getBookByIsbn()*), podemos usar un modelo completo que incluya toda la información relevante que contenga todos los atributos del libro, incluyendo sinopsis, editoriales y autores. Esto asegura que el modelo utilizado en la respuesta del controlador esté optimizado y ajustado al contexto de cada operación, manteniendo la separación de responsabilidades entre la capa de presentación y la capa de dominio.

Para el resto de modelos sería lo mismo. Por ejemplo, en el detalle de un libro podemos devolver sólo el nombre y *slug* de los autores, mientras que en el detalle de un autor devolveríamos todos los datos del mismo.

Vamos a crear los modelos para ver diferentes opciones. La idea es que la respuesta del listado de libros sea:

```
1  {
2      "data": [
3          {
4              "isbn": "9780142424179",
5              "titleEs": "El principito",
6              "titleEn": "The Little Prince",
7              "basePrice": 15.99,
8              "discountPercentage": 10.00,
9              "price": 14.39,
10             "cover": "http://images.cesguiro.es/books/9780
11         },
12         {
13             "isbn": "9780142410363",
14             "titleEs": "Matilda",
15             "titleEn": "Matilda",
16             "basePrice": 14.99,
17             "discountPercentage": 5.00,
18             "price": 14.24,
19             "cover": "http://images.cesguiro.es/books/9780
20         },
21         {
22             "isbn": "9780142418222",
23             "titleEs": "Charlie y la fábrica de chocolate",
24             "titleEn": "Charlie and the Chocolate Factory",
25             "basePrice": 13.99,
26             "discountPercentage": 15.00,
27             "price": 11.89,
28             "cover": "http://images.cesguiro.es/books/9780
29         },
30         ...
31     ],
32     "pageNumber": 1,
33     "pageSize": 10,
34     "totalElements": 24,
35     "totalPages": 3
36 }
```

Por otra parte, el detalle de un libro contendrá la siguiente información:

```

1  {
2      "isbn": "9780060557912",
3      "titleEs": "Buenos presagios",
4      "titleEn": "Good Omens",
5      "synopsisEs": "Buenos presagios cuenta la historia de
6      "synopsisEn": "Good Omens tells the story of an angel
7      "basePrice": 16.99,
8      "discountPercentage": 0.00,
9      "price": 16.99,
10     "cover": "http://images.cesguiro.es/books/978006055791
11     "publicationDate": "1990-05-01",
12     "publisher": {
13         "name": "Alfaguara",
14         "slug": "alfaguara"
15     },
16     "authors": [
17         {
18             "name": "Terry Pratchett",
19             "slug": "terry-pratchett"
20         },
21         {
22             "name": "Neil Gaiman",
23             "slug": "neil-gaiman"
24         }
25     ]
26 }

```

En nuestro caso, dividiremos los modelos en dos tipos: *Summary* y *Detail*, cada uno destinado a un propósito específico. Los modelos *Summary* se utilizarán para representar el resumen de los modelos, mientras que los modelos *Detail* ofrecerán información detallada sobre un recurso individual.

Para implementar esta división de manera eficiente, podemos aprovechar el tipo de datos `Record`

(<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Record.html>) introducido en Java 16. Los records nos permiten crear clases inmutables de manera concisa, ideales para representar estos modelos sin la sobrecarga de escribir código repetitivo como getters, setters o constructores.

Los modelos los crearemos en el package `/controller/webModel/response` para las respuestas y `/controller/webModel/request` para los datos de entrada (para insertar libros, actualizar autores...).

De esta forma, nuestros modelos de respuesta quedarían:

```

1  public record AuthorDetailResponse (
2      String name,
3      String nationality,
4      String biographyEs,
5      String biographyEn,
6      Integer birthYear,
7      Integer deathYear,
8      String slug
9  ) {
10 }

```



```
1 public record AuthorSummaryResponse(  
2     String name,  
3     String slug  
4 ) {  
5 }  
  
1 public record BookDetailResponse(  
2     String isbn,  
3     String titleEs,  
4     String titleEn,  
5     String synopsisEs,  
6     String synopsisEn,  
7     BigDecimal basePrice,  
8     BigDecimal discountPercentage,  
9     BigDecimal price,  
10    String cover,  
11    LocalDate publicationDate,  
12    PublisherSummaryResponse publisher,  
13    List<AuthorSummaryResponse> authors  
14 )  
15 {}  
  
1 public record BookSummaryResponse(  
2     String isbn,  
3     String titleEs,  
4     String titleEn,  
5     BigDecimal basePrice,  
6     BigDecimal discountPercentage,  
7     BigDecimal price,  
8     String cover  
9 ) {  
10 }
```

```
1 public record PublisherDetailResponse(  
2     String name,  
3     String slug  
4 ) {  
5 }  
  
1 public record PublisherSummaryResponse(  
2     String name,  
3     String slug  
4 ) {  
5 }
```

Para convertir entre los modelos de dominio y los modelos de presentación, tendremos que crear nuestros propios mapeadores, ya sea manuales o utilizando MapStruct, tal como se hizo en temas anteriores. Esto permitirá mantener la capa de presentación separada de la lógica de dominio y adaptar los datos al formato que espera la API().

Respuesta

Para utilizar los nuevos modelos en las respuestas del controlador, debemos realizar cambios en los métodos que manejan las solicitudes. Por ejemplo, cambiaremos el tipo de retorno del método *findAllBooks()* para que devuelva

Page<BookSummaryResponse> y el método *getBookByIsbn()* para que devuelva un objeto *BookDetailResponse*:

```

1  @RestController
2  @RequestMapping("/api/books")
3  public class BookController {
4
5      private final BookService bookService;
6
7      public BookController(BookService bookService) {
8          this.bookService = bookService;
9      }
10
11     @GetMapping
12     public Page<BookSummaryResponse> findAllBooks(@Request
13                                                    @Request
14                                                    Page<BookDto> bookDtoPage = bookService.getAll(pag
15
16     List<BookSummaryResponse> bookSummaries = bookDtoF
17         .map(BookMapper::fromBookDtoToBookSummaryF
18         .toList();
19     return new Page<>(
20         bookSummaries,
21         bookDtoPage.pageNumber(),
22         bookDtoPage.pageSize(),
23         bookDtoPage.totalElements(),
24         bookDtoPage.totalPages()
25     );
26 }
27
28 @GetMapping("/{isbn}")
29 public BookDetailResponse getBookByIsbn(@PathVariable
30     return BookMapper.fromBookDtoToBookDetailResponse(
31 }

```

ResponseEntity

Para mejorar el manejo de las respuestas en el controlador, vamos a cambiar el tipo de respuesta a *ResponseEntity* (<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/http/ResponseEntity.html>). La clase *ResponseEntity* es una parte fundamental de Spring que permite representar una respuesta HTTP, incluyendo el cuerpo, los encabezados y el estado de la respuesta. Esto proporciona un mayor control sobre cómo se envían las respuestas al cliente:

```

1  @GetMapping
2  public ResponseEntity<Page<BookSummaryResponse>> findAllBooks(
3
4      Page<BookDto> bookDtoPage = bookService.getAll(page, size);
5
6      List<BookSummaryResponse> bookSummaries = bookDtoPage.
7          .map(BookMapper::fromBookDtoToBookSummaryResponse)
8          .toList();
9
10     Page<BookSummaryResponse> bookSummaryPage = new Page<>
11         (bookSummaries,
12         bookDtoPage pageNumber(),
13         bookDtoPage pageSize(),
14         bookDtoPage totalElements(),
15         bookDtoPage totalPages());
16
17     return new ResponseEntity<>(bookSummaryPage, HttpStatus.OK);
18 }
19
20 @GetMapping("/{isbn}")
21 public ResponseEntity<BookDetailResponse> getBookByIsbn(@RequestParam
22     BookDetailResponse bookDetailResponse = BookMapper.fromBookDtoToBookDetailResponse(bookDto);
23     return new ResponseEntity<>(bookDetailResponse, HttpStatus.OK);
24 }

```

Usar *ResponseEntity* en el controlador ofrece varias ventajas que mejoran la interacción con el cliente de la API():

- **Control del estado de la respuesta:** Permite establecer el código de estado HTTP que se desea enviar al cliente, lo que es útil para indicar el resultado de la operación (por ejemplo, `HttpStatus.OK`, `HttpStatus.NOT_FOUND`, etc.).
- **Personalización de encabezados:** Puedes agregar encabezados personalizados a la respuesta, lo que puede ser útil para informar sobre el tipo de contenido, la caché, etc.
- **Flexibilidad en el cuerpo de la respuesta:** Permite enviar cualquier tipo de objeto como cuerpo de la respuesta, manteniendo la capacidad de enviar datos complejos sin perder la estructura de la respuesta HTTP.

Tratamiento de excepciones

Podríamos tratar las excepciones en los controladores utilizando bloques *try... catch*, pero esto podría resultar en un código repetitivo y difícil de mantener, especialmente en aplicaciones grandes. Afortunadamente, Spring nos ofrece una forma de centralizar el tratamiento de excepciones a través de las anotaciones `@ControllerAdvice` (<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/bind/annotation/ControllerAdvice.html>) y `@RestControllerAdvice` (<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/bind/annotation/RestControllerAdvice.html>).

El uso de **@ControllerAdvice** permite definir un único lugar para manejar todas las excepciones que pueden ocurrir en los controladores de la aplicación. Esto no solo simplifica la gestión de errores al evitar la duplicación de lógica en cada

controlador, sino que también asegura que todas las respuestas de error tengan una estructura y formato consistentes, facilitando así a los consumidores de la API () la comprensión y el manejo de los errores.

Sin embargo, es importante distinguir entre ambos tipos de controladores:

- **@ControllerAdvice** está pensado para aplicaciones MVC tradicionales que devuelven vistas (Thymeleaf, JSP...). Cuando maneja una excepción, el valor devuelto se interpreta como una vista, a menos que se anote el método con `@ResponseBody` (<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/bind/annotation/ResponseBody.html>).
- **@RestControllerAdvice** es una especialización de `@ControllerAdvice` que incluye automáticamente `@ResponseBody`. Esto significa que todas las respuestas generadas se serializan directamente a JSON (o al formato configurado), igual que ocurre con los controladores anotados con `@RestController`.

Por ello, **en APIs REST es recomendable utilizar @RestControllerAdvice**, ya que asegura que todas las respuestas de error se devuelvan correctamente serializadas sin necesidad de añadir `@ResponseBody` en cada método.

Además, `@ControllerAdvice` / `@RestControllerAdvice` permiten manejar diferentes tipos de excepciones de manera específica, devolviendo respuestas personalizadas según la situación, con códigos HTTP adecuados y mensajes descriptivos. Así se logra un código más limpio y mantenible, separando la lógica de manejo de errores de la lógica de negocio.

Para manejar las excepciones en la aplicación de manera efectiva, crearemos dos clases en el paquete *exception*: *ErrorMessage* y *ApiExceptionHandler*.

La clase *ErrorMessage* contiene dos propiedades: *error* y *message*. Estas propiedades se utilizan para enviar información relevante sobre el error que ocurrió.

```
1  public class ErrorMessage {
2
3      private final String error;
4      private final String message;
5
6      public ErrorMessage(Exception exception) {
7          this.error = exception.getClass().getSimpleName();
8          this.message = exception.getMessage();
9      }
10
11     public String getError() {
12         return error;
13     }
14
15     public String getMessage() {
16         return message;
17     }
18 }
```

Asegúrate de añadir los getters en las clases de respuesta (*ErrorMessage*), ya que si no Jackson no podrá serializar los campos a JSON y el *@RestControllerAdvice* no devolverá correctamente el cuerpo de error al cliente.

La clase *ApiExceptionHandler* utiliza la anotación *@RestControllerAdvice* para manejar excepciones globalmente. Aquí, se maneja la excepción *ResourceNotFoundException* devolviendo un *ErrorMessage* con un código de estado 404 (NO FOUND).

Para capturar el resto de las excepciones de manera más segura y evitar exponer detalles sensibles sobre el error interno, podemos crear un método en la clase *ApiExceptionHandler* para que devuelva un mensaje genérico como "Internal error". Esto garantiza que cualquier excepción no manejada específicamente también se trate de manera adecuada, proporcionando una respuesta coherente al cliente.

```
1  @RestControllerAdvice
2  public class ApiExceptionHandler {
3
4      @ResponseStatus(HttpStatus.NOT_FOUND)
5      @ExceptionHandler({
6          ResourceNotFoundException.class
7      })
8      @ResponseBody
9      public ErrorMessage notFoundRequest(ResourceNotFoundException e) {
10         return new ErrorMessage(exception);
11     }
12
13     @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
14     @ExceptionHandler(Exception.class)
15     @ResponseBody
16     public ErrorMessage handleGeneralException(Exception e) {
17         return new ErrorMessage(exception);
18     }
19 }
```

Inserciones, actualizaciones y borrados

Para las operaciones de creación y actualización de libros, necesitaremos modelos de entrada (request) distintos, ya que la información requerida no siempre es la misma:

- **BookInsertRequest:** Se utiliza al crear un nuevo libro.
- **BookUpdateRequest:** Se utiliza al actualizar un libro existente e incluye el identificador id.

```

1  public record BookInsertRequest (
2      String isbn,
3      String titleEs,
4      String titleEn,
5      String synopsisEs,
6      String synopsisEn,
7      String cover,
8      @JsonFormat(pattern = "dd-MM-yyyy")
9      LocalDate publicationDate,
10     BigDecimal basePrice,
11     BigDecimal discountPercentage,
12     Long publisherId,
13     Long[] authorIds
14 ) {
15 }

1  public record BookUpdateRequest (
2      Long id,
3      String isbn,
4      String titleEs,
5      String titleEn,
6      String synopsisEs,
7      String synopsisEn,
8      String cover,
9      @JsonFormat(shape = JsonFormat.Shape.STRING, patte
10     LocalDate publicationDate,
11     BigDecimal basePrice,
12     BigDecimal discountPercentage,
13     Long publisherId,
14     Long[] authorIds
15 ) {
16 }

```

Anotaciones JSON y formato fechas

La anotación **@JsonFormat** de Jackson permite definir el formato de serialización y deserialización de los campos, en este caso `LocalDate`.

Por defecto, Jackson 2.X no sabe cómo convertir automáticamente `LocalDate` a un string en el formato deseado, y puede generar errores de parseo.

Para que Spring con Jackson 2.X maneje correctamente `LocalDate` y otras clases de la APL() de tiempo (java.time.*), es necesario registrar el módulo `jackson-datatype-jsr310`.

En el `pom.xml`, añadimos:

```

1  <dependency>
2      <groupId>com.fasterxml.jackson.datatype</groupId>
3      <artifactId>jackson-datatype-jsr310</artifactId>
4      <version>${com.fasterxml.jackson.version}</version>
5  </dependency>

```

Esto permite que Jackson pueda serializar y deserializar correctamente `LocalDate`, `LocalDateTime` y otras clases de la APL() de tiempo de Java 8+.

Esta dependencia es especialmente importante en versiones de Spring que utilizan Jackson 2.X, ya que no está activada por defecto.

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-Noncommercial-Share Alike 4.0 International

08_3 - Capa presentación: idioma, paginación y roles

En esta sección, abordaremos aspectos esenciales como la gestión del idioma, la paginación de los datos y la implementación de roles de usuario. Comenzaremos con el manejo del idioma, que es crucial para ofrecer una experiencia de usuario adaptada a las preferencias lingüísticas de cada usuario.

Idioma

Crearemos una configuración que permita que nuestra aplicación soporte múltiples idiomas de manera dinámica, utilizando la cabecera `Accept-Language` (https://es.wikipedia.org/wiki/Anexo:Cabeceras_HTTP) de las solicitudes HTTP. Esto nos permitirá ofrecer contenido en el idioma que el usuario prefiera, mejorando así la accesibilidad y la usabilidad de nuestra aplicación.

A través de la implementación de interceptores y un gestor de idiomas, garantiremos que el idioma seleccionado por el usuario se aplique en todas las partes de la aplicación. Esto nos permitirá, por ejemplo, traducir títulos, mensajes de error y otros elementos visuales según el idioma seleccionado, brindando así una experiencia más personalizada y atractiva.

Para almacenar el idioma, usaremos la clase `Locale`

(<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Locale.html>) de Java. La clase `Locale` representa un conjunto de configuraciones específicas de un idioma y una región. Se utiliza para identificar la configuración regional de la aplicación, lo que es crucial para la internacionalización. Un objeto `Locale` puede contener información sobre el idioma (como "es" para español o "en" para inglés), el país (como "ES" para España o "US" para Estados Unidos) y, en algunos casos, la variante específica de un idioma. Esto permite a la aplicación adaptar el contenido, formato de fechas y números, y otros aspectos a las preferencias del usuario.

Lo primero será crear el package **locale** en **common**, donde meteremos todas nuestras clases.

LanguageUtils

Primero, crearemos la clase **LanguageUtils** que contendrá dos métodos: **setCurrentLocale()**, para establecer el `Locale` proporcionado, y **getCurrentLanguage()**, que devolverá el idioma correspondiente o el idioma por defecto si no se ha establecido ninguno.

La clase utiliza un objeto `ThreadLocal` (<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/ThreadLocal.html>) para almacenar el `Locale` actual, asegurando que cada hilo de ejecución tenga su propio valor.

La clase **ThreadLocal** en Java proporciona una forma de almacenar datos de manera que cada hilo tenga su propia copia de la información. Esto es útil en aplicaciones multihilo, ya que evita que los hilos compartan el mismo estado y, por lo tanto, minimiza los problemas de concurrencia. Cada vez que un hilo accede a un objeto **ThreadLocal**, obtiene un valor específico para ese hilo, lo que permite un manejo seguro de datos sin necesidad de sincronización explícita.

```
1 public class LanguageUtils {  
2  
3     private static final ThreadLocal<Locale> currentLocale = new ThreadLocal<>();  
4  
5     public static void setCurrentLocale(Locale locale) {  
6         currentLocale.set(locale);  
7     }  
8  
9     public static String getCurrentLanguage() {  
10        Locale locale = currentLocale.get();  
11        return locale != null ? locale.getLanguage() : Locale.getDefault().getLanguage();  
12    }  
13 }
```

Interceptores en Java

Los interceptores en Java son componentes que permiten interceptar y modificar el comportamiento de las solicitudes y respuestas en una aplicación web. Actúan como filtros que pueden realizar diversas acciones antes y después de que se procese una solicitud, lo que permite implementar funcionalidades como la autenticación, la autorización, el registro y la gestión de idiomas.

En el contexto de Spring, los interceptores son utilizados principalmente para ejecutar lógica de pre-procesamiento y post-procesamiento en las peticiones HTTP. Esto es útil para tareas como validar permisos de acceso, registrar información sobre la solicitud o, en nuestro caso, gestionar el idioma de la aplicación según las preferencias del usuario.

Para adaptar nuestra aplicación a diferentes idiomas, hemos creado un interceptor personalizado que extiende `LocaleChangeInterceptor` (<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/servlet/i18n/LocaleChangeInterceptor.html>). Este interceptor se encargará de leer la cabecera **Accept-Language** de las solicitudes entrantes y establecer el `Locale` actual utilizando la clase **LanguageUtils**.


```

1  public class CustomLocaleChangeInterceptor extends LocaleChangeInterceptor {
2
3      private final String defaultLanguage;
4
5      public CustomLocaleChangeInterceptor(String defaultLanguage) {
6          this.defaultLanguage = defaultLanguage;
7      }
8
9      @Override
10     public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) {
11         String lang = request.getHeader("Accept-Language");
12         Locale locale = lang != null ? Locale.forLanguageTag(lang) : Locale.of(defaultLanguage);
13
14         LanguageUtils.setCurrentLocale(locale);
15
16         return super.preHandle(request, response, handler);
17     }
18 }

```

La clase tendrá la propiedad **defaultLanguage**, la cual inyectaremos en el constructor.

El método **preHandle** será el encargado de ejecutar las acciones correspondientes cuando se lance el filtro.

La interfaz `HttpServletRequest` (<https://jakarta.ee/specifications/platform/9/apidocs/jakarta/servlet/http/httpServletRequest>) es parte de la **API** de servlets de Java y representa la solicitud HTTP que un cliente envía a un servidor. Proporciona métodos para acceder a información sobre la solicitud, como los parámetros, encabezados y atributos. En el contexto del interceptor, se utiliza para obtener la cabecera **Accept-Language** que especifica el idioma preferido por el cliente.

La interfaz `HttpServletResponse` (<https://jakarta.ee/specifications/platform/9/apidocs/jakarta/servlet/http/httpServletResponse>) también forma parte de la **API** de servlets y representa la respuesta HTTP que un servidor envía de vuelta al cliente. Permite al desarrollador modificar la respuesta, como establecer códigos de estado, agregar encabezados y enviar datos al cliente. En el interceptor, aunque no se modifica directamente, se pasa como argumento para permitir que se interactúe con la respuesta si es necesario.

El parámetro **handler** es un objeto que representa el controlador que procesará la solicitud. En el contexto de Spring, puede ser un controlador (un método en un controlador anotado con **@RequestMapping**) u otros componentes que manejan la lógica de la aplicación. Este parámetro se utiliza para determinar qué acción tomar después de que se complete el procesamiento del interceptor.

El método **preHandle** se ejecuta antes de que la solicitud sea procesada por el controlador. En este método, primero se obtiene el valor de la cabecera **Accept-Language** de la solicitud. Si se encuentra un valor, se convierte a un objeto **Locale** utilizando el método `forLanguageTag` ([https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Locale.html#forLanguageTag\(java.lang.String\)](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Locale.html#forLanguageTag(java.lang.String))). En caso de que no haya un valor, se utiliza el idioma por defecto.

El método **Locale.forLanguageTag(String languageTag)** es un método estático de la clase **Locale** que convierte un identificador de idioma (tag) en un objeto **Locale**. Este método permite crear fácilmente instancias de **Locale** utilizando una representación estándar de idioma, como "es-ES" para español de España o "en-US" para inglés de Estados Unidos.

Luego, se establece el **Locale** actual en el contexto de hilo a través de la clase **LanguageUtils**, asegurando que el idioma seleccionado se utilice a lo largo de la aplicación.

Finalmente, se llama al método `preHandle` ([https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/servlet/18n/LocaleChangeInterceptor.html#preHandle\(jakarta.servlet.http.HttpServletRequest,jakarta.servlet.http.HttpServletResponse,HandlerMethod\)](https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/servlet/18n/LocaleChangeInterceptor.html#preHandle(jakarta.servlet.http.HttpServletRequest,jakarta.servlet.http.HttpServletResponse,HandlerMethod))) de la clase padre para continuar con el procesamiento normal de la solicitud. Esto es importante porque la clase padre puede contener lógica que también necesita ejecutarse para garantizar que el interceptor funcione correctamente. Al llamar a este método, se permite que se ejecute la lógica base de la clase padre antes de continuar con el procesamiento de la solicitud.

LocaleConfig

La clase **LocaleConfig** es una clase de configuración que se encargará de definir cómo se gestiona el idioma en la aplicación. Esta clase está anotada con **@Configuration**, lo que indica que contiene definiciones de *beans* que se utilizarán en el contexto de Spring.

Para definir el idioma por defecto, vamos a utilizar el archivo **application-properties**, donde crearemos una propiedad con el valor:

```
app.language.default=es
```

Spring, nos permite inyectar los valores de las propiedades del fichero en clases que pueda manejar (que sean beans). Al anotar la clase con **@Configuration**, ésta se convierte en un bean de Spring, con lo que podemos acceder a dichas propiedades.

Para ello, usaremos la anotación de Spring **@Value**, encerrando entre **"\${}"** el nombre de la propiedad.

```

1  @Configuration
2  public class LocaleConfig implements WebMvcConfigurer {
3
4      @Value("${app.language.default}")
5      private String defaultLanguage;
6
7      @Override
8      public void addInterceptors(InterceptorRegistry registry) {
9          registry.addInterceptor(new CustomLocaleChangeInterceptor(defaultLanguage));
10     }
11 }

```

El método **addInterceptors()** es una implementación de la interfaz `WebMvcConfigurer` (<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/servlet/config/annotation/WebMvcConfigurer.html>) y se utiliza para registrar interceptores adicionales en el contexto de Spring. En este caso, se añade nuestro interceptor personalizado

CustomLocaleChangeInterceptor. Al agregar este interceptor, se asegura que cada vez que se procese una solicitud, se ejecutará la lógica de nuestro interceptor personalizado antes de que llegue a los controladores, permitiendo así que la aplicación maneje correctamente el idioma según las preferencias del usuario.

Ahora, ya podemos saber el idioma del usuario en nuestros mapeadores:

```

1  @RequiredArgsConstructor
2  public class CategoryRowMapper implements CustomRowMapper<Category> {
3
4      @Override
5      public Category mapRow(ResultSet resultSet, int rowNum) throws SQLException {
6          String language = LanguageUtils.getCurrentLanguage();
7          Category category = new Category();
8          category.setName(resultSet.getString("categories.name_" + language));
9          category.setSlug(resultSet.getString("categories.slug_"));
10         return category;
11     }
12 }

```

Es importante tener en cuenta que si el usuario solicita un idioma que no está disponible en la base de datos, esto puede provocar un error. Por lo tanto, sería recomendable implementar una verificación que asegure que el idioma solicitado tenga un campo correspondiente en la base de datos antes de intentar acceder a él. Esto puede ayudar a evitar excepciones y garantizar una experiencia de usuario más robusta.

Si el usuario no proporciona la cabecera **Accept-Language** o envía un valor como **es-***, la respuesta será:

```

1  {
2      "isbn": "9780142424179",
3      "title": "El principito",
4      "price": 15.99,
5      "discount": 10.0,
6      "cover": "http://images.cesguiro.es/books/9780142424179.webp"
7  }

```

Por otro lado, si el usuario incluye la cabecera **Accept-Language** con el valor **en-GB()**, la respuesta cambiará a:

```

1  {
2      "isbn": "9780142424179",
3      "title": "The Little Prince",
4      "price": 15.99,
5      "discount": 10.0,
6      "cover": "http://images.cesguiro.es/books/9780142424179.webp"
7  }

```

Paginación

La paginación es un aspecto esencial en la gestión de grandes volúmenes de datos, ya que permite dividir los resultados en varias páginas, mejorando la usabilidad y el rendimiento de la aplicación. Para implementar la paginación en nuestra aplicación, crearemos la clase **PaginatedResponse** en el package **webmodel**, que se encargará de encapsular la respuesta paginada.

```

1  @Data
2  @AllArgsConstructor
3  public class PaginatedResponse<T> {
4      private List<T> data;
5      private int total;
6      private int currentPage;
7      private int pageSize;
8      private String next;
9      private String previous;
10
11     public PaginatedResponse(List<T> data, int total, int currentPage, int pageSize, String baseUrl) {
12         this.data = data;
13         this.total = total;
14         this.currentPage = currentPage;
15         this.pageSize = pageSize;
16         this.next = createNextLink(baseUrl);
17         this.previous = createPreviousLink(baseUrl);
18     }
19
20     private String createNextLink(String baseUrl) {
21         if(currentPage * pageSize < total) {
22             return baseUrl + "?page=" + (currentPage + 1) + "&size=" + pageSize;
23         }
24         return null;
25     }
26
27     private String createPreviousLink(String baseUrl) {
28         if(currentPage > 1) {
29             return baseUrl + "?page=" + (currentPage - 1) + "&size=" + pageSize;
30         }
31         return null;
32     }
33 }

```

Igual que con el idioma por defecto, para facilitar el proceso crearemos un par de propiedades en **application.properties**, la url base de nuestra aplicación, y el número de recursos por página por defecto:

```

1 app.base.url=http://localhost:8080
2 app.pageSize.default=10

```

A continuación, inyectaremos esos valores en nuestros controladores y modificaremos el método **getAll()** en el controlador de libros (**BookController**) para utilizar la paginación:

```

1 @RestController
2 @RequiredArgsConstructor
3 @RequestMapping(BookController.URL)
4 public class BookController {
5
6     public static final String URL = "/api/books";
7     @Value("${app.base.url}")
8     private String baseUrl;
9
10    @Value("${app.pageSize.default}")
11    private String defaultPageSize;
12
13    private final BookService bookService;
14
15    @GetMapping
16    public ResponseEntity<PaginatedResponse<BookCollection>> getAll(
17        @RequestParam(defaultValue = "1") int page,
18        @RequestParam(required = false) Integer size) {
19        int pageSize = (size != null) ? size : Integer.parseInt(defaultPageSize);
20        List<BookCollection> books = bookService
21            .getAll(page - 1, pageSize)
22            .stream()
23            .map(BookMapper.INSTANCE::toBookCollection)
24            .toList();
25
26        int total = bookService.count();
27
28        PaginatedResponse<BookCollection> response = new PaginatedResponse<>(books, total, page, page < total ? page + 1 : null);
29        return new ResponseEntity<>(response, HttpStatus.OK);
30    }

```

Para obtener el total de registros, utilizaremos un método en el servicio que contará la cantidad de libros disponibles en la base de datos. Esto nos permitirá proporcionar información sobre cuántos elementos hay en total, lo que es crucial para la paginación. De esta manera, podemos calcular si hay una página siguiente o anterior y gestionar la navegación entre las diferentes páginas de resultados.

La anotación **@RequestParam** se utiliza para extraer parámetros de consulta de la **URL()** de la solicitud HTTP. En este caso, se utilizará para recibir los valores de **page** y **size** desde la petición. Page tendrá un valor predeterminado de 1, por si el usuario no especifica este parámetro. El parámetro size, lo hacemos no obligatorio (con **required = false**) y miramos si nos lo han pasado. Si no, leemos el valor por defecto.

En la clase **BookService**, agregaremos dos métodos para manejar la lógica de negocio:

```

1 List<Book> getAll(int page, int size);
2 int count();

```

La implementación de estos métodos será:

```

1 @Override
2 public List<Book> getAll(int page, int size) {
3     return bookRepository.getAll(page, size);
4 }
5
6 @Override
7 public int count() {
8     return bookRepository.count();
9 }

```

En la interfaz **BookRepository**, también definiremos los métodos necesarios para interactuar con la base de datos:

```

1 List<Book> getAll(int page, int size);
2 int count();

```

La implementación en la clase que maneja la persistencia será:

```

1 @Override
2 public List<Book> getAll(int page, int size) {
3     String sql = """
4         SELECT * FROM books
5         LIMIT ? OFFSET ?
6     """;
7     return jdbcTemplate.query(sql, new BookRowMapper(), size, page * size);
8 }
9
10 @Override
11 public int count() {
12     String sql = """
13         SELECT COUNT(*) FROM books
14     """;
15     return jdbcTemplate.queryForObject(sql, Integer.class);
16 }

```

Con esta implementación, la aplicación ahora puede manejar la paginación de los resultados de los libros, lo que proporciona una experiencia de usuario más eficiente.

Roles

En nuestra aplicación, es fundamental gestionar de manera efectiva la información presentada a diferentes tipos de usuarios. Mientras que los usuarios estándar necesitan un acceso simplificado y centrado en la experiencia de lectura, los administradores requieren acceso a información más detallada para poder gestionar los libros adecuadamente. Esto genera una problemática: ¿cómo proporcionar la información necesaria sin sobrecargar al usuario?

Por ejemplo, cuando un usuario solicita los detalles de un libro, la respuesta contiene el título en el idioma solicitado:

```
1 {
2   "isbn": "9780142424179",
3   "title": "El principito",
4   "price": 15.99,
5   "discount": 10.0,
6   "cover": "http://images.cesguiro.es/books/9780142424179.webp"
7   ...
8 }
```

Sin embargo, los administradores necesitan editar libros y, para eso, requieren acceso a campos como *title_es* y *title_en* de la bbdd, así como información adicional sobre los autores y las categorías. La solución es separar la lógica del usuario de la del administrador, organizando nuestra aplicación en paquetes específicos. Así, cuando un administrador solicita los detalles de un libro, la respuesta incluye un JSON más completo, como el siguiente:

```
1 {
2   "id": 1,
3   "isbn": "9780142424179",
4   "titleEs": "El principito",
5   "titleEn": "The Little Prince",
6   "synopsisEs": "El principito es una novela corta y la obra más famosa del escritor y ...",
7   "synopsisEn": "The Little Prince is a novella and the most famous work of the French ..."
8   ...
9 }
```

Para manejar la diferenciación entre usuarios y administradores en nuestra aplicación, implementamos una clara separación de componentes por roles. Esta separación se traduce en la creación de paquetes específicos dentro de cada capa de nuestra arquitectura: **user** y **admin**. Esta estructura no solo ayuda a mantener el código organizado, sino que también permite escalar y mantener la aplicación de manera más efectiva.

Podríamos optar por implementar dos aplicaciones diferentes: una para usuarios finales y otra para administradores. Esta separación facilitaría la gestión de permisos y roles, además de permitir un enfoque más específico para cada tipo de usuario. En nuestro caso, vamos a juntar las dos funcionalidades en la misma aplicación

Para implementar la separación de funcionalidades por roles, comenzaremos por renombrar todos nuestros controladores, servicios y repositorios. Por ejemplo, el controlador de libros se convertirá en *BookUserController*, el servicio en *BookUserService* y el repositorio en *BookRepository*. Esta convención de nombres nos ayudará a identificar claramente qué componentes están destinados a la lógica de usuario.

Además, organizaremos todos nuestros componentes relacionados en el paquete *user*. Esta estructura facilitará la navegación y mantenimiento del código, permitiendo una gestión más eficiente de los roles y sus respectivas funcionalidades dentro de la aplicación.

Lo primero será crear los modelos de dominio de administrador, incluyendo todos los campos con los idiomas:

```
1 public class Author {
2
3     private long id;
4     private String name;
5     private String nationality;
6     private String biographyEs;
7     private String biographyEn;
8     private int birthYear;
9     private int deathYear;
10 }

1 public class Category {
2
3     private long id;
4     private String nameEs;
5     private String nameEn;
6     private String slug;
7 }

1 public class Genre {
2
3     private long id;
4     private String nameEs;
5     private String nameEn;
6     private String slug;
7 }

1 public class Publisher {
2
3     private long id;
4     private String name;
5     private String slug;
6 }
```

```

1  public class Book {
2
3      private long id;
4      private String isbn;
5      private String titleEs;
6      private String titleEn;
7      private String synopsisEs;
8      private String synopsisEn;
9      private BigDecimal price;
10     private float discount;
11     private String cover;
12     private Publisher publisher;
13     private Category category;
14     private List<Genre> genres;
15     private List<Author> authors;
16 }

```

En el modelo `Book` del administrador, añadiremos el siguiente método para gestionar la visualización del título del libro según el idioma elegido por el administrador:

```

1  public String getTitle() {
2      String language = LanguageUtils.getCurrentLanguage();
3      if ("en".equals(language)) {
4          return titleEn;
5      }
6      return titleEs;
7  }

```

Este método permitirá que, al mostrar los detalles del libro, se devuelva el título en el idioma seleccionado. Así, cuando el administrador consulte el listado de libros, los datos se presentarán en el siguiente formato:

```

1  "data": [
2      {
3          "isbn": "9780142424179",
4          "title": "El principito"
5      },
6      {
7          "isbn": "9780142410363",
8          "title": "Matilda"
9      },
10     {
11         "isbn": "9780142418222",
12         "title": "Charlie y la fábrica de chocolate"
13     },
14     {
15         ...

```

A continuación, crearemos el modelo **BookCollection** en la capa de presentación para el administrador. Este modelo representará los datos simplificados que se mostrarán en el listado de libros. La definición del modelo será la siguiente:

```

1  public record BookCollection(
2      String isbn,
3      String title
4  ) { }

```

Este modelo se ubicará en el package *admin*, permitiendo que se utilice específicamente para las funcionalidades del administrador en la gestión de libros. De esta manera, garantizamos una separación clara de las responsabilidades y facilitamos el acceso a la información necesaria para la administración de la biblioteca.

Para facilitar la gestión de respuestas paginadas y asegurar la reutilización de componentes entre los controladores de usuario y administrador, crearemos un paquete *common* dentro de la capa de controlador. En este paquete, colocaremos nuestra clase *PaginatedResponse*, que se encargará de estructurar las respuestas paginadas de manera consistente.

Lo siguiente será crear el mapeador *BookMapper*. Este mapeador se encargará de convertir los objetos del modelo de dominio *Book* al modelo de presentación *BookCollection*.

```

1  @Mapper
2  public interface BookMapper {
3
4      BookMapper INSTANCE = Mappers.getMapper(BookMapper.class);
5
6      @Mapping(target = "title", source = "book.title")
7      BookCollection toBookCollection(Book book);
8
9  }

```

En este mapeador, utilizamos el método *getTitle()* que hemos definido previamente en la clase *Book*. Este método se encarga de devolver el título del libro en el idioma correspondiente, según la preferencia del administrador. De esta forma, aseguramos que el modelo de presentación refleje correctamente la información localizada que el administrador necesita para gestionar los libros.

A continuación, procederemos a crear los servicios del administrador. Estos servicios serán prácticamente idénticos a los servicios del usuario, con la diferencia de que llamarán a los repositorios del administrador para obtener y gestionar los datos necesarios. La implementación de estos servicios asegurará que la lógica de negocio para el administrador esté claramente separada de la del usuario:

```

1  @Service
2  @RequiredArgsConstructor
3  public class BookAdminServiceImpl implements BookAdminService {
4
5      private final BookAdminRepository bookAdminRepository;
6
7      @Override
8      public List<Book> getAll() {
9          return bookAdminRepository.getAll();
10     }
11
12     @Override
13     public List<Book> getAll(int page, int size) {
14         return bookAdminRepository.getAll(page, size);
15     }
16
17     @Override
18     public int count() {
19         return bookAdminRepository.count();
20     }
21
22     @Override
23     public Book findByIsbn(String isbn) {
24         return bookAdminRepository.findByIsbn(isbn).orElseThrow(() -> new ResourceNotFoundException(isbn));
25     }
26 }

```

En la capa de persistencia, también crearemos un paquete *common* donde ubicaremos nuestro *CustomRowMapper*, que servirá para reutilizar la lógica de mapeo entre los resultados de las consultas y los modelos de dominio en ambos controladores, de usuario y administrador.

Los mapeadores del administrador contendrán toda la información de la base de datos. Por ejemplo, el mapeador de Book se definirá de la siguiente manera:

```

1  @Override
2  public Book mapRow(ResultSet resultSet, int rowNum) throws SQLException {
3      Book book = new Book();
4      book.setId(resultSet.getLong("id"));
5      book.setIsbn(resultSet.getString("isbn"));
6      book.setTitleEs(resultSet.getString("title_es"));
7      book.setTitleEn(resultSet.getString("title_en"));
8      book.setSynopsisEs(resultSet.getString("synopsis_es"));
9      book.setSynopsisEn(resultSet.getString("synopsis_en"));
10     book.setPrice(new BigDecimal(resultSet.getString("books.price")));
11     book.setDiscount(resultSet.getFloat("books.discount"));
12     book.setCover(resultSet.getString("books.cover"));
13     if (this.existsColumn(resultSet, "publishers.id")) {
14         book.setPublisher(publisherRowMapper.mapRow(resultSet, rowNum));
15     }
16     if (this.existsColumn(resultSet, "categories.id")) {
17         book.setCategory(categoryRowMapper.mapRow(resultSet, rowNum));
18     }
19     return book;
20 }
21 }

```

Los repositorios del administrador serán muy similares a los del usuario, con la única diferencia de que utilizaremos los mapeadores correspondientes para convertir los *ResultSet* en modelos de la capa de dominio del administrador.

```

1  @Repository
2  @RequiredArgsConstructor
3  public class BookAdminRepositoryImpl implements BookAdminRepository {
4
5      private final JdbcTemplate jdbcTemplate;
6
7      private final AuthorAdminRepository authorAdminRepository;
8      private final GenreAdminRepository genreAdminRepository;
9
10     @Override
11     public List<Book> getAll() {
12         String sql = """
13             SELECT * FROM books
14             """;
15         return jdbcTemplate.query(sql, new BookRowMapper());
16     }
17
18     @Override
19     public List<Book> getAll(int page, int size) {
20         String sql = """
21             SELECT * FROM books
22             LIMIT ? OFFSET ?
23             """;
24         return jdbcTemplate.query(sql, new BookRowMapper(), size, page * size);
25     }
26
27     @Override
28     public int count() {
29         String sql = """
30             SELECT COUNT(*) FROM books
31             """;
32         return jdbcTemplate.queryForObject(sql, Integer.class);
33     }
34
35     @Override
36     public Optional<Book> findByIsbn(String isbn) {
37         String sql = """
38             SELECT * FROM books
39             LEFT JOIN categories ON books.category_id = categories.id
40             LEFT JOIN publishers ON books.publisher_id = publishers.id
41             WHERE books.isbn = ?
42             """;
43         try {
44             Book book = jdbcTemplate.queryForObject(sql, new BookRowMapper(), isbn);
45             book.setAuthors(authorAdminRepository.getByIsbnBook(isbn));
46             book.setGenres(genreAdminRepository.getByIsbnBook(isbn));
47             return Optional.of(book);
48         } catch (Exception e) {
49             return Optional.empty();
50         }
51     }
52 }

```

Esta duplicidad de código en los repositorios será abordada cuando añadamos los DAOs, lo que permitirá unificar la lógica de acceso a datos en un solo lugar.

De esta forma, hemos logrado separar la lógica del administrador de la del usuario. Al implementar paquetes y componentes específicos para cada rol, garantizamos que ambos grupos utilicen casos de uso diferentes con modelos adaptados a sus necesidades. Los administradores tienen acceso a información más detallada y a funcionalidades específicas para la gestión de libros, mientras que los usuarios disfrutan de una experiencia más simplificada y centrada en la lectura. Esta separación no solo mejora la claridad del código, sino que también permite una mejor mantenibilidad y escalabilidad de la aplicación.

2024/10/13 08:49() · cesguiro

Repositorio

<https://github.com/cesguiro/dws-spring/tree/develop/dws-presentation> (<https://github.com/cesguiro/dws-spring/tree/develop/dws-presentation>)

clase/daw/dws/1eval/presentation3.txt Última modificación: 2025/10/05 10:52 por cesguiro

cesguiro



Excepto donde se indique lo contrario, el contenido de este wiki esta bajo la siguiente licencia:
CC Attribution-NonCommercial-Share Alike 4.0 International